

Computer Vision

2026. 1. 5

3C AI Campus
박 준 오

<http://bit.ly/49jZ8iz>

수업참여코드
rzmh76ev



Google Classroom

Computer Vision

수업 코드
rzmh76ev

새 공지사항

다시 올리기

곧 마감되는 과제
기한이 곧 돌아오는 과제 없음
모두 보기

Joonoh Park
AM 10:46

교안입니다. 다운로드 받으세요

0.딥러닝 선수지식.pdf

Computer Vision

댓글 추가

1) Machine Learning Intro



Driving a car
Driving a machine learning

1) Machine Learning Intro

Machine Learning

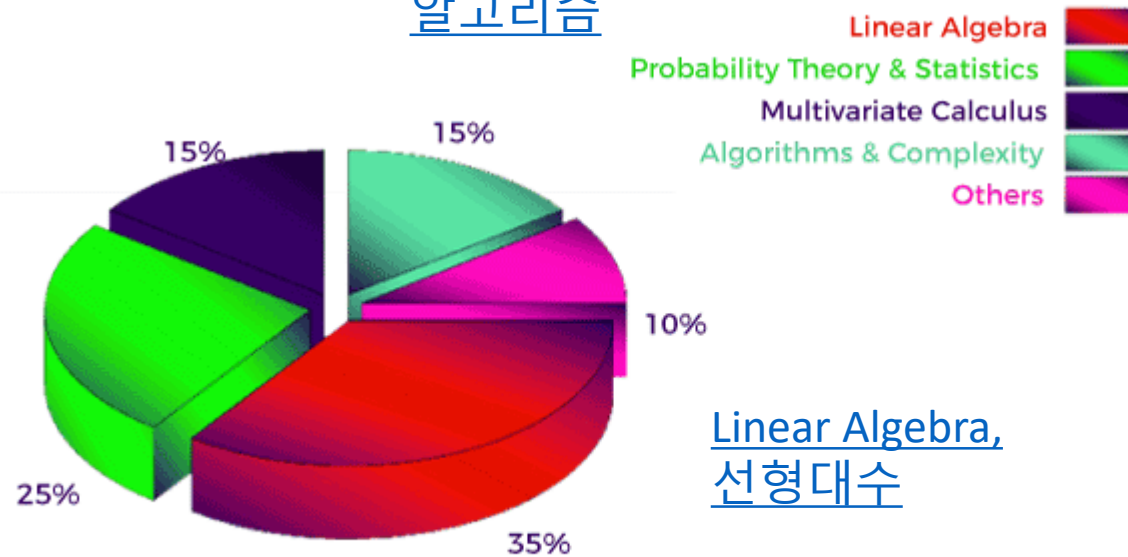
선수지식

1) Machine Learning Intro

Multivariable Calculus
다변수 미적분

Algorithms
알고리즘

Probability & Statistics
확률과 통계



Linear Algebra,
선형대수

Importance of Maths Topics Needed For Machine Learning

1) Machine Learning Intro Python

제1장 변수, 자료형, 연산자

1) 변수

변수는 값을 저장하는 메모리 공간을 가리키는 식별자입니다.
변수의 이름은 영문자, 숫자, 밑줄(_)로 구성되어야 하며, 대소문자를 구분합니다. (**예약어** 주의 !!)

2) 자료형

정수(int), 부동 소수점(float), 문자열(str), 불리언(boolean).
파이썬은 동적으로 타입을 결정.
변수에 어떤 값을 할당하느냐에 따라 자료형이 자동으로 결정.

3) 연산자

산술 연산자, 비교 연산자, 논리 연산자,
할당 연산자: 등호(=), 복합연산자(x +=1)

1) Machine Learning Intro Python

제2장 조건문과 반복문

1) 조건문

if, elif (else if의 줄임말), else 키워드를 사용하여 조건문을 작성

2) 반복문 (for, while)

for 문은 순차 자료형(sequence: 리스트, 튜플, 문자열 등) 순차적으로 실행

while 문은 주어진 조건이 참인 동안 코드 블록을 반복적으로 실행.

range() 함수

```
for i in range(5):  
    print(i)
```

range(stop) : 0 ~ stop-1

range(start, stop) : start ~ stop-1

range(start, stop, step) : start ~ stop-1 by step

1) Machine Learning Intro Python

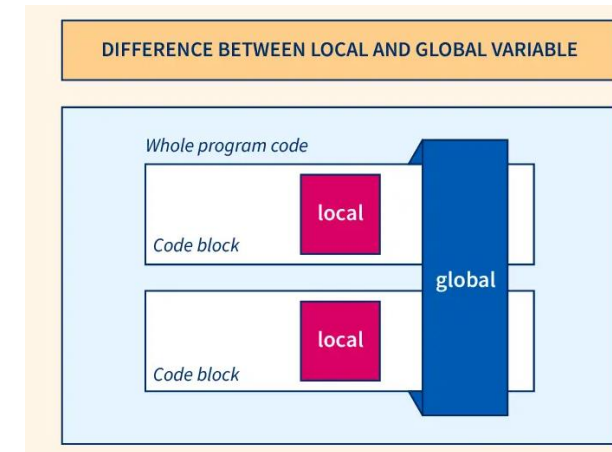
제3장 함수

1) 함수의 정의 : 메서드, 프로시저, 서브루틴, 루틴,

```
def 함수이름(매개변수):  
    함수내용  
    return 반환값
```

2) 지역변수(local variable) vs 전역변수(global variable)

3) parameter, argument, return



```
function sum(param1, param2){  
    return param1 + param2;  
}
```

Parameters

```
sum(5, 6);
```

Arguments

1) Machine Learning Intro **Python**

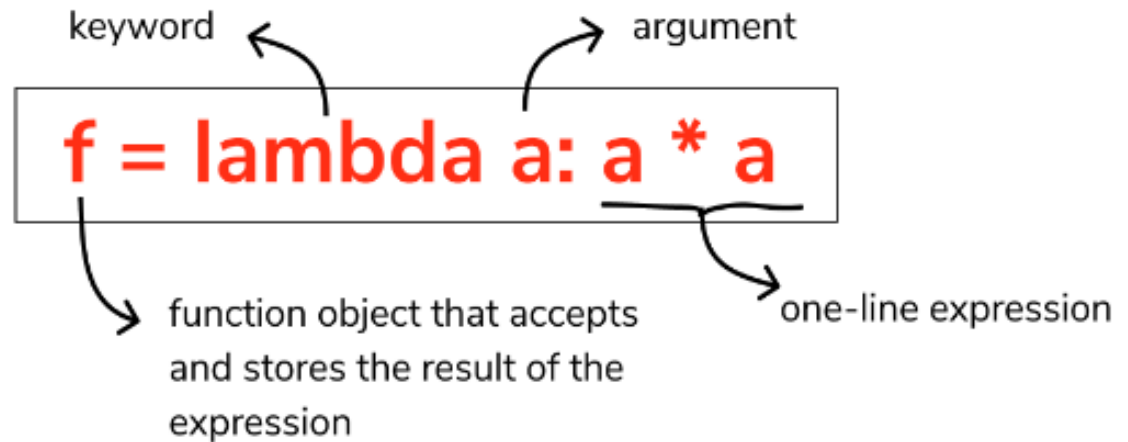
4) Lambda 함수

이름 없이 정의된 간단한 함수.
간단한 연산이나 함수를 인자로 전달할 때 사용.

```
# 일반 함수  
def add(a, b):  
    return a + b
```

```
# Lambda 함수  
add_lambda = lambda a, b: a + b
```

```
# 사용 예시  
print(add(3, 4)) # 출력: 7  
print(add_lambda(3, 4)) # 출력: 7
```



1) Machine Learning Intro Python

5) 콜백 함수(callback 함수)

다른 함수에 인자로 전달되는 함수.

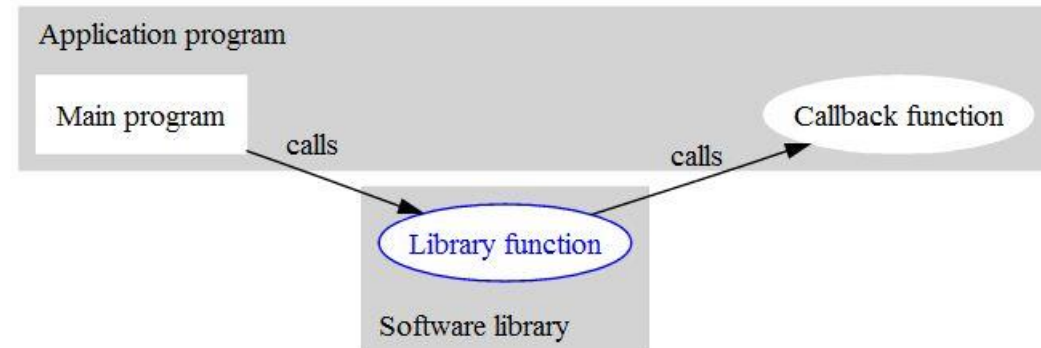
비동기처리, 이벤트 처리, 또는 다른 함수의 정의된 지점에서 호출될 때 사용.
특정 프레임워크/라이브러리에서 제공하는 경우에만 콜백 함수를 인자로 받을 수 있다

```
def greeting(name):  
    return "안녕, " + name
```

```
def farewell(name):  
    return name + ", 안녕히 가세요."
```

```
def handle_user(name, callback):  
    print(callback(name))
```

```
handle_user("영희", greeting) # 출력: 안녕, 영희  
handle_user("철수", farewell) # 출력: 철수, 안녕히 가세요.
```



1) Machine Learning Intro Python

6) *args (가변 위치 인자, 튜플 키워드 인자)

함수 정의에서 *args를 사용하면, 함수 호출 시 제공된 위치 기반 인자들을 튜플로 묶어서 처리.

목적: 함수를 호출할 때 몇 개의 위치 인자가 전달될지 미리 알 수 없을 때 유용.

예시:

```
def add_numbers(*args):  
    return sum(args)
```

```
print(add_numbers(1, 2, 3)) # 결과는 6  
print(add_numbers(1, 2, 3, 4, 5)) # 결과는 15
```

1) Machine Learning Intro Python

7) **kwargs (가변 키워드 인자, 딕셔너리 키워드 인자)

함수 정의에서 **kwargs를 사용하면, 함수 호출 시 제공된 키워드 인자들을 딕셔너리 형태(키값:밸류값)으로 묶어서 처리할 수 있다.

목적: 함수를 호출할 때 몇 개의 키워드 인자가 전달될지, 또 어떤 키워드 인자들이 전달될지 미리 알 수 없을 때 유용.

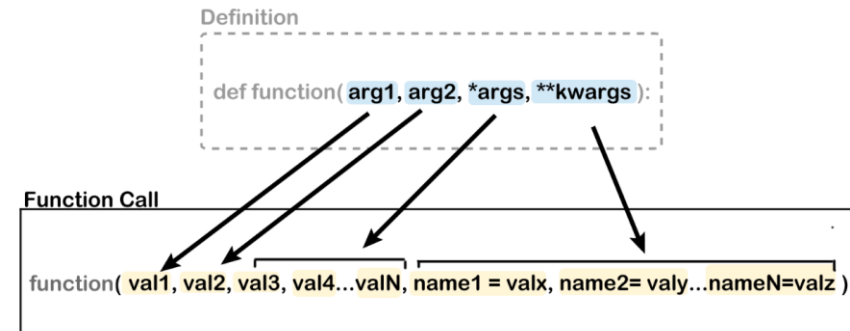
예시:

```
def print_info(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")
```

```
print_info(name="John", age=30, country="USA")
```

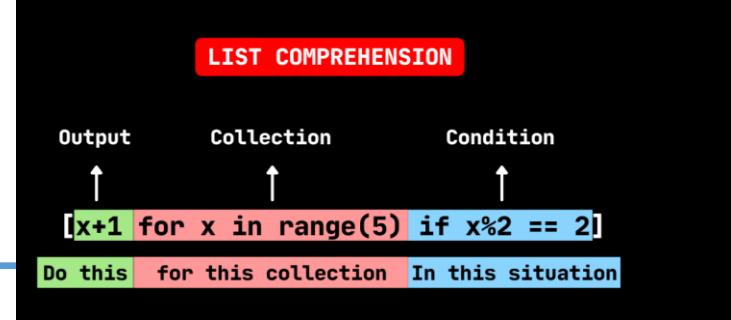


name: John age: 30 country: USA



**키워드 인자를 먼저 사용하고 위치 기반 인자를 뒤에 사용한다면 모호해지므로
위치 기반 인자를 먼저 사용하고 그 뒤에 키워드 인자를 사용해야 한다(Syntax error)**

1) Machine Learning Intro Python



제4장 리스트, 튜플, 딕셔너리 등의 자료구조

1) 리스트(List)

리스트는 여러 값을 순서대로 저장하는 자료구조.
리스트의 요소는 **변경 가능하며(mutable)**, 다양한 타입의 데이터를 포함할 수 있습니다.

일반적으로 데이터 포인트의 시퀀스나 특성(feature) 값을 저장할 때 사용됩니다.
특히, **동적으로 데이터를 추가하거나 삭제**해야 할 경우 유용

2) 튜플(Tuple)

튜플은 여러 값을 순서대로 저장하는 자료구조.
튜플의 요소는 **변경 불가능하며(immutable)**, 다양한 타입의 데이터를 포함할 수 있습니다. 튜플은 **고정된 값이 필요할 때** 사용됩니다.

1) Machine Learning Intro Python

Dictionary in Python

Dict={1: "Python", 2: "Java", 3: "C++"}

Keys		Value
1	→	Python
2	→	Java
3	→	C++

3) 딕셔너리(Dictionary)

딕셔너리는 **키(Key)**와 **값(Value)**의 쌍으로 데이터를 저장.
각 키는 고유해야 하며, 딕셔너리 내에서 값을 검색할 때 사용.
딕셔너리의 값은 변경 가능.

4) 배열(Array)

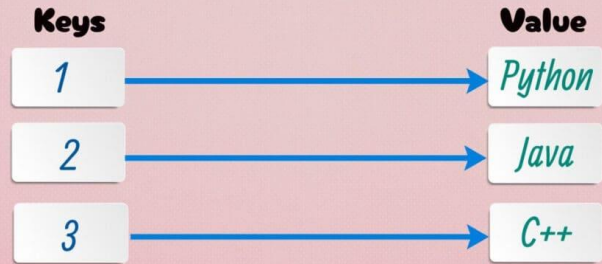
배열은 보통 NumPy 라이브러리를 통해 사용.

모든 요소가 같은 데이터 타입을 가져야. 이는 메모리 사용을 최적화하고 연산 속도를 빠르게 하는 데 도움. **크기가 고정되어** 있는 경우가 많다

1) Machine Learning Intro Python

Dictionary in Python

Dict = {1: "Python", 2: "Java", 3: "C++"}



5) 집합(Set)

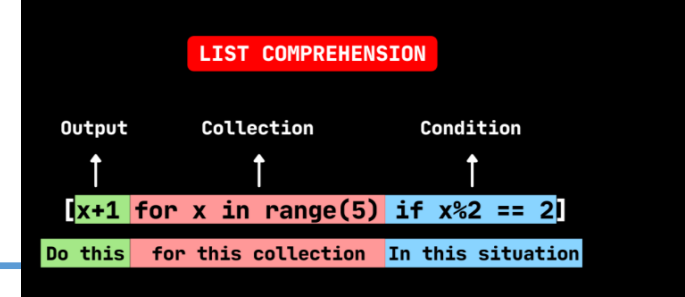
집합(set)은 중복된요소를 허용하지 않으며 순서가 없는 컬렉션.

집합을 사용하면 데이터의 중복제거, 데이터의 존재 여부를 빠르게 체크

1) Machine Learning Intro Python

항목	list (파이썬 기본 리스트)	numpy.ndarray (넘파이 배열)
정의	파이썬 내장 자료형	넘파이에서 제공하는 수치 계산용 자료형
구조	요소 타입이 달라도 가능 ([1, 'a', 3.5])	요소 타입이 모두 같아야 함 (int, float 등)
차원	중첩 리스트로 표현 ([[1,2],[3,4]])	다차원 배열 지원 (ndarray.shape)
연산 지원	수학 연산 불가 ([1,2] + [3,4] = [1,2,3,4])	수학 연산 가능 ([1,2] + [3,4] = [4,6])
브로드캐스팅	지원 안 함	지원함 (자동 크기 맞춤 연산)
슬라이싱	가능하나 느림	빠르고 유연한 슬라이싱
속도	일반적 계산에 적당	대규모 수치 연산에 훨씬 빠름
라이브러리 의존성	없음 (기본 내장)	import numpy as np 필요
딥러닝에서의 사용	거의 사용하지 않음	tensor 변환 이전 데이터 전처리에 사용 가능
대표 사용처	간단한 자료 저장, 일반 프로그래밍	벡터/행렬/수치 연산, 머신러닝/딥러닝

1) Machine Learning Intro Python



6) 리스트 컴프리헨션(List Comprehension)

조건문 등을 사용하여, 특정 조건을 만족하는 리스트를 쉽게 생성

[표현식 : for 항목 in 반복가능객체 if 조건문]

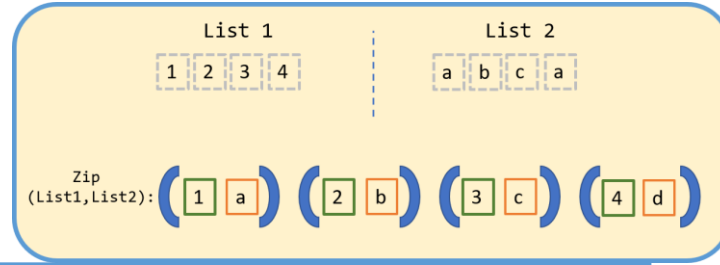
예시1)

```
numbers = [1, 2, 3, 4, 5]
squares = [n**2 for n in numbers]
print(squares) # [1, 4, 9, 16, 25]
```

예시2)

```
even_squares = [n**2 for n in numbers if n % 2 == 0]
print(even_squares) # [4, 16]
```

1) Machine Learning Intro Python



7) zip() 함수

여러 개의 iterable(예: 리스트, 튜플 등)을 **병렬로 묶어서**, 각 iterable의 동일한 인덱스에 있는 요소들을 **튜플로 묶어주는** 함수

예시 1) 리스트를 묶기

```
names = ["Alice", "Bob", "Charlie"]
```

```
ages = [25, 30, 35]
```

```
zipped = zip(names, ages)
```

```
print(list(zipped)) # 출력: [('Alice', 25), ('Bob', 30), ('Charlie', 35)]
```

예시 2) 세 개의 리스트를 묶기

```
names = ["Alice", "Bob", "Charlie"]
```

```
ages = [25, 30, 35]
```

```
cities = ["New York", "Los Angeles", "Chicago"]
```

```
zipped = zip(names, ages, cities)
```

```
print(list(zipped)) # 출력: [('Alice', 25, 'New York'), ('Bob', 30, 'Los Angeles'), ('Charlie', 35, 'Chicago')]
```

1) Machine Learning Intro **Python**

제5장 파일 및 데이터 처리(CSV/JSON)

1) CSV

- 파이썬 내장함수 사용한 csv파일 핸들링
- csv 라이브러리를 사용한 csv 파일 핸들링
- pandas 라이브러리를 사용한 csv 파일 핸들링

PYTHON CSV FILE

1. reader()
2. writer()
3. DictReader()
4. DictWriter()

```
import csv
f = open("data.csv", 'r')
row = csv.reader(f)
for i in row:
    print(i)
```

With examples and
Questions



1) Machine Learning Intro Python

2) JSON

```
{  
  "best_student": {  
    "name": "Ram",  
    "subject": "python",  
    "age": "24",  
    "marks": "98"  
  }  
}
```

JSON (JavaScript Object Notation) 파일은 데이터를 저장하고 교환하기 위한 경량의 데이터 포맷

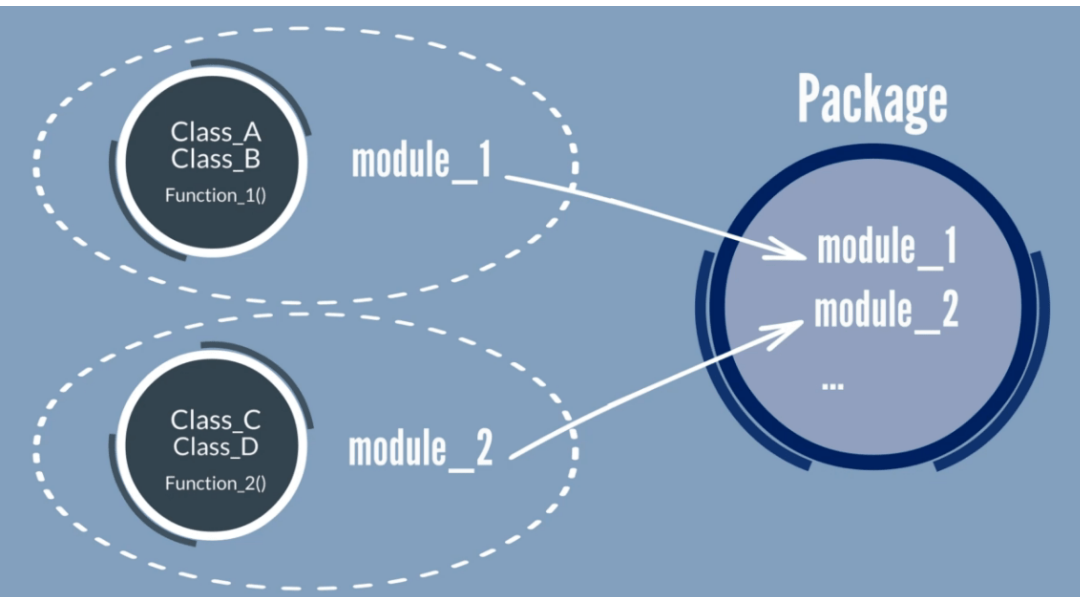
JSON 파일은 웹 애플리케이션에서 설정 파일, 데이터 전송 등 다양한 목적으로 널리 사용. 공공데이터 분석 등에도 널리 적용

```
import json  
data = { "name": "홍길동", "age": 25, "email": "hong@example.com"}  
with open('user.json', 'w', encoding='utf-8') as f:  
    json.dump(data, f, ensure_ascii=False, indent=4)
```

ensure_ascii=False: JSON 문자열을 ASCII가 아닌 유니코드로 저장합니다. 한글과 같은 비 ASCII 문자 저장하기 위해.
indent=4: JSON 데이터를 4칸 들여쓰기를 사용합니다.

1) Machine Learning Intro **Python**

제6장 클래스, 모듈, 패키지, 라이브러리



1) Machine Learning Intro Python

1. 클래스 작성: 기본 구조

- 클래스란?

특정 기능이나 데이터를 하나의 “설계도(blueprint)”로 묶은 사용자 정의 자료형

- 객체(Object) ?

클래스 설계를 바탕으로 실제로 만들어지는 “실행 가능한 실체”.

1) Machine Learning Intro Python

1. 클래스 작성: 기본 구조

```
class Animal:
    def __init__(self, name):
        self.name = name # 객체의 속성

    def speak(self):
        print(f"{self.name}이(가) 소리를 냅니다.")
```

```
dog = Animal("강아지")
dog.speak() # 출력: 강아지이(가) 소리를 냅니다.
```

- `__init__`: 객체가 생성될 때 자동 호출되는 "생성자"
- `self`: 이 클래스의 자기 자신
- `self.name`: 객체 내부에 저장되는 변수 (속성)

1) Machine Learning Intro Python

2. 클래스 상속: 기존 클래스를 확장

```
class Dog(Animal): # Animal 클래스를 상속받음
    def speak(self): # 부모 메서드 오버라이딩
        print(f"{self.name}이(가) 멍멍 짭니다.")
```

```
puppy = Dog("푸들")
puppy.speak() # 출력: 푸들이(가) 멍멍 짭니다.
```

개념	설명
Animal	부모 클래스 (superclass, base class)
Dog	자식 클래스 (subclass, derived class)
speak()	메서드 오버라이딩 (같은 이름, 다른 내용)
super()	부모 클래스의 메서드나 생성자를 호출할 때 사용

1) Machine Learning Intro Python

3. PyTorch의 클래스

PyTorch에서는 모델을 만들 때 항상 **nn.Module**을 상속받는 클래스를 정의

- 클래스는 설계도, 객체는 실제 물건

예를 들어, `class Cake:`은 케이크를 만드는 설계도

`my_cake = Cake()`는 그 설계도로 만든 진짜 케이크

클래스 = 모델 정의, 객체 = 학습에 쓸 모델 인스턴스

```
import torch
import torch.nn as nn

# 모델 정의 (클래스 생성)
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()    # 부모 클래스 초기화
        self.fc = nn.Linear(10, 1)     # 선형 레이어 하나 정의 (입력 10 → 출력 1)

    def forward(self, x):
        return self.fc(x)              # 순전파: 입력 x에 대해 계산 수행
```

1) Machine Learning Intro **Python**

요소	설명
nn.Module	PyTorch가 제공하는 모델 클래스의 부모 클래스
__init__()	모델의 레이어를 정의하는 곳
forward()	데이터가 들어왔을 때 계산 흐름을 정의하는 곳
self.	정의한 레이어들을 클래스 내부에서 공유할 수 있게 만듦

1) Machine Learning Intro **Python**

```
class Net(nn.Module):  
    |  
    |─ __init__(self):    ← 구조 정의 (레이어 구성)  
    |   |─ self.fc1  
    |   |─ self.relu  
    |   └─ self.fc2  
    |  
    └─ forward(self, x):  ← 입력 데이터가 통과하는 경로  
        └─ x → fc1 → ReLU → fc2 → 출력
```

1) Machine Learning Intro **Python**

구성 요소	설명
<code>__init__()</code>	모델의 레이어와 서브모듈을 정의
<code>forward()</code>	입력을 받아 출력을 반환하는 연산 흐름 정의
<code>parameters()</code>	학습 가능한 파라미터들을 반환
<code>children()</code> / <code>modules()</code>	포함된 레이어(모듈)들을 순회 가능
<code>register_buffer()</code>	파라미터는 아니지만 저장할 값 (예: 평균, 표준편차) 등록
<code>register_parameter()</code>	사용자 정의 파라미터를 추가
<code>.to()</code> , <code>.cuda()</code> , <code>.cpu()</code>	장치(GPU/CPU) 간 이동 지원
<code>.train()</code> / <code>.eval()</code>	학습 / 평가 모드 전환
<code>.state_dict()</code> / <code>load_state_dict()</code>	학습된 파라미터 저장 / 불러오기

1) Machine Learning Intro Python

제7장 예외처리, 반복자, 데코레이터, 동적 코드 실행

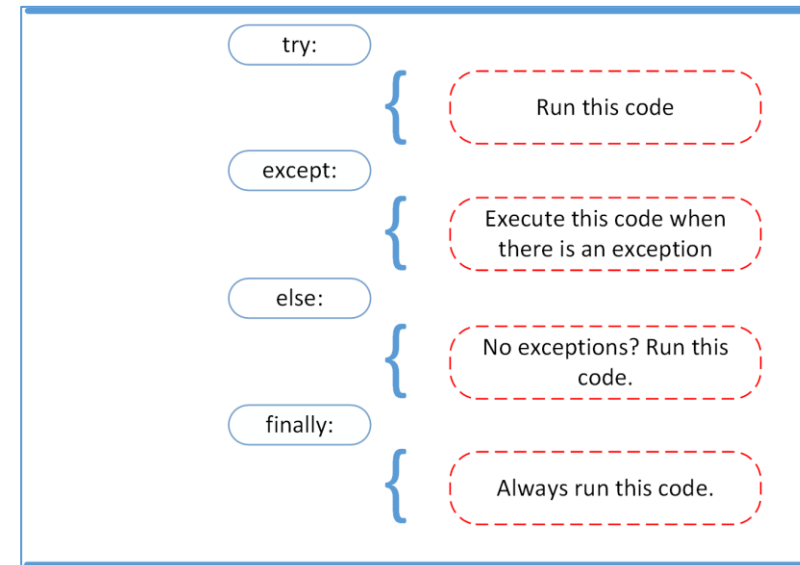
1) 예외 처리 (try ~ except ~ else ~ finally ~)

프로그램의 예기치 못한 오류나 예외 상황을 적절히 처리

프로그램이 강제 종료되지 않도록 하는 것이 예외 처리의 주된 목적

```
try: # 예외 발생 가능 코드
except 예외유형1: # 예외 처리 코드 1
except 예외유형2: # 예외 처리 코드 2
else: # 예외가 발생하지 않았을 때 실행될 코드
finally: # 예외 발생 여부와 상관없이 실행될 코드예시
```

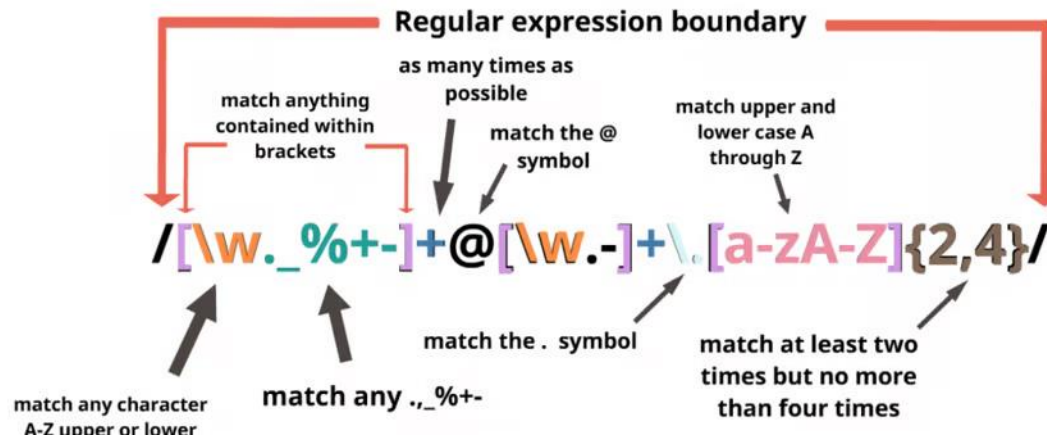
```
try:
    with open('example.txt', 'r') as file:
        content = file.read()
        print(content)
except FileNotFoundError:
    print("The file was not found.")
except IOError:
    print("An I/O error occurred.")
finally:
    print("Execution completed.")
```



1) Machine Learning Intro Python

제8장 정규표현식의 이해와 활용

정규 표현식(Regular Expression, Regex)은
문자열 패턴을 정의, 탐색, 매칭, 수정 등의 작업을 수행
(문자열, 시간표현, 정규표현식)



1) Machine Learning Intro Python

제 9 장 기타 : 머신러닝 학습용 리눅스 환경과 명령어

pip	Python installer package, 새로운 패키지를 설치/업데이트
wget	web get, 웹 상의 파일을 다운로드
tar	tape archive, 아카이브 파일(tar, gz, zip 등)을 압축 해제
conda	파이썬 가상환경과 패키지를 관리하는 도구
git	Git저장소에서 코드를 가져오거나 업데이트
mkdir	make directory
cp	copy, 파일이나 디렉터리를 복사
mv	move, 파일이나 디렉터리를 이동시키거나 이름을 변경
ls	list, 현재 디렉터리의 파일과 디렉터리 목록을 보여주는
rm	remove, 파일이나 디렉터리를 삭제
cat	concatenate, 파일의 내용을 터미널에 빠르게 출력
kill	프로세스를 강제 종료
pwd	print working directory, 현재 작업 디렉터리 출력
grep	파일 내용에서 특정 패턴을 검색
curl	client url, curl은 데이터를 전송하기 위한 명령어

1) Machine Learning Intro **Pandas**



1) Machine Learning Intro **Pandas**

1) 시리즈(Series).

1차원 배열과 유사한 자료 구조, **데이터와 인덱스(index)**가 함께 저장
각 데이터는 인덱스를 통해 접근할 수 있으며,
인덱스는 0부터 시작하는 숫자, 사용자가 원하는 값으로 설정가능

Series

	apples
0	3
1	2
2	0
3	1

+

Series

	oranges
0	0
1	3
2	7
3	2

=

DataFrame

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

1) Machine Learning Intro **Pandas**

2) 데이터 프레임(Data frame)

2차원 데이터 구조로, 행(row)과 열(column)로 구성.

데이터프레임은 엑셀 스프레드시트나 SQL 테이블과 비슷한 형태

각 열은 **시리즈(Series)** 객체로 구성되어 있으며, 행과 열 모두에 인덱스를 사용할 수 있습니다.

Series			Series			DataFrame		
	apples			oranges			apples	oranges
0	3		0	0		0	3	0
1	2	+	1	3	=	1	2	3
2	0		2	7		2	0	7
3	1		3	2		3	1	2

1) Machine Learning Intro **Pandas**

2) 데이터 프레임(Data frame)

(1)**리스트의 리스트**를 사용하여 데이터프레임 생성

```
import pandas as pd

data = [ [1, 'Alice', 25],
          [2, 'Bob', 30],
          [3, 'Charlie', 35] ]
df = pd.DataFrame(data, columns=['ID', 'Name', 'Age'])
print(df)
```

	ID	Name	Age
0	1	Alice	25
1	2	Bob	30
2	3	Charlie	35

1) Machine Learning Intro **Pandas**

2) 데이터 프레임(Data frame)

(2) **딕셔너리**를 사용하여 데이터프레임 생성

```
import pandas as pd
```

```
data = {  
    'ID': [1, 2, 3],  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35] }
```

```
df = pd.DataFrame(data)  
print(df)
```

	ID	Name	Age
0	1	Alice	25
1	2	Bob	30
2	3	Charlie	35

1) Machine Learning Intro **Pandas**

2) 데이터 프레임(Data frame)

데이터 접근 : 열(column) 접근

```
print(df['Name'])
```

출력:

0 Alice

1 Bob

2 Charlie

Name: Name,

dtype: object

1) Machine Learning Intro **Pandas**

2) 데이터 프레임(Data frame)

데이터 접근: 데이터 프레임(DataFrame)의 행을 접근하는 두 가지 방법
(loc, iloc)

`print(df.iloc[1])` # 인덱스를 사용하여 접근(**인덱스가 두번째인 행**)

출력:

```
ID    2
Name  Bob
Age   30
Name: 1, dtype: object
```

`Print(df.loc[1])` # 행 이름을 사용하여 접근(**행의 레이블이 1인 행**)

출력:

```
ID    2
Name  Bob
Age   30
Name: 1, dtype: object
```

특정 값 접근

`print(df.at[1, 'Name'])` # 특정 행과 열을 지정하여 접근

출력:

Bob

1) Machine Learning Intro **Pandas**

3) 기타 판다스의 주요 선수 지식.

데이터 선택 및 필터링

인덱싱(loc, iloc), **조건식 활용**, **누락값 처리** 등

데이터 정렬sort_values(), sort_index() 등을 이용한 정렬

데이터 그룹화와 집계groupby를 활용한 그룹 연산 (sum, mean 등)

데이터 병합merge, join 등을 활용한 데이터 결합**피벗 테이블**(Pivot Table)

데이터를 특정 변수를 기준으로 재구조화

누락값 처리dropna(), fillna() 등을 활용한 누락값 제거 및 대체

데이터 입출력CSV, Excel, SQL 등 다양한 데이터 소스로부터 데이터 불러오기/내보내기

데이터 전처리 : 범주형 데이터 처리 등 머신러닝을 위한 전처리

1) Machine Learning Intro

실습) 머신러닝의 효율적학습을 위해 먼저 이해해야 할 용어 50개를 뽑아서 공유하세요(난이도별 분야별 ...)

✓ 1. 기본 개념 (기초를 튼튼히)

1. 데이터셋 (Dataset)
2. 피쳐 (Feature)
3. 레이블 (Label)
4. 정답 (Ground Truth)
5. 모델 (Model)
6. 학습 (Training)
7. 추론 (Inference / Prediction)
8. 정확도 (Accuracy)
9. 손실 함수 (Loss Function)
10. 최적화 (Optimization)

✓ 2. 알고리즘 및 기법

11. 회귀 (Regression)
12. 분류 (Classification)
13. 클러스터링 (Clustering)
14. 지도학습 (Supervised Learning)
15. 비지도학습 (Unsupervised Learning)
16. 강화학습 (Reinforcement Learning)
17. k-최근접 이웃 (KNN)
18. 결정트리 (Decision Tree)
19. 랜덤 포레스트 (Random Forest)
20. 서포트 벡터 머신 (SVM)

1) Machine Learning Intro

✓ 3. 딥러닝 기초

- 21. 신경망 (Neural Network)
- 22. 퍼셉트론 (Perceptron)
- 23. 다층 퍼셉트론 (MLP)
- 24. 은닉층 (Hidden Layer)
- 25. 활성화 함수 (Activation Function)
- 26. ReLU / Sigmoid / Tanh
- 27. 경사 하강법 (Gradient Descent)
- 28. 오차 역전파 (Backpropagation)
- 29. 에폭 (Epoch)
- 30. 배치 사이즈 (Batch Size)

✓ 4. 성능 및 과적합 관련 용어

- 31. 과적합 (Overfitting)
- 32. 과소적합 (Underfitting)
- 33. 정규화 (Regularization)
- 34. 드롭아웃 (Dropout)
- 35. 교차검증 (Cross Validation)
- 36. 검증셋 (Validation Set)
- 37. 테스트셋 (Test Set)
- 38. 바이어스 (Bias)
- 39. 분산 (Variance)
- 40. 하이퍼파라미터 (Hyperparameter)

1) Machine Learning Intro

✓ 5. 실전 및 활용 관련 용어

- 41. 파라미터 튜닝 (Parameter Tuning)
- 42. 전이 학습 (Transfer Learning)
- 43. 파인튜닝 (Fine-tuning)
- 44. 피쳐 스케일링 (Feature Scaling)
- 45. 표준화 / 정규화 (Standardization / Normalization)
- 46. 차원 축소 (Dimensionality Reduction)
- 47. PCA (주성분 분석)
- 48. t-SNE (고차원 시각화 기법)
- 49. 혼동 행렬 (Confusion Matrix)
- 50. ROC 곡선 / AUC

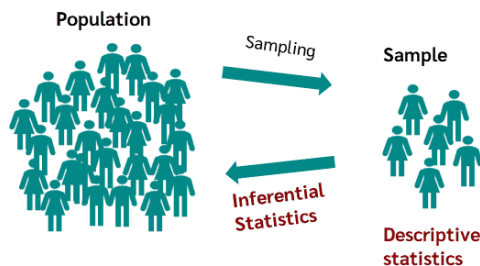
1) Machine Learning Intro 통계 기초

1) Machine Learning Intro

통계 기초

1) Machine Learning Intro 통계 기초

- **기술통계학**(descriptive): 수집된 데이터 자체를 요약하고 설명하는 것에 초점
데이터의 중심 경향과 변동성을 파악하기 위한 기본 통계 개념
평균, 중간값, 분산, 표준편차 등을 통해 데이터의 특성을 이해할 수 있다.
- **확률통계학**(Inferential): 수집된 데이터로 모집단(전체 데이터)을 추론하는 것이 목적
데이터가 어떻게 분포되어 있는지를 나타내며,
정규분포와 이항분포.



1) Machine Learning Intro 통계 기초

기술통계학:

평균 (Mean): 데이터 세트의 모든 값의 합을 데이터의 개수로 나눈 값.

예시: 주어진 데이터 [2, 4, 6, 8, 10]의 평균은 $(2+4+6+8+10)/5 = 6$.

머신러닝 사용 예시:

평균은 데이터의 중심 경향을 나타내며,
데이터의 특성을 이해하고 이상치를 감지하는 데 사용됩니다.

중간값 (Median): 데이터 세트를 크기 순으로 정렬했을 때 가운데 위치한 값.

예시: 데이터 [2, 4, 6, 8, 10]의 중간값은 6입니다. 데이터 [2, 4, 6, 8, 10, 12]의 중간값은 $(6+8)/2 = 7$.

머신러닝 사용 예시: 중간값은 데이터의 분포를 이해하고 이상치에 영향을 받지 않는 중심 경향을 파악하는 데 유용.

1) Machine Learning Intro 통계 기초

기술통계학:

분산 (Variance): 데이터의 각 값이 평균으로부터 얼마나 떨어져 있는지

예시: 데이터 [2, 4, 6, 8, 10]의 분산은 $[(2-6)^2 + (4-6)^2 + (6-6)^2 + (8-6)^2 + (10-6)^2]/5 = 8$.

머신러닝 사용 예시: 분산은 데이터의 변동성,
모델의 성능 평가와 특성의 중요도 분석에 사용.

표준편차 (Standard Deviation):

설명: 분산의 제곱근으로, 데이터의 분포를 나타내는 값.

예시: 데이터 [2, 4, 6, 8, 10]의 표준편차는 $\sqrt{8} \approx 2.83$ 입니다.

머신러닝 사용 예시: 표준편차는 데이터의 변동성을 직관적으로 이해하는 데 사용되며, 데이터 전처리 및 정규화 과정에서 활용.

1) Machine Learning Intro **통계 기초**

확률 통계학

정규분포 (Normal Distribution):

설명: 데이터가 평균을 중심으로 대칭적으로 분포하는 종 모양의 분포.
평균과 표준편차에 의해 정의됩니다.

- **연속 확률 분포**(continuous).

예시: 사람의 키, 시험 점수 등 많은 자연 현상과 사회적 현상이 정규분포를 따릅니다.

자연 현상에서의 불확실성과 변동성 때문에 발생하며,
중앙에 몰려 있고 양쪽으로 갈수록 희박해지는 패턴을 보입니다.(중심극한 정리)

머신러닝 사용 예시:

정규분포는 가우시안 나이브 베이즈 분류기, 정규화, PCA 등 에 사용.

데이터가 정규분포를 따를 때,

평균을 중심으로 데이터의 대다수가 존재하며, 이를 통해 예측과 분석이 용이.

1) Machine Learning Intro 통계 기초

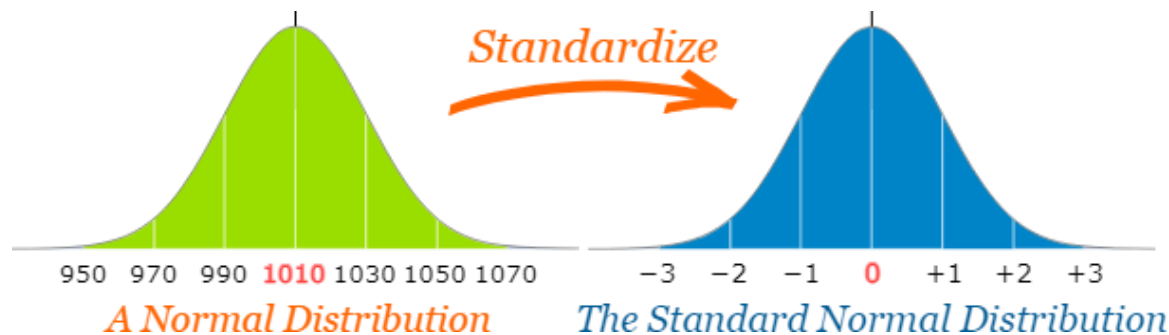
표준 정규 분포(Standard Normal Distribution)

기존 정규분포에서 평균을 빼고 표준편차로 나눈 결과의 분포로
평균이 0이고, 분산이 1인 정규 분포

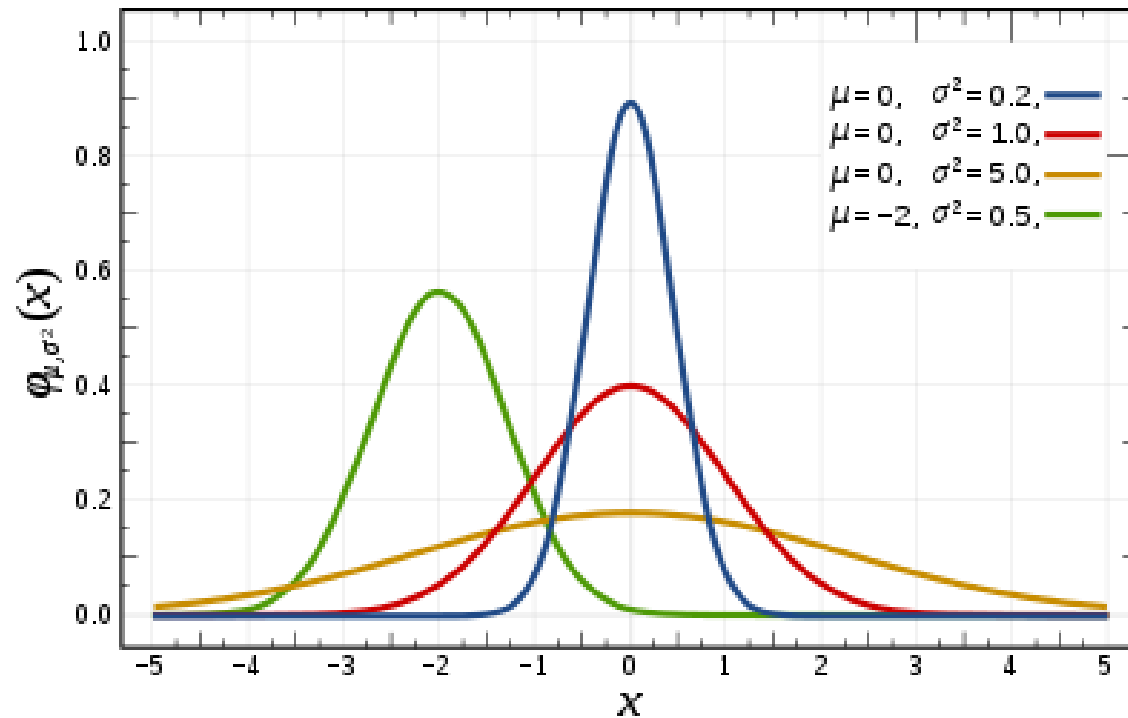
관측값이 평균에서 몇배의 표준편차만큼 떨어져 분포하는지를 알 수 있다.

표준 정규분포로 데이터를 변환하는 이유는 모델이 더 빠르고 안정적으로 학습
StandardScaler()

$$Z = \frac{X - \mu}{\sigma}$$



1) Machine Learning Intro 통계 기초



첨도(kurtosis)와 왜도(skewness)

1) Machine Learning Intro 통계 기초



Min-Max 정규화 vs 표준화(Standardization)

표준화 (Standardization)

- 표준화는 데이터의 값을 **평균이 0, 표준편차가 1이 되도록 변환.**
- 데이터가 정규분포에 가깝게 변환되며, **이상치의 영향을 상대적으로 덜 받는다.**
- 다양한 분포를 가지는 데이터를 동일한 기준으로 비교.
- 선형 회귀, 로지스틱 회귀: 모델의 가중치가 큰 값을 갖지 않도록 하여 수렴을 빠르게
- PCA(주성분 분석): 데이터를 표준화하여 각 주성분이 동일한 중요도를 가지도록

표준화

```
scaler = StandardScaler()
```

```
standardized_data = scaler.fit_transform(data)
```

```
print("표준화된 데이터:\n", standardized_data)
```

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

1) Machine Learning Intro 통계 기초

Min-Max 정규화 (Normalization)

- Min-Max 정규화는 데이터의 값을 특정 범위(보통 0과 1)로 변환
- 데이터의 분포가 동일한 스케일로 조정되어 각 특성이 동일한 중요도를 가지게 된다.
- 이상치(Outliers)가 존재할 경우 정규화된 값들이 왜곡될 수 있다.

• Min-Max 정규화에서 최대값으로 나누지 않고 최소값을 고려하는 이유

데이터를 [0, 1] 범위로 정확하게 변환하고,
모든 값이 min부터 얼마나 떨어져 있는지를 기준으로 상대적 크기를 보존합니다.

- 이미지 처리: 픽셀 값의 범위가 0에서 255인 경우, 이를 0에서 1 사이로 정규화하여 모델의 학습을 용이하게.
- 신경망 입력값: 각 입력값의 범위를 0과 1 사이로 정규화하여 학습 속도를 높이고, 수렴을 안정화.

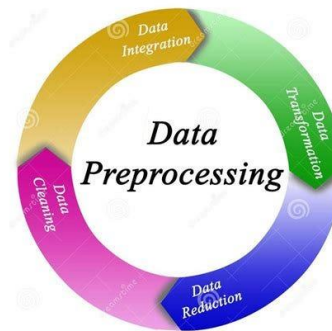
Min-Max 정규화

```
scaler = MinMaxScaler()
```

```
normalized_data = scaler.fit_transform(data)
```

1) Machine Learning Intro 데이터 전처리 기초

1) Machine Learning Intro 데이터 전처리 기초



1) Machine Learning Intro 데이터 전처리 기초

데이터 전처리(Data Preprocessing)

머신러닝 모델을 학습시키기 전에 데이터를 준비하고 정리하는 과정
데이터를 분석하고 모델의 성능을 최적화하기 위해 필수적

결측치 (Missing Values) : 데이터셋에 누락된 값이 있을 때 이를 처리하는 방법,
제거: 결측치가 있는 행 또는 열을 삭제.
대체: 결측치를 평균, 중간값, 최빈값 등으로 대체

데이터 정규화 (Normalization) : 데이터의 값을 일정한 범위로 변환
Min-Max 정규화를 사용하여 데이터를 $[0, 1]$ 범위로 변환

표준화 (Standardization) : 데이터의 평균을 0, 표준편차를 1로 변환

Feature Engineering : 모델의 성능을 향상시키기 위해 새로운 특성(Feature)을 만드는 과정

1) Machine Learning Intro 데이터 전처리 기초

차원 축소 (Dimensionality Reduction)

데이터의 특성 수를 줄여서 모델의 성능을 향상시키고 계산 효율성을 높이는 방법.
주성분 분석(PCA) 등을 사용하여 차원을 축소

라벨 인코딩 (Label Encoding)

범주형 데이터를 숫자로 변환하는 과정.

원-핫 인코딩 (One-Hot Encoding)

범주형 데이터를
해당 범주에만 1을, 나머지에는 0을 입력

데이터 분할 (Train-Test Split)

모델의 학습과 평가를 위해 데이터를 훈련 세트와 테스트 세트로 나누는 과정

Label Encoding

Island
0
0
1
1
2
2

```
le = LabelEncoder()  
le.fit_transform(df["island"])
```

Original

Island
Torgersen
Torgersen
Biscoe
Biscoe
Dream
Dream

```
df["Island"].unique()  
['Torgersen', 'Biscoe', 'Dream']
```

One-Hot Encoding

Torgersen	Biscoe	Dream
1	0	0
1	0	0
0	1	0
0	1	0
0	0	1
0	0	1

```
add_columns = pd.get_dummies(df["island"])  
df = df.join(add_columns)
```

1) Machine Learning Intro 실습 1)

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

# 데이터 생성
data = {'Age': [25, 30, None, 24, 29], 'Salary': [50000, 54000, 58000, None, 62000], 'Category': ['A', 'B', 'A', 'B', 'C']}
df = pd.DataFrame(data)

# 결측치 처리
df['Age'].fillna(df['Age'].mean(), inplace=True)
df.dropna(subset=['Salary'], inplace=True)

# 데이터 정규화
scaler = MinMaxScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])

# 표준화
scaler = StandardScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])

# 특성 공학
df['Age_Salary_Ratio'] = df['Age'] / df['Salary']

# 차원 축소
pca = PCA(n_components=2)
reduced_data = pca.fit_transform(df[['Age', 'Salary']])

# 라벨 인코딩
encoder = LabelEncoder()
df['Category'] = encoder.fit_transform(df['Category'])

# 원-핫 인코딩
df = pd.get_dummies(df, columns=['Category'])

# 데이터 분할
X_train, X_test, y_train, y_test = train_test_split(df[['Age', 'Salary']], df['Age_Salary_Ratio'], test_size=0.2, random_state=42)

print(df)
print("Reduced data:", reduced_data)
print("Train set:", X_train, y_train)
print("Test set:", X_test, y_test)
```

	Age	Salary	Age_Salary_Ratio	Category_0	Category_1	Category_2
0	-1.432078	-1.341641	1.067408	True	False	False
1	1.171700	-0.447214	-2.620001	False	True	False
2	-0.390567	0.447214	-0.873334	True	False	False
4	0.650945	1.341641	0.485185	False	False	True

Reduced data: [[1.96131538 -0.06394878]
[-0.51228939 1.14474492]
[-0.04005538 -0.59240015]
[-1.40897061 -0.48839599]]

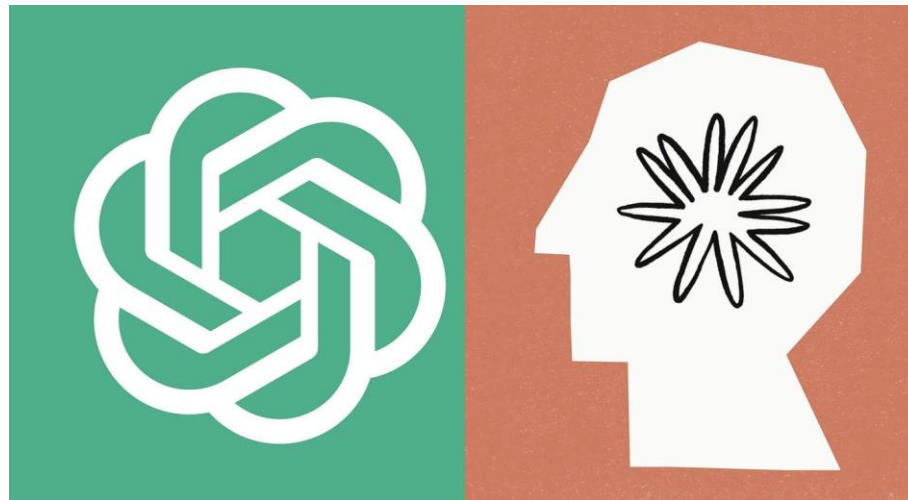
Train set: Age Salary
4 0.650945 1.341641
0 -1.432078 -1.341641
2 -0.390567 0.447214 4 0.485185
0 1.067408
2 -0.873334
Name: Age_Salary_Ratio, dtype: float64

Test set: Age Salary
1 1.1717 -0.447214 1 -2.620001
Name: Age_Salary_Ratio, dtype: float64

1) Machine Learning Intro 실습 2)

실습 1) 앞서 배운 내용을 바탕으로 예시코드의 해석과 오류를 분석해 보세요.

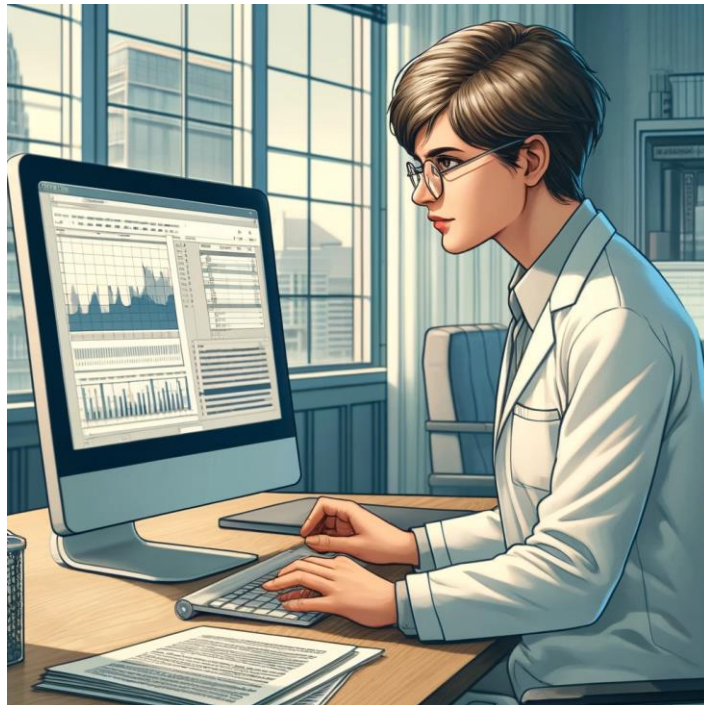
실습 2) 앞서 배운 내용을 바탕으로 예시코드를 생성하여 코드별 해설, 결과값의 해석을 코멘트하여 공유해 봅시다.



1) Machine Learning Intro 데이터 전처리 기초

탐색적 데이터 분석(Exploratory Data Analysis, EDA)

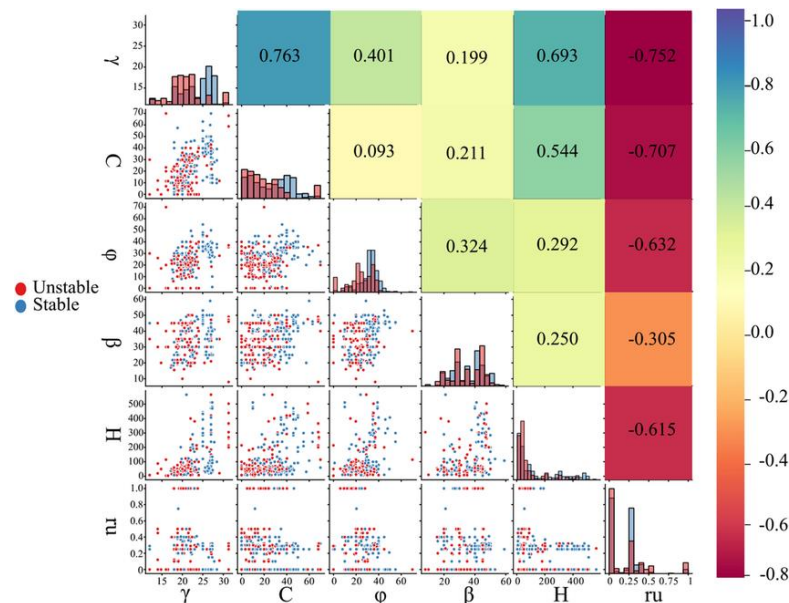
데이터를 시각적으로 분석하고 통계적으로 요약하여 주요 특징, 패턴, 이상치, 가설을 이해하는 과정



1) Machine Learning Intro 데이터 전처리 기초

데이터 시각화 (Data Visualization)

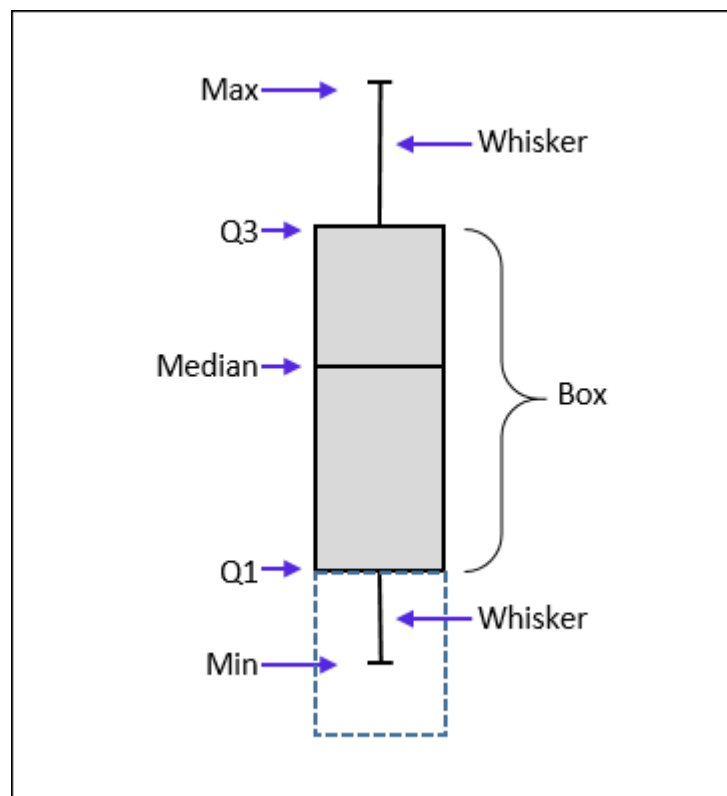
- 설명: 데이터를 그래프로 시각적으로 표현하여 이해하는 과정.
- 주요 용어:
 - 히스토그램 (Histogram): 데이터의 분포를 보여주는 막대 그래프.
 - 박스플롯 (Box Plot): 데이터의 분포와 이상치를 시각적으로 표현.
 - 산점도 (Scatter Plot): 두 변수 간의 관계를 보여주는 점 그래프.
 - 히트맵 (Heatmap): 변수 간의 상관 관계를 색상으로 표현.



1) Machine Learning Intro 데이터 전처리 기초

데이터 시각화 (Data Visualization)

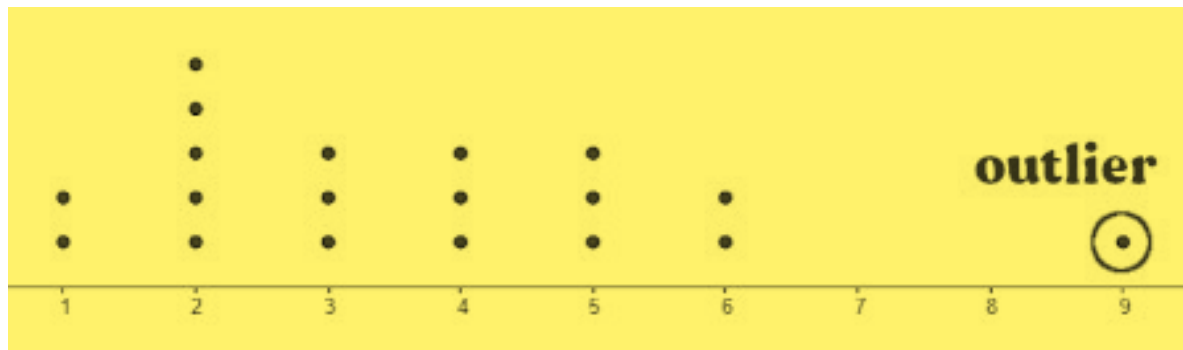
- 박스플롯 (Box Plot): 데이터의 분포와 이상치를 시각적으로 표현.



1) Machine Learning Intro 데이터 전처리 기초

이상치 탐지 (Outlier Detection)

- 설명: 데이터 내에서 다른 값들과 크게 벗어난 값을 찾는 과정.
- 주요 용어:
 - 이상치 (Outliers): 데이터의 다른 값들과 비교하여 현저히 벗어난 값들.



1) Machine Learning Intro 데이터 전처리 기초

탐색적 데이터 분석(Exploratory Data Analysis, EDA)

데이터를 시각적 분석, 통계적 요약으로 주요 특징, 패턴, 이상치, 가설을 이해하는 과정

기술통계량 (Descriptive Statistics) : 데이터의 주요 특성을 요약하여 보여주는 통계량.

주요 측정 지표:

- 평균 (Mean): 데이터의 평균 값.
- 중앙값 (Median): 데이터의 중간 값.
- 분산 (Variance): 데이터 값들이 평균으로부터 얼마나 떨어져 있는지 나타내는 지표.
- 표준편차 (Standard Deviation): 분산의 제곱근, 데이터의 변동성을 나타냅니다.
- 최소값 (Min) / 최대값 (Max): 데이터의 최소값과 최대값.
- 사분위수 (Quartiles): 데이터를 네 부분으로 나눈 값들.

1) Machine Learning 코랩

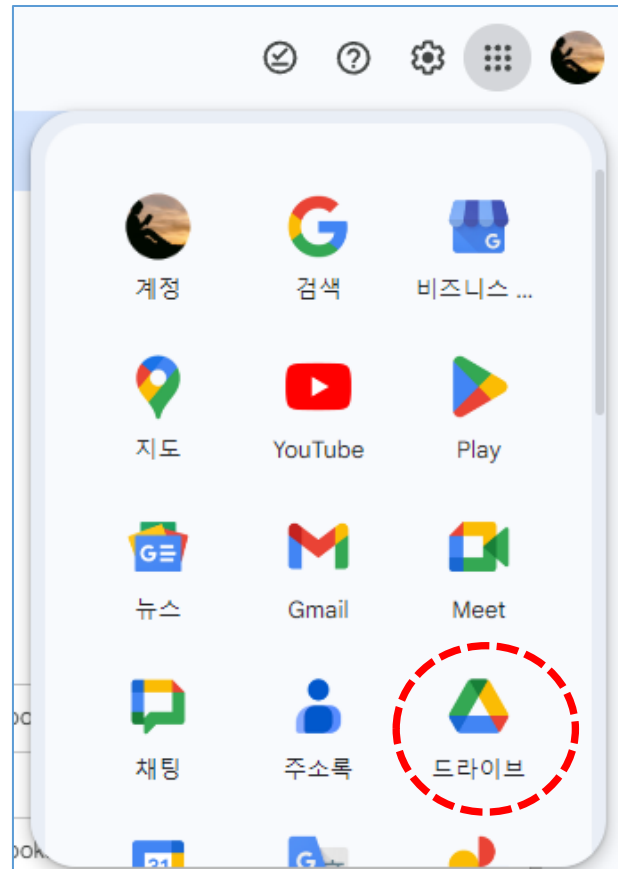


Google Colaboratory

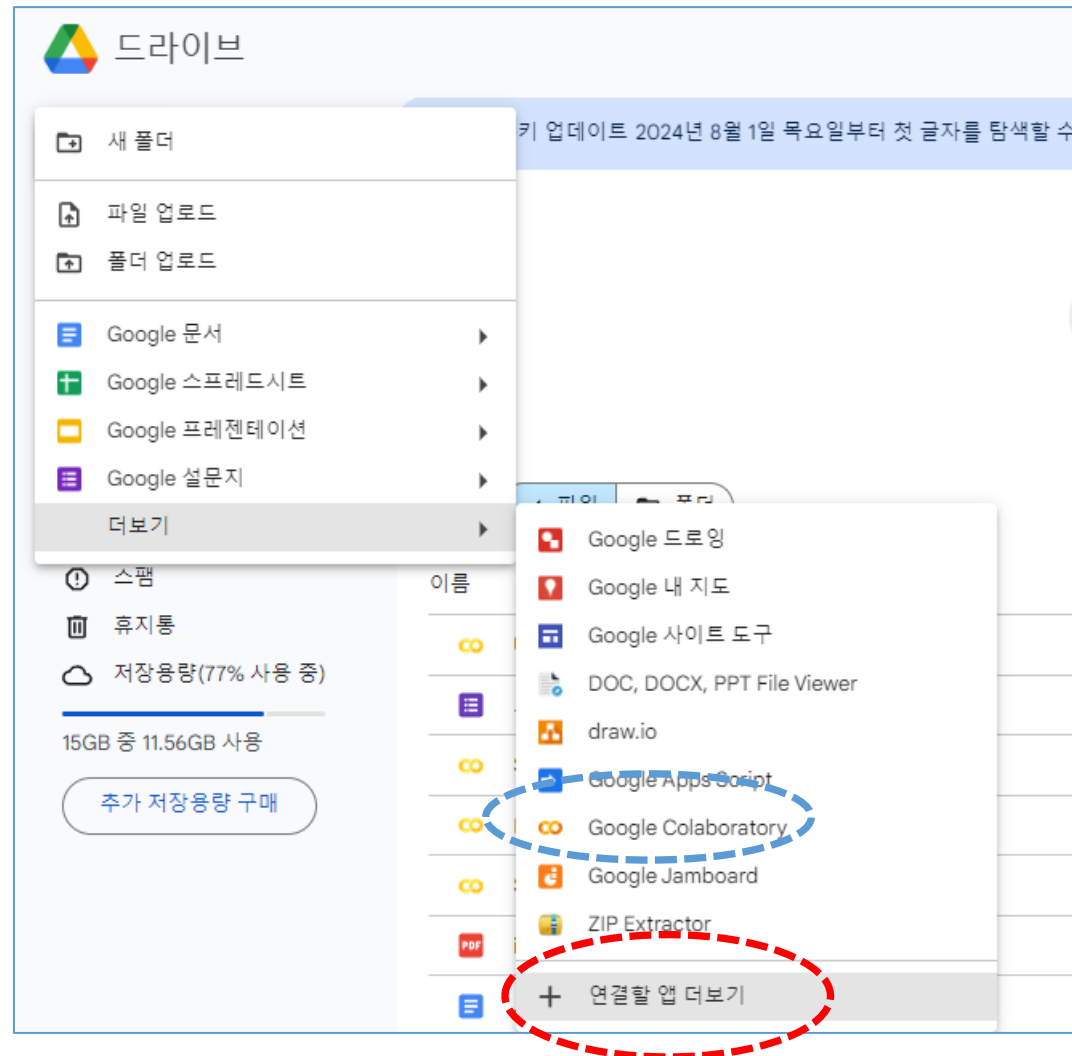
1) Machine Learning 코랩

크롬의 구글 홈에서 구글 드라이브를 선택 !

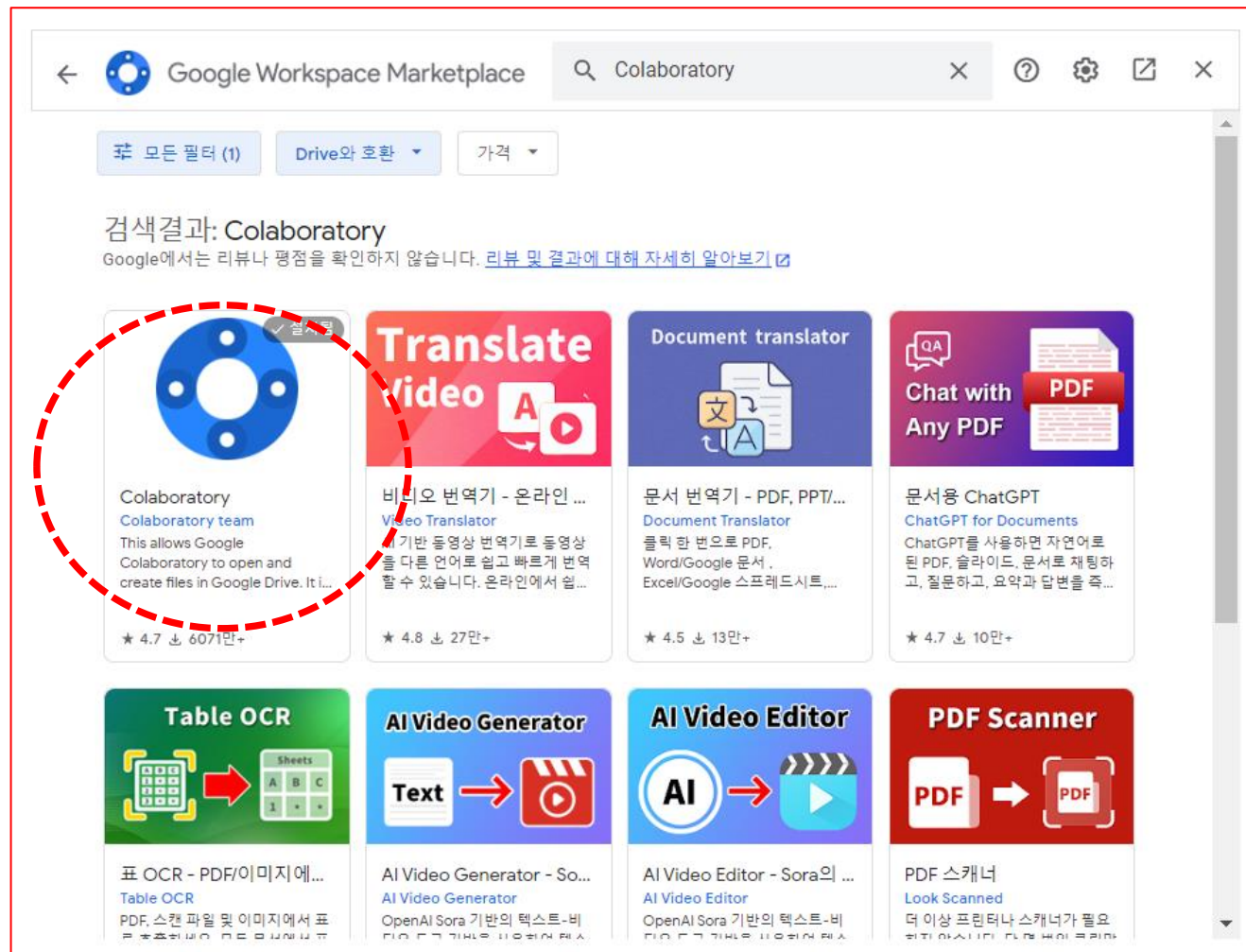
정상적인 사용을 위해선 크롬의 정상 로그인 필요 !!



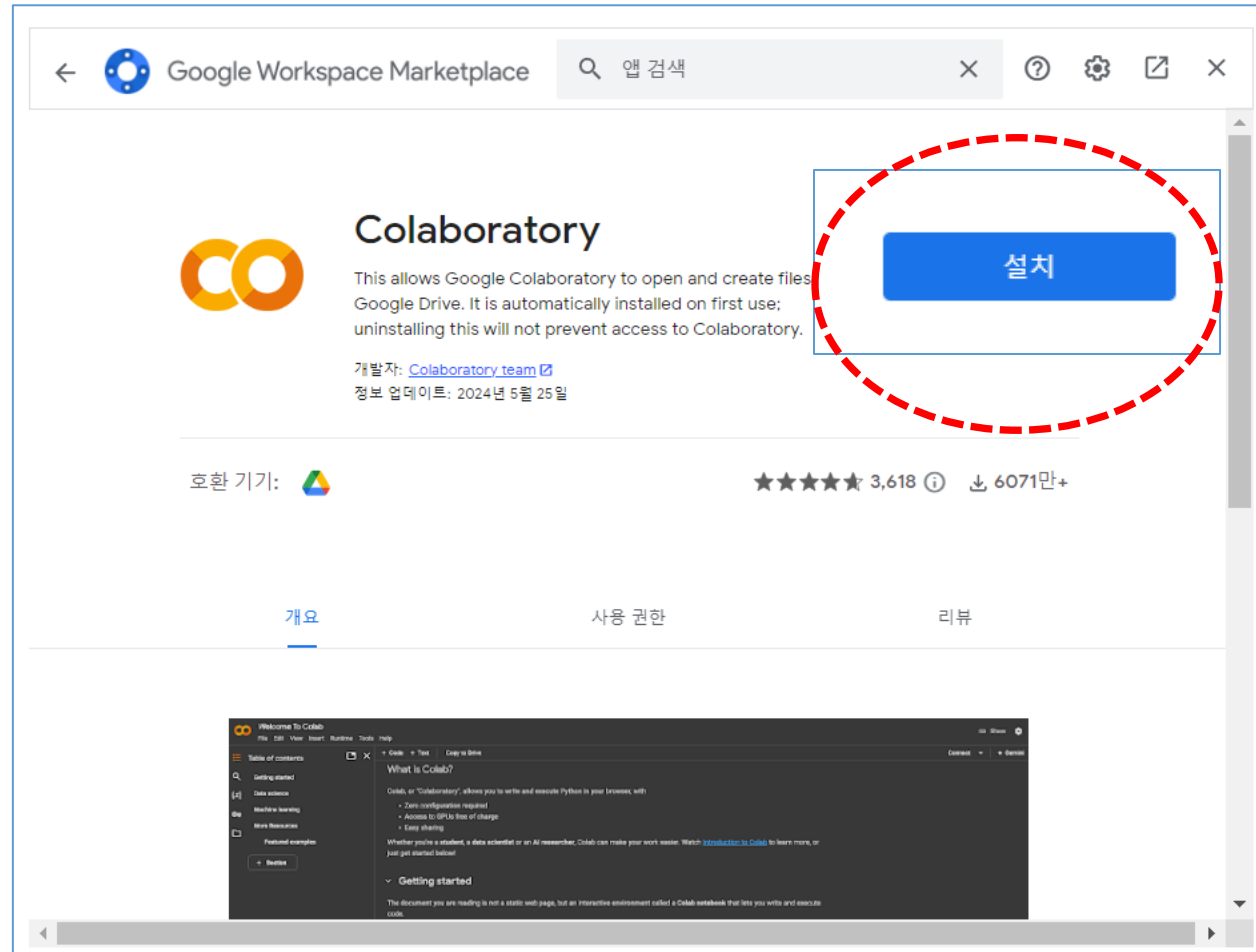
1) Machine Learning 코랩



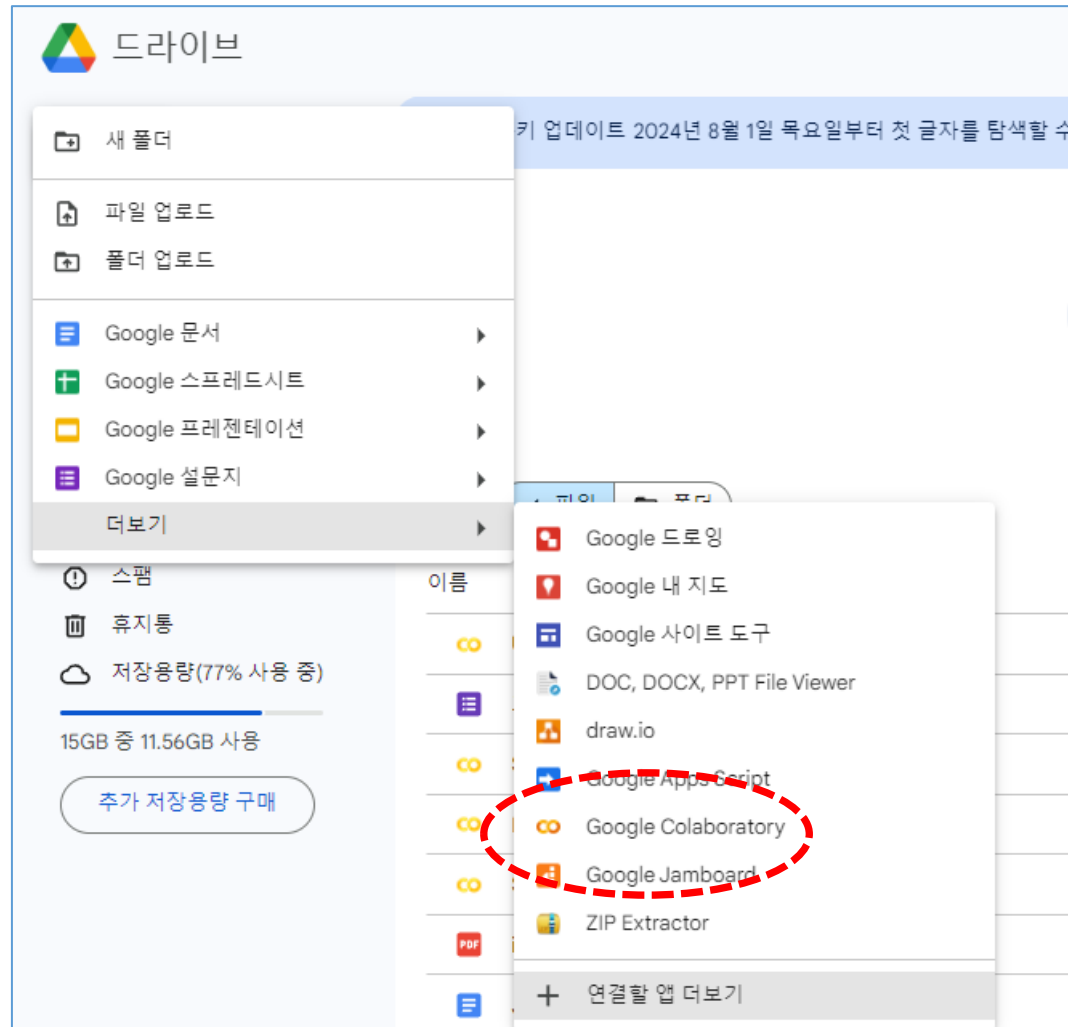
1) Machine Learning 코랩



1) Machine Learning 코랩



1) Machine Learning 코랩



1) Machine Learning 코랩

검색창에서 코랩을 검색하여 접속

정상적인 사용을 위해선 크롬의 정상 로그인이 필요 !!

Colaboratory에 오신 것을 환영합니다

파일 수정 보기 삽입 런타임 도구 도움말

목차

- 시작하기
- 데이터 과학
- 머신러닝
- 추가 리소스
- 추천 예시

+ 섹션

Colab 시작 페이지

(신규) Gemini API를 사용해 보세요

- [Generate a Gemini API key](#)
- [Talk to Gemini with the Speech-to-Text API](#)
- [Gemini API: Quickstart with Python](#)
- [Gemini API code sample](#)
- [Compare Gemini with ChatGPT](#)
- [More notebooks](#)

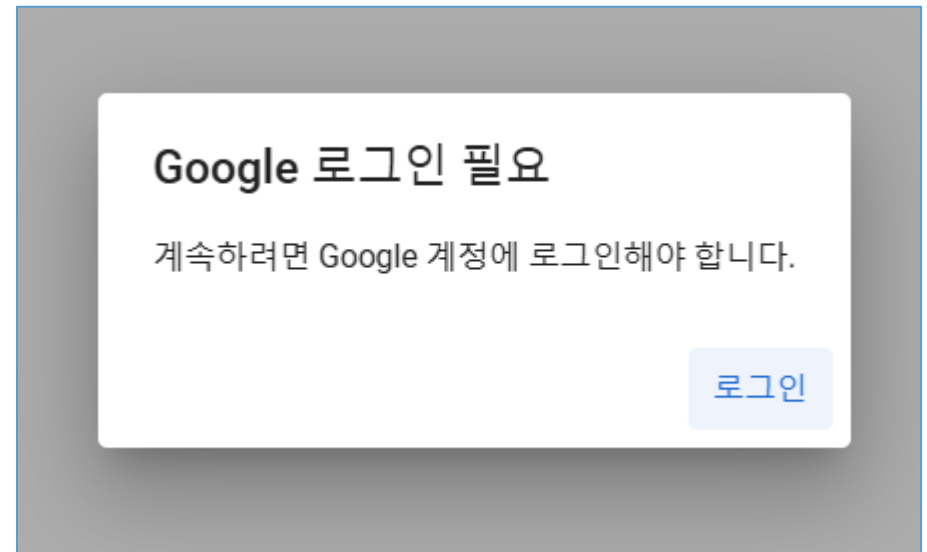
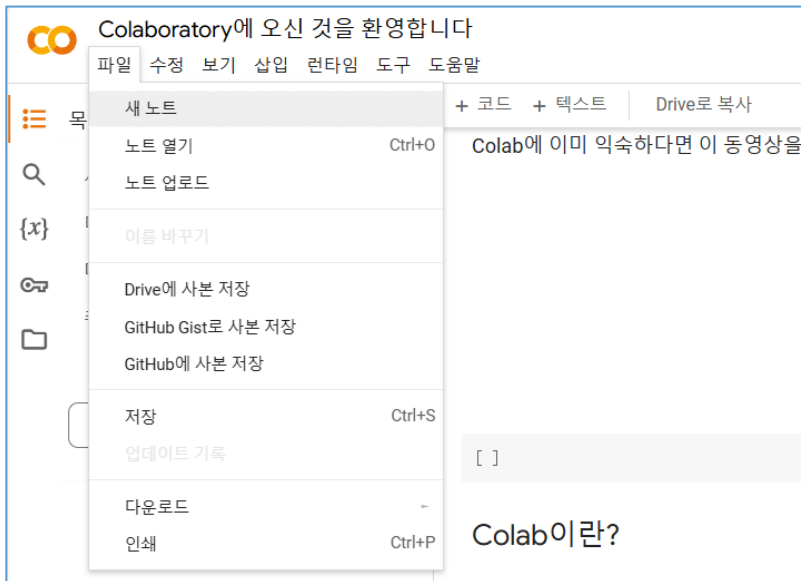
Colab에 이미 익숙하다면 이 동영상상을 통해 양방향 테이블, 코드 실행 기록 보기, 명령어 팔레트에 관해 알아보세요.

3 Cool Google Colab Features

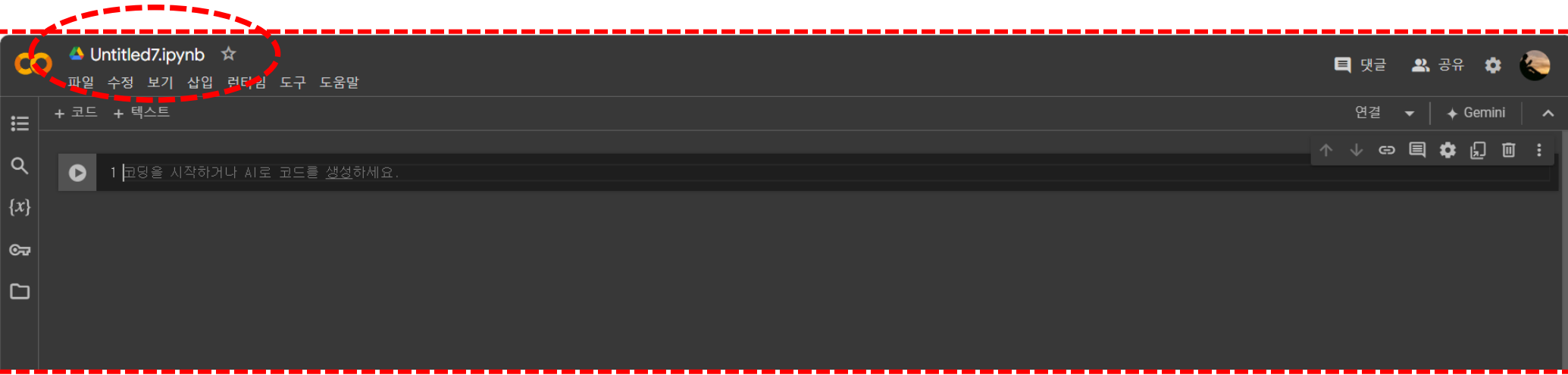
[]

1) Machine Learning 코랩

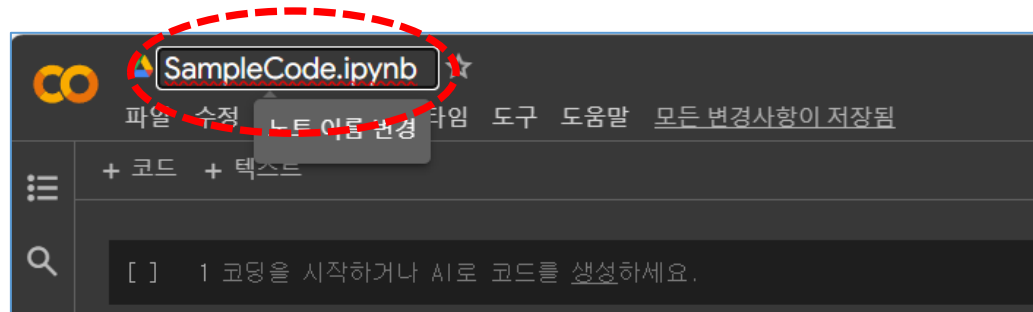
새 노트 열기/ 노트 열기/ 노트 업로드



1) Machine Learning 코랩

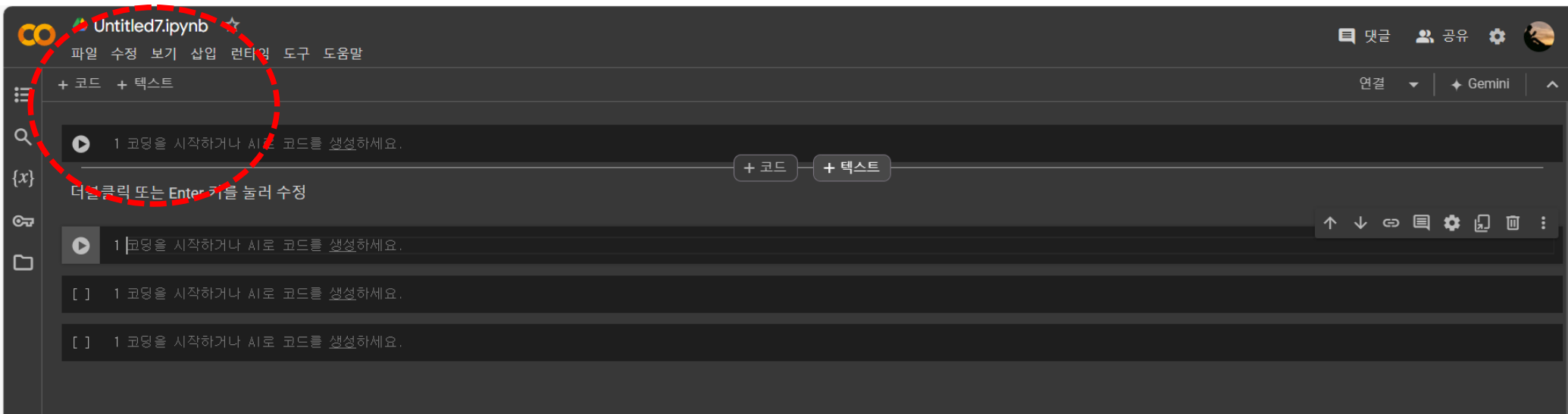


좌측 상단에서 노트의 이름 변경 !!



1) Machine Learning 코랩

코드 창 하단(또는 좌측 상단)에서 **추가적인 코드/텍스트** 창을 추가 가능



1) Machine Learning 코랩

텍스트 창에서 마크다운 문법 사용가능 !!

The screenshot displays the Google Colab interface. At the top, the title bar shows 'Untitled7.ipynb' and various icons for chat, sharing, and settings. Below the title bar, there are tabs for '+ 코드' (Code) and '+ 텍스트' (Text). The main workspace is divided into two panels. The left panel, labeled '+ 코드', contains a code editor with the following content:

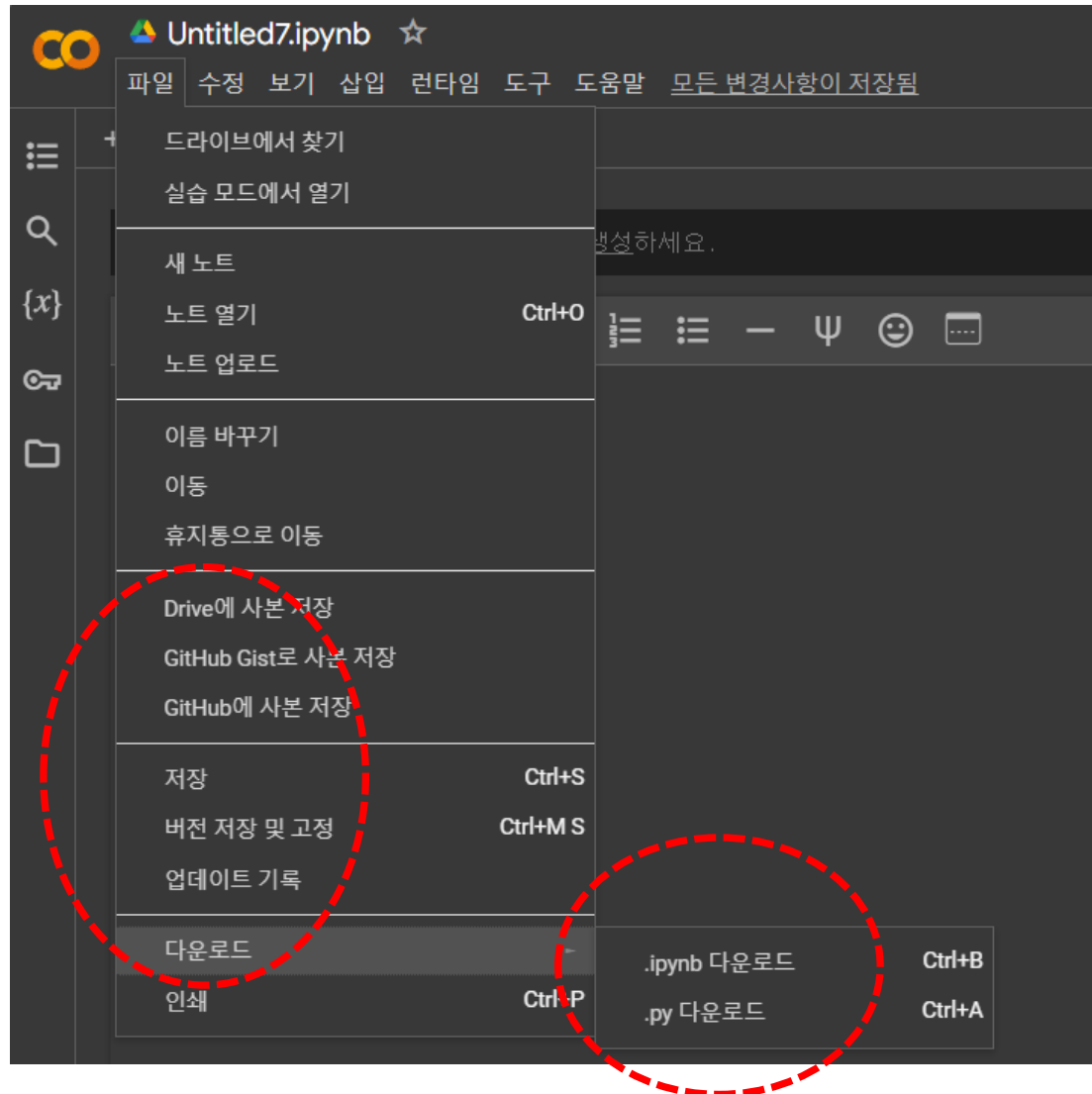
```
[ ] 1 코딩을 시작하거나 시로 코드를 생성하세요.  
  
#마크다운1  
##마크다운2  
###마크다운3  
  
- 목차1  
- 목차2  
- 목차3  
  
> 블록1  
> 블록2  
> 블록3  
  
...  
#코드 1  
print("Hello World")  
...
```

 The right panel, labeled '+ 텍스트', contains a text editor with the following content:

마크다운1
마크다운2
마크다운3
- 목차1
• 목차2
• 목차3
블록1 블록2 블록3
#코드 1
print("Hello World")

 Red dashed circles highlight the markdown formatting in both panels. The left panel shows a list of items with different markers (hashes for headings, hyphens for lists, and greater-than signs for blocks). The right panel shows the rendered output of these markdown elements, including headings, a list, and a block of text.

1) Machine Learning 코랩

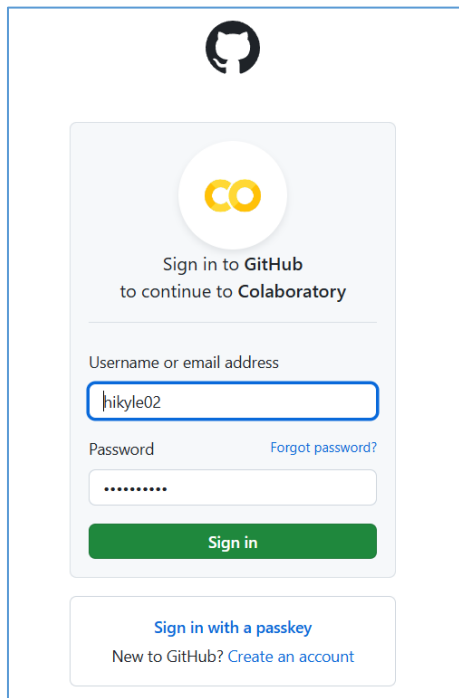


작성된 노트를
다운로드 가능

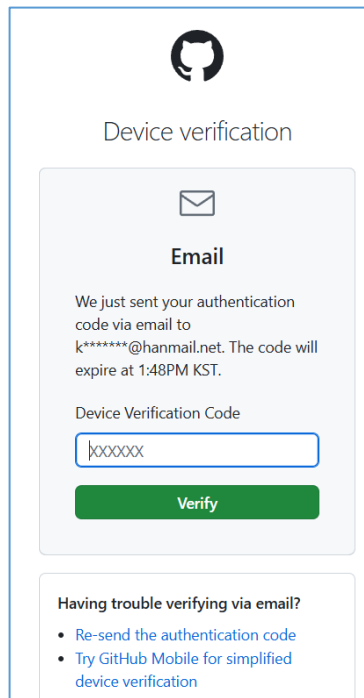
(.py ,
.ipynb)

1) Machine Learning 코랩

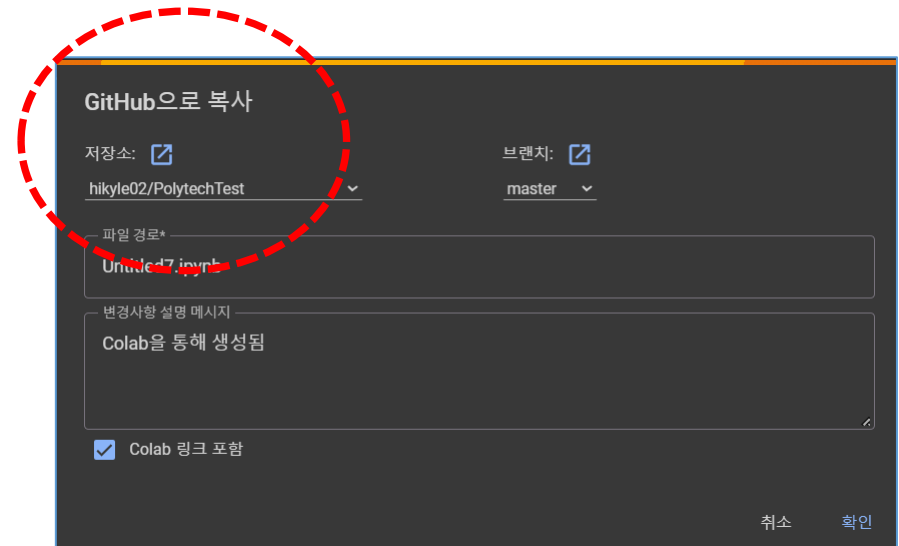
깃허브에 연결하여 저장가능 !!



The image shows the GitHub login interface. At the top is the GitHub logo. Below it is a 'CO' logo with the text 'Sign in to GitHub to continue to Colaboratory'. There are two input fields: 'Username or email address' containing 'hikyle02' and 'Password' with masked characters. A 'Sign in' button is at the bottom. A link for 'Sign in with a passkey' and a link for 'New to GitHub? Create an account' are also visible.



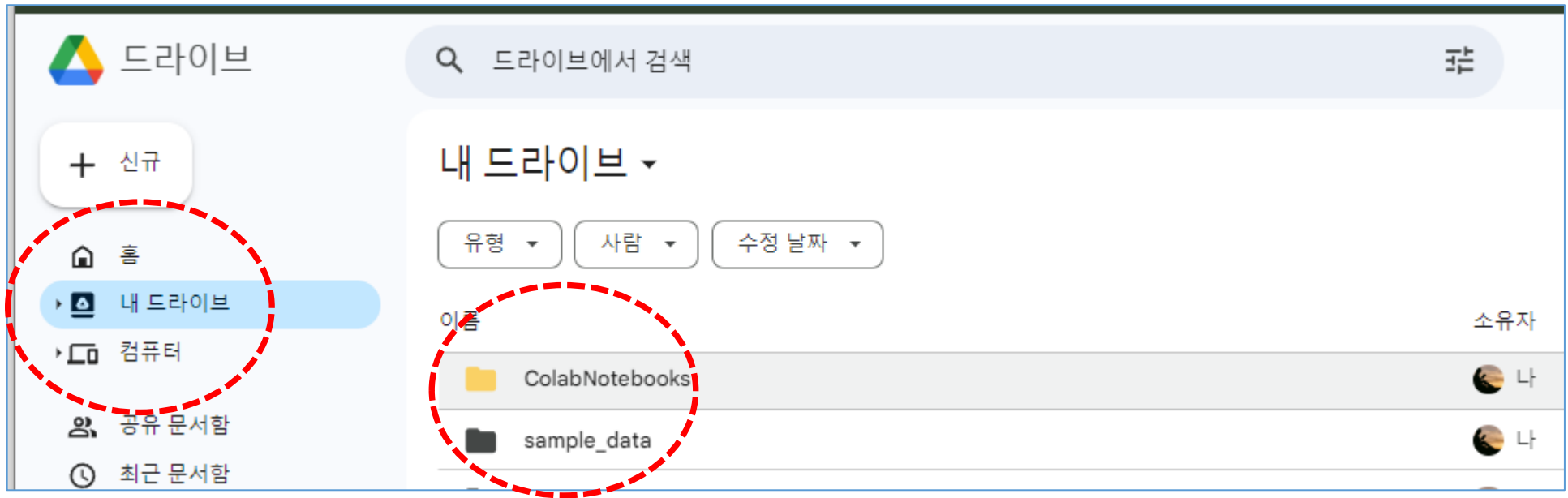
The image shows the GitHub device verification screen. It has the GitHub logo and the text 'Device verification'. Below is an 'Email' section with an envelope icon. It states: 'We just sent your authentication code via email to k*****@hanmail.net. The code will expire at 1:48PM KST.' There is a 'Device Verification Code' input field containing '1000000' and a green 'Verify' button. At the bottom, there is a link 'Having trouble verifying via email?' followed by two bullet points: 'Re-send the authentication code' and 'Try GitHub Mobile for simplified device verification'.



The image shows the GitHub Colab integration screen. A red dashed circle highlights the top section. The title is 'GitHub으로 복사'. Below it are two sections: '저장소:' (Repository) with a dropdown menu showing 'hikyle02/PolytechTest' and '브랜치:' (Branch) with a dropdown menu showing 'master'. Below these is a text area for '파일 경로*' (File path) containing 'Untitled47.ipynb'. There is a section for '변경사항 설명 메시지' (Commit message) with the text 'Colab을 통해 생성됨'. At the bottom, there is a checkbox labeled 'Colab 링크 포함' which is checked. In the bottom right corner, there are two buttons: '취소' (Cancel) and '확인' (Confirm).

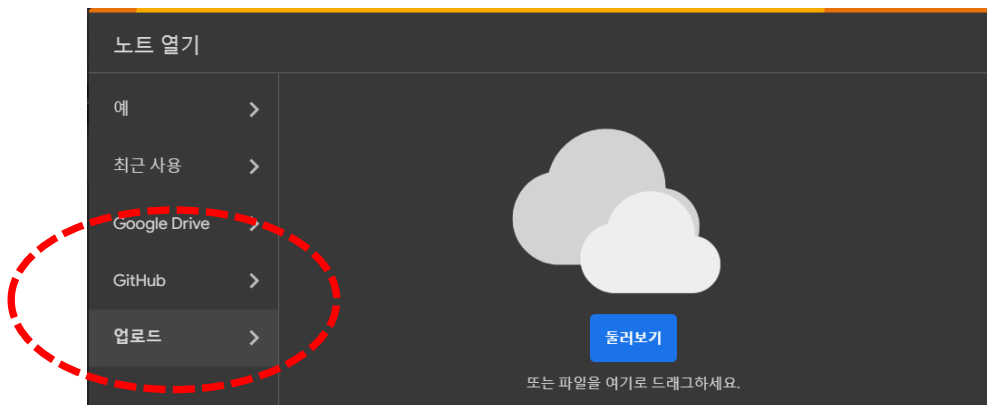
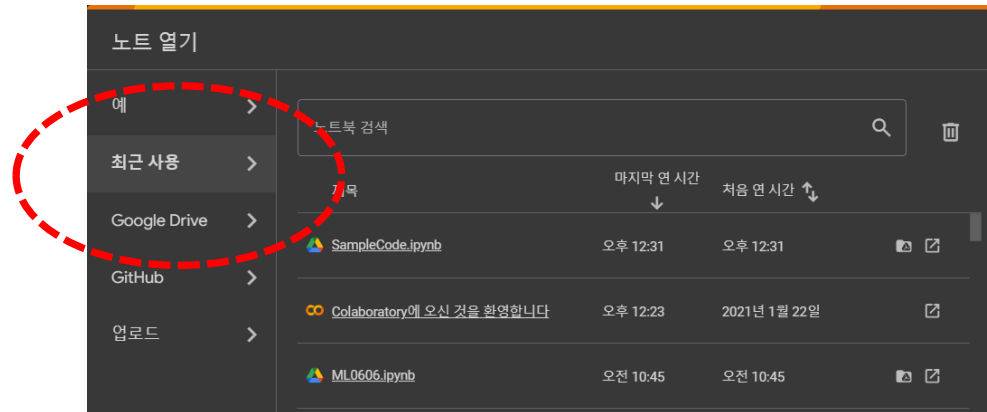
1) Machine Learning 코랩

작성된 노트는 기본적으로 구글 드라이브의 ColabNotebooks라는 폴더에 저장된다



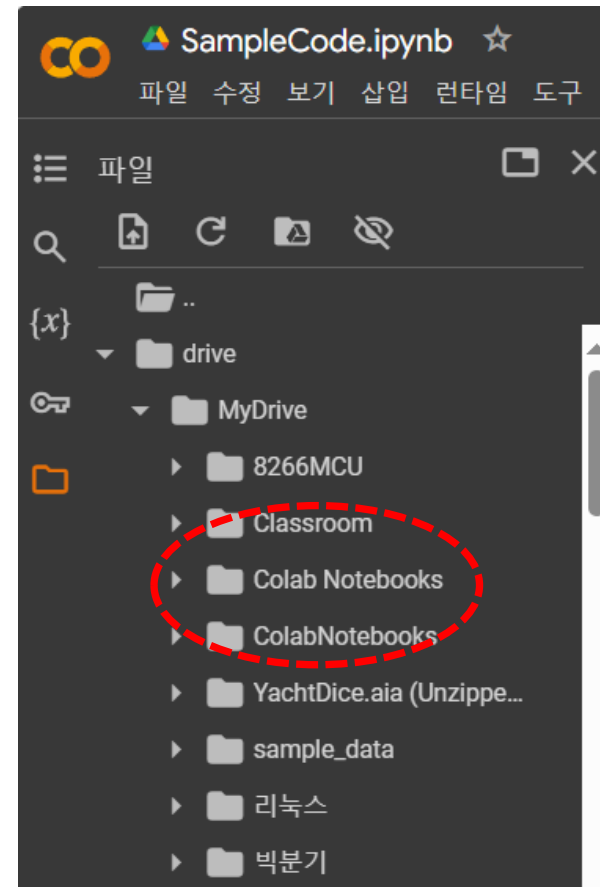
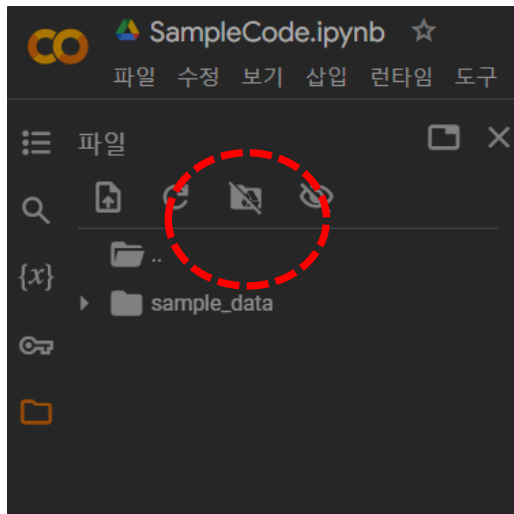
1) Machine Learning 코랩

노트 열기 : 최근사용/업로드/구글 드라이브/깃허브



1) Machine Learning 코랩

좌측 창을 확대하여 구글 드라이브 연결/파일업로드 가능

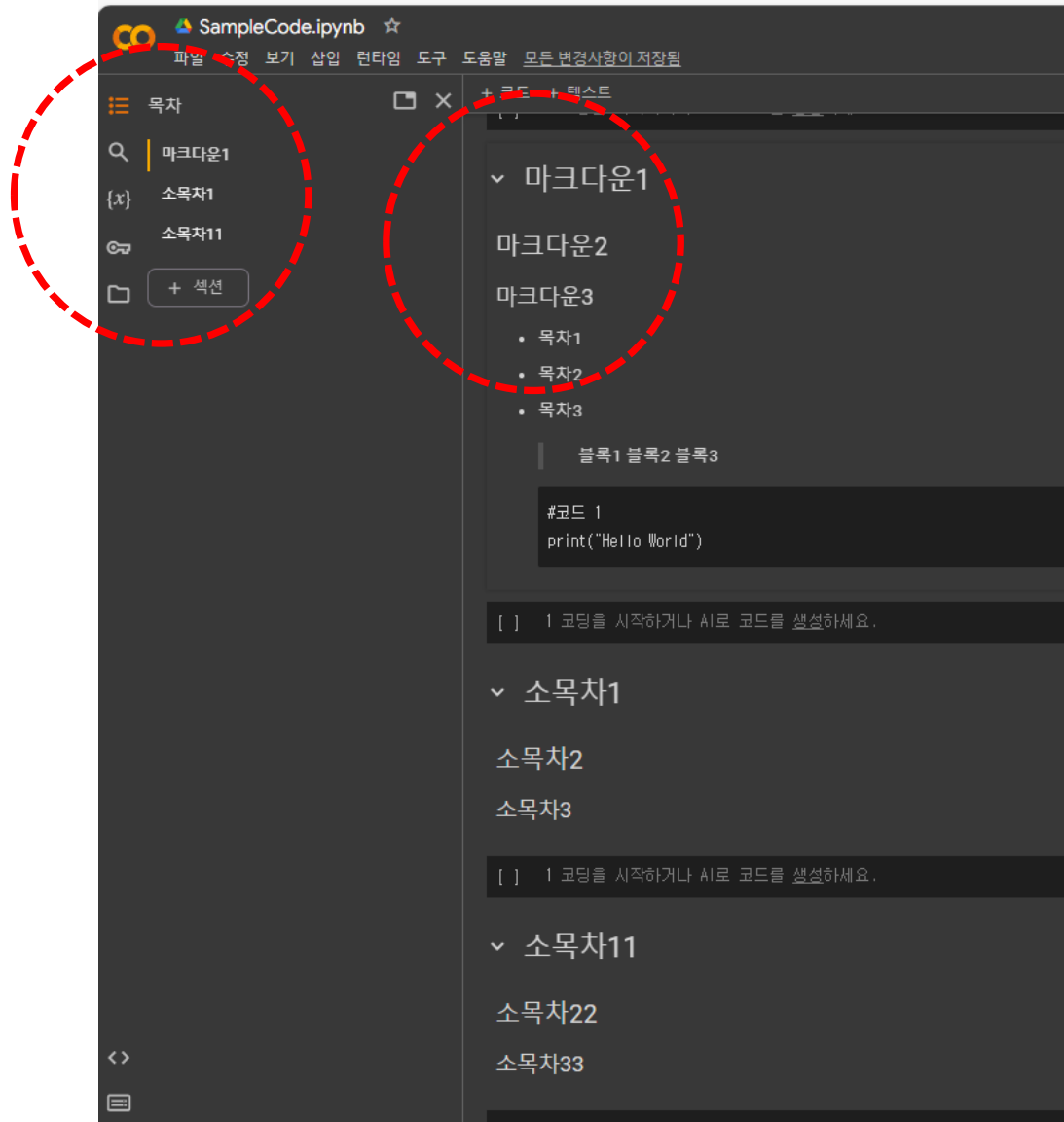


노트북이 Google Drive 파일에 액세스하도록 허용하시겠습니까?

Google Drive에 연결하면 액세스 권한이 취소될 때까지 이 노트북에서 실행된 코드가 Google Drive의 파일을 수정할 수 있습니다.

아니요 [Google Drive에 연결](#)

1) Machine Learning 코랩

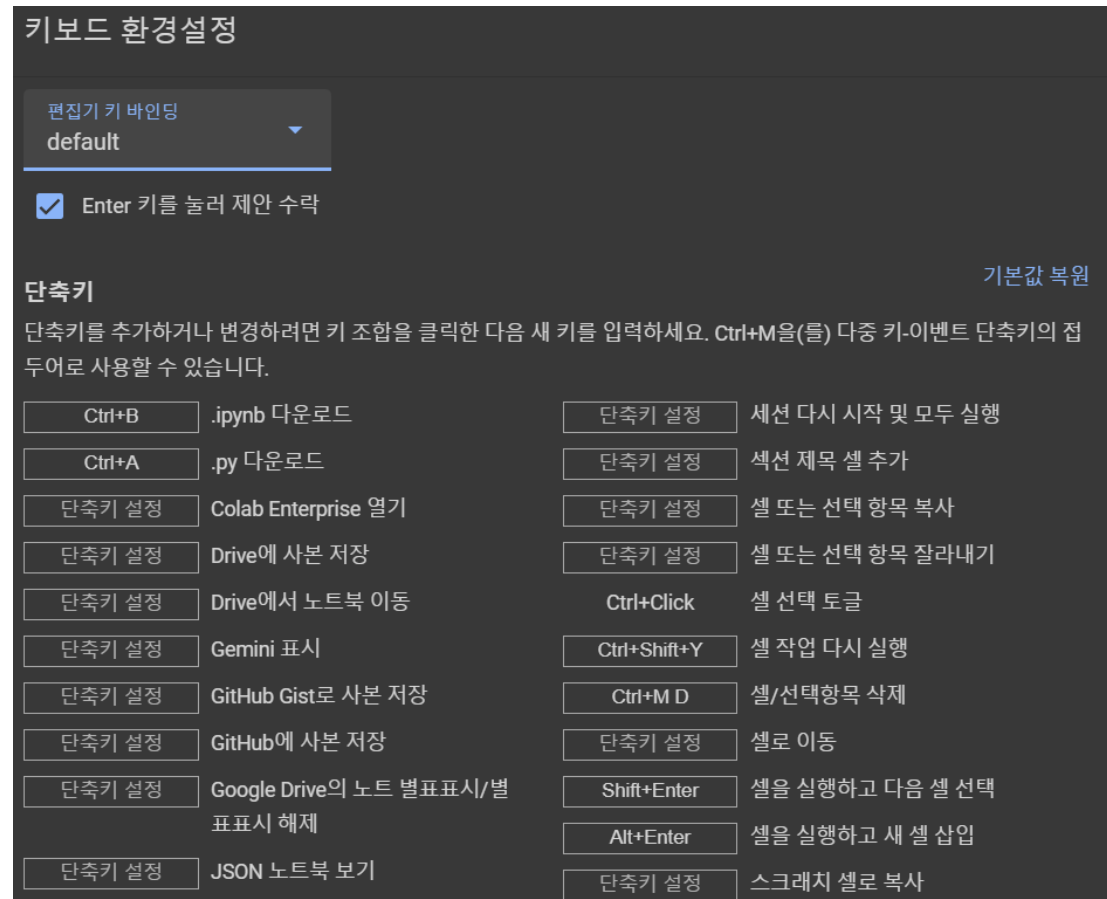
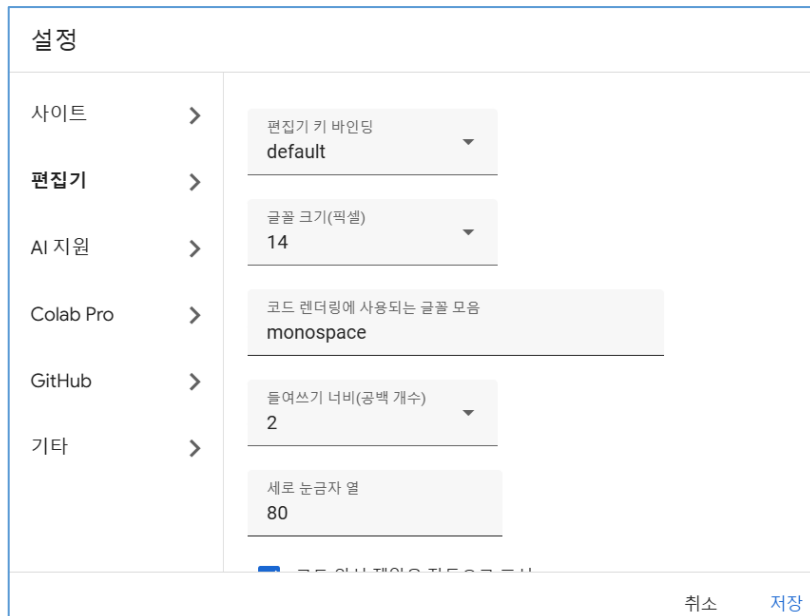


마크 다운 문법 중

#가 1개인 큰목차만

좌측 목차에 나타난다 !!

1) Machine Learning 코랩



1) Machine Learning 코랩

SampleCode.ipynb ☆

파일 수정 보기 삽입 런타임 도구 도움말 모든 변경사항이 저장됨

목차

마크다운1

{x} 소목차1

소목차11

+ 섹션

모두 실행	Ctrl+F9
이전 셀 실행	Ctrl+F8
초점이 맞춰진 셀 실행	Ctrl+Enter
선택항목 실행	Ctrl+Shift+Enter
이후 셀 실행	Ctrl+F10
실행 중단	Ctrl+M I
세션 다시 시작	Ctrl+M .
세션 다시 시작 및 모두 실행	
런타임 연결 해제 및 삭제	
런타임 유형 변경	
세션 관리	
리소스 보기	
런타임 로그 보기	

런타임 유형 변경

런타임 유형

Python 3

하드웨어 가속기 ?

☒ CPU ☐ T4 GPU ☐ A100 GPU ☐ L4 GPU

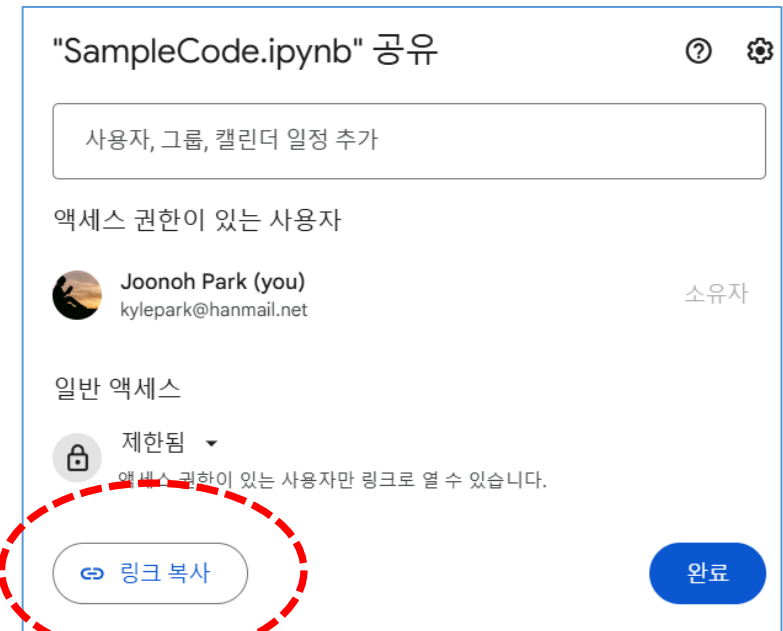
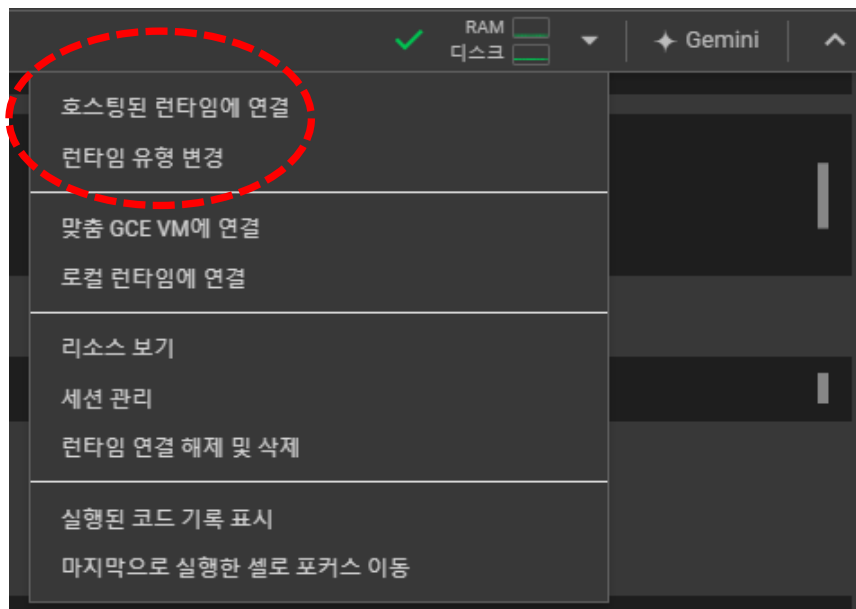
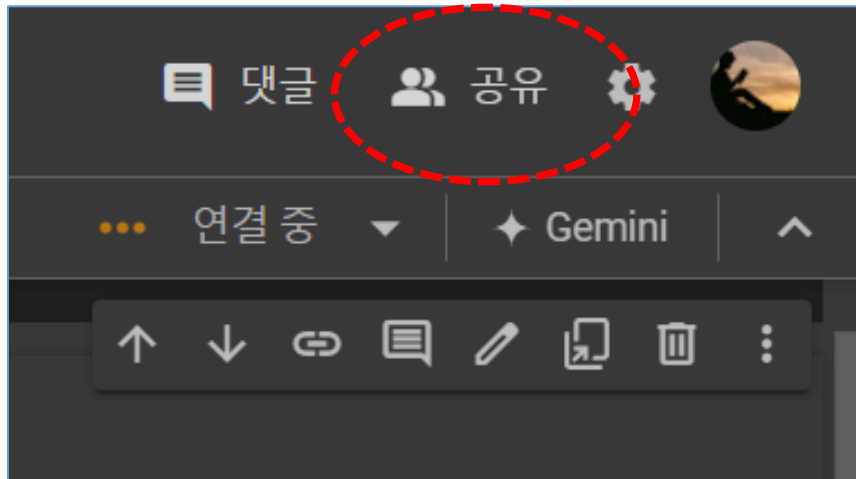
☐ V100 GPU (deprecated) ☐ TPU (deprecated)

☐ TPU v2

프리미엄 GPU를 이용하실까요? 추가 컴퓨팅 단위 구매

취소 저장

1) Machine Learning 코랩



1) Machine Learning 코랩 실습

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 데이터 생성
data = {'Age': [25, 30, 24, 29, 40, 50, 35], 'Salary': [50000, 54000, 58000, 62000, 70000, 80000, 65000]}
df = pd.DataFrame(data)

# 기술통계량
print("기술통계량:\n", df.describe())

# 히스토그램
sns.histplot(df['Age'])
plt.title('Age Distribution')
plt.show()

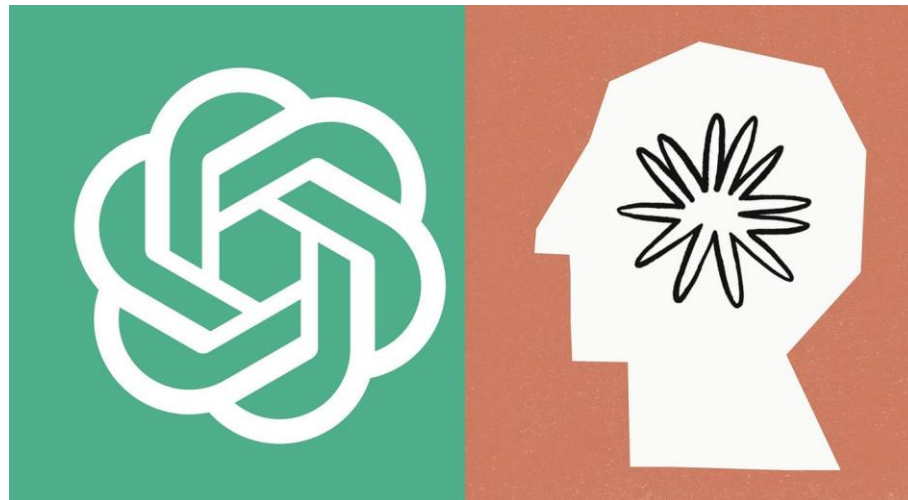
# 박스플롯
sns.boxplot(x=df['Age'])
plt.title('Age Box Plot')
plt.show()

# 산점도
sns.scatterplot(x='Age', y='Salary', data=df)
plt.title('Age vs Salary Scatter Plot')
plt.show()

# 상관 분석
correlation = df.corr()
print("상관계수:\n", correlation)
sns.heatmap(correlation, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

1) Machine Learning Intro 실습 2)

실습 2) 앞서 배운 내용을 바탕으로 예시코드를 생성하여 코드별 해설, 결과값의 해석을 코멘트하여 공유해 봅시다.

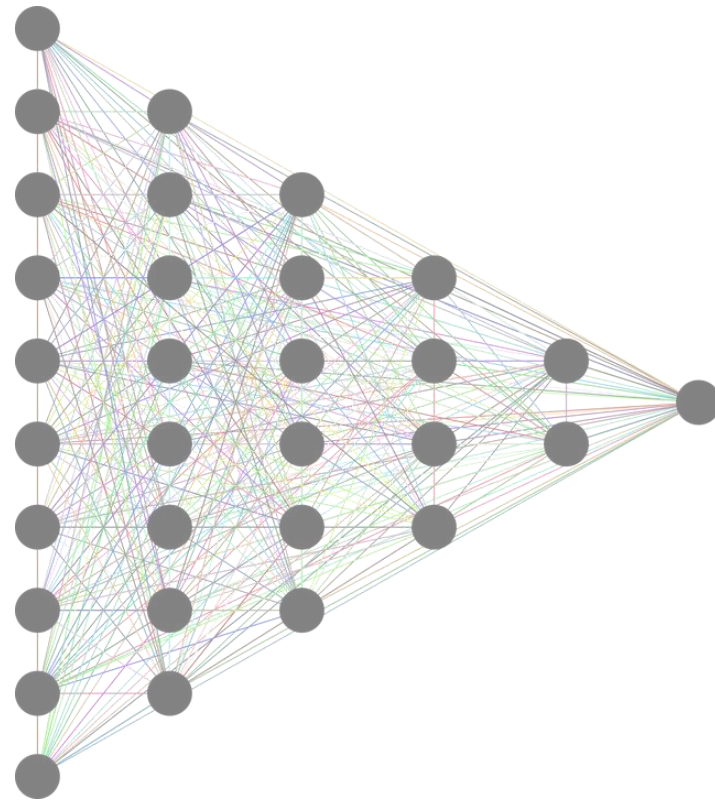


I. Deep learning

- 1. Deep Learning (ANN)**
- 2. Deep Learning (CNN)**
- 3. Transfer Learning**
- 4. Object Detection(YOLO)**

I. Deep learning

**MACHINE
LEARNING**

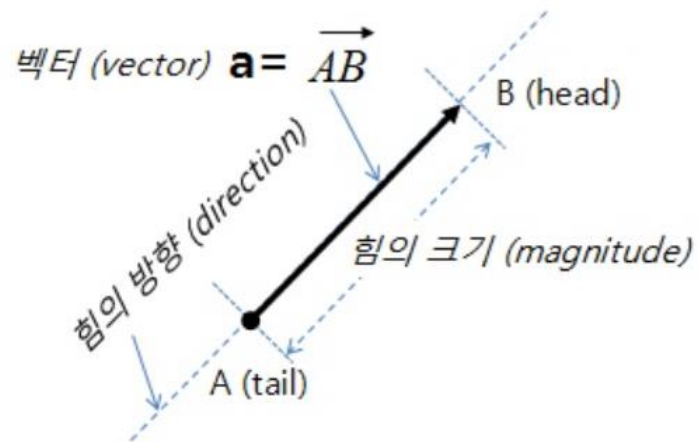


I. Deep learning(선수지식)

1. 벡터(Vector): 벡터는 크기와 **방향**을 가지는 양을 나타내는 개념.

크기는 양수로 표현.

벡터의 방향은 양의 방향과 음의 방향을 가질 수 있다



- 화살표의 길이 : 힘의 크기(magnitude) $|\overrightarrow{AB}|$
- 화살표의 방향 : 힘의 방향(direction)

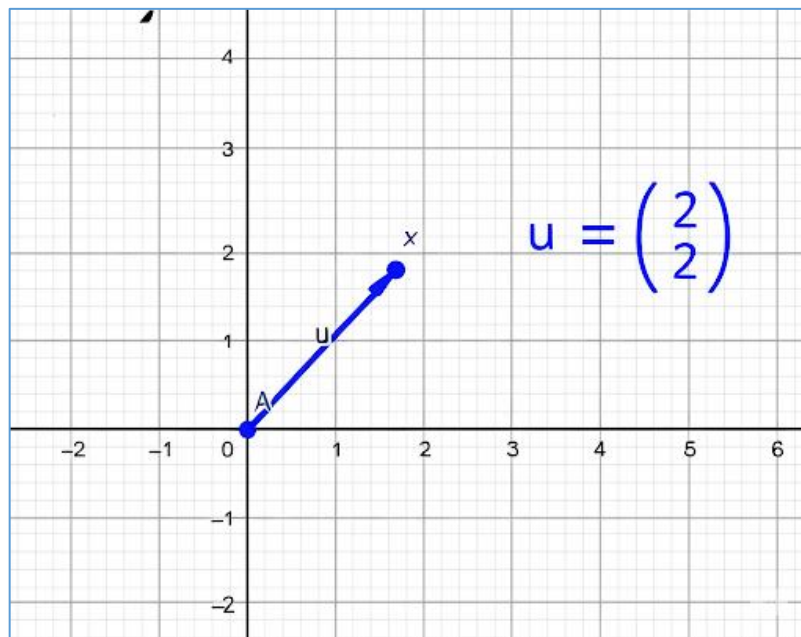
I. Deep learning(선수지식)

1. 벡터(Vector): 벡터는 좌표를 갖는다.

벡터는 특정 좌표계에서의 위치를 나타내며,

각 좌표는 벡터의 성분

일반적으로 2차원 또는 3차원 좌표계에서 벡터는 (x, y) 또는 (x, y, z) 와 같이 좌표로 표현. 좌표는 벡터의 방향과 크기를 결정



I. Deep learning(선수지식)

1. 벡터(Vector): 열벡터($1 \times N$)/행벡터($N \times 1$).
스칼라, 행렬과 구분된다.

Scalar

24

Vector

$\begin{bmatrix} 2 & -8 & 7 \end{bmatrix}$

row

or
column $\begin{bmatrix} 2 \\ -8 \\ 7 \end{bmatrix}$

Matrix

$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}$

row(s) $\times$ column(s)

I. Deep learning(선수지식)

1. 벡터(Vector):

선형대수학 관점

벡터는 방향과 크기를 가진 양을 나타냅니다.

벡터의 원소 개수는 벡터가 놓인 공간의 차원을 의미

예를 들어, 4차원 벡터는 4차원 공간의 한 점 또는 원점으로부터 그 점을 가리키는 화살표

머신러닝/프로그래밍 관점

NumPy 같은 라이브러리의 1차원 **배열 (4,)**은 4개의 요소를 가진 1차원 배열(1D array)

배열의 각 요소는 벡터의 각 성분에 해당하며,

머신러닝에서는 **하나의 데이터 샘플(예: 한 사람의 키, 몸무게, 나이)**을 표현하는 데 사용

I. Deep learning(선수지식)

1. 벡터(Vector) vs 행렬(Matrix):

행렬 (Matrix)은 2차원 배열

선형대수학에서는
선형 변환(linear transformation)을 표현하거나, 연립방정식을 푸는 데 사용합니다.

머신러닝에서는
데이터셋을 나타낼 때 행렬을 사용합니다.

shape = (m, n)에서 **m은 샘플(samples)의 개수,**
n은 각 샘플이 가진 특징(features)의 개수.

즉, 행렬의 각 행(row)은 하나의 데이터 샘플을, 각 열(column)은 하나의 특징을 의미.

I. Deep learning(선수지식)

1. 벡터(Vector) vs 배열(Array, Tensor)

배열 (Array, Tensor)

선형대수학에는 '배열'이라는 용어 자체는 없지만, **텐서(Tensor)**라는 개념이 있습니다. 텐서는 스칼라(0차원), 벡터(1차원), 행렬(2차원)을 고차원으로 일반화한 개념. 이 때문에 머신러닝에서 고차원 배열을 '텐서'라고 부릅니다.

프로그래밍 관점에서

NumPy의 ndarray나 PyTorch의 Tensor는 다양한 차원의 데이터를 다루기 위한 자료 구조 이미지 데이터처럼 복잡한 구조를 다룰 때 특히 유용.

- (10, 28, 28): **10장의 28x28 흑백 이미지**. 10은 샘플 개수, 28x28은 이미지 크기.
- (10, 28, 28, 3): **10장의 28x28 컬러(RGB) 이미지**. 마지막 3은 컬러 채널(channel)

I. Deep learning(선수지식)

1. 벡터(Vector) vs 배열(Array, Tensor)

구분	설명	예시 (아이리스 데이터셋)
벡터 차원 (선형대수학)	한 샘플이 가지는 성분 개수 = 특징 개수	꽃 한 송이 데이터 → [5.1, 3.5, 1.4, 0.2] → 4차원 벡터
배열 차원 (프로그래밍/머신러닝)	데이터가 저장된 축(axis) 개수	전체 데이터셋 shape = (150, 4) → 2차원 배열 (150개의 샘플 × 4개의 성분)
특징 차원 (머신러닝)	입력 데이터의 속성 개수	꽃의 특징(feature) = 꽃받침 길이, 꽃받침 폭, 꽃잎 길이, 꽃잎 폭 → 4차원 입력 공간

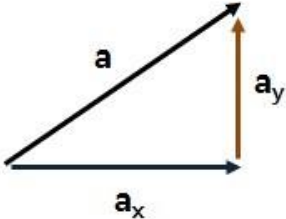
I. Deep learning(선수지식)

1. 벡터(Vector): 벡터 r 의 크기/길이는 $|r|$ 로 표기. 피타고라스 정리로 구한다.

벡터의 힘의 크기 표기법 (notation of magnitude of vector)

$$|\vec{AB}| \quad \text{or} \quad \|\vec{AB}\|$$

벡터의 힘의 크기 계산법 (calculation of magnitude of vector)


$$|\vec{AB}| = |\vec{a}| = \|\vec{a}\| = \sqrt{a_x^2 + a_y^2}$$

[R 분석과 프로그래밍] <http://rfriend.tistory.com>

I. Deep learning(선수지식)

2. 행렬(Matrix): 행렬은 **n 차원 배열**로 구성된 **숫자의 집합**.
행렬은 데이터의 표현과 변환을 위해 널리 사용.

벡터의 차원은 요소의 개수에 의해 결정,
행렬의 차원은 행과 열의 수에 의해 결정.

	제1열	제2열	제3열	제4열
제1행	-1	2	-3	-1
제2행	1	-2	3	4
제3행	0	2	-5	0

I. Deep learning(선수지식)

2. 행렬(Matrix): 행렬은 n 차원 배열로 구성된 숫자의 집합.

•행의 개수: **데이터의 개수**를 의미.

행의 개수는 데이터셋의 크기를 나타내는 지표.

•열의 개수: 데이터 포인트의 **특성 또는 속성**(기하학: 그래프의 축/차원)

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1j} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2j} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{ij} & \cdots & a_{in} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mj} & \cdots & a_{mn} \end{bmatrix}.$$

I. Deep learning(선수지식)

2. 행렬(Matrix): 단위행렬(곱해진 임의의 행렬, 자신이 나오게 한다)

✓ 1. 단위행렬 (Identity Matrix)

● 정의:

- 대각선에는 모두 1, 나머지는 모두 0인 정방행렬
- 행렬 곱셈에서의 항등원 역할:

$$A \cdot I = A, \quad I \cdot A = A$$

● 예시 (3×3 단위행렬):

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

I. Deep learning(선수지식)

2. 행렬(Matrix): 역행렬(단위 행렬이 나오게 하는 역행렬)

✓ 2. 역행렬 (Inverse Matrix)

● 정의:

- 어떤 행렬 A 에 대해,

$$A \cdot A^{-1} = I$$

를 만족하는 행렬 A^{-1} 을 역행렬이라 함

● 조건:

- 반드시 **정방행렬(정사각형)**이어야 하며
- **가역(역행렬 존재)**하려면 행렬식(det)이 0이 아니어야 함

● 예시:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \Rightarrow A^{-1} = \begin{bmatrix} -2 & 1 \\ 1.5 & -0.5 \end{bmatrix}$$

I. Deep learning(선수지식)

2. 행렬(Matrix): 전치행렬(행과 열을 서로 맞바꾼 행렬)

✓ 3. 전치행렬 (Transpose Matrix)

● 정의:

- 행과 열을 뒤바꾼 행렬
(즉, $A[i][j] \rightarrow A^T[j][i]$)

● 표기:

- A^T 또는 $A^\top$

● 예시:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

I. Deep learning(선수지식)

2. 행렬(Matrix): 전치행렬(행과 열을 서로 맞바꾼 행렬)

이미지 처리 : 이미지 데이터는 보통 가로, 세로, 채널(RGB)의 3차원 행렬. 전치 행렬을 이용하여 가로, 세로를 바꾸어 세로, 가로, 채널의 3차원 행렬로 변환. 이미지 데이터를 다양한 딥러닝 모델에 적용하면 더욱 정확한 결과.

자연어 처리 : 문서(문장)를 행으로, 단어를 열로 구성한 행렬. 전치 행렬을 이용하여 문서(문장)-단어 행렬을 단어-문서(문장) 행렬로 변환. 다양한 텍스트 분석 기법에 적용되며, 문서(문장) 간 유사도 측정, 토픽 모델링 등 다양한 분석에 활용.

$$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}^T = \begin{bmatrix} 6 & 1 \\ 4 & -9 \\ 24 & 8 \end{bmatrix}$$

I. Deep learning(선수지식)

2. 행렬(Matrix): 전치행렬(행과 열을 서로 맞바꾼 행렬)

✓ 어텐션 점수 계산:

$$\text{score} = \frac{Q \cdot K^T}{\sqrt{d_k}}$$

- Q : [batch_size, num_heads, seq_len, d_k]
- K^T : [batch_size, num_heads, d_k, seq_len] ← 전치(transpose)
- 내적 결과 → [batch_size, num_heads, seq_len, seq_len]: 각 쌍의 유사도

I. Deep learning(선수지식)

2. 행렬(Matrix): 전치행렬(행과 열을 서로 맞바꾼 행렬)

```
import torch
import math

# 샘플 Q, K, V 생성
batch_size = 1
num_heads = 1
seq_len = 3
d_k = 4

# 예시 Q, K, V 텐서
Q = torch.tensor([[[[1., 0., 1., 0.],
                    [0., 1., 0., 1.],
                    [1., 1., 1., 1.]]]]) # shape: [1, 1, 3, 4]

K = torch.tensor([[[[1., 0., 1., 0.],
                    [1., 1., 0., 0.],
                    [0., 1., 0., 1.]]]]) # shape: [1, 1, 3, 4]

# K 전치: 마지막 두 차원 전치 (d_k x seq_len)
K_T = K.transpose(-1, -2) # shape: [1, 1, 4, 3]

# 어텐션 스코어 계산
score = torch.matmul(Q, K_T) / math.sqrt(d_k) # shape: [1, 1, 3, 3]

print("Q @ K^T 결과:\n", score.squeeze())
```

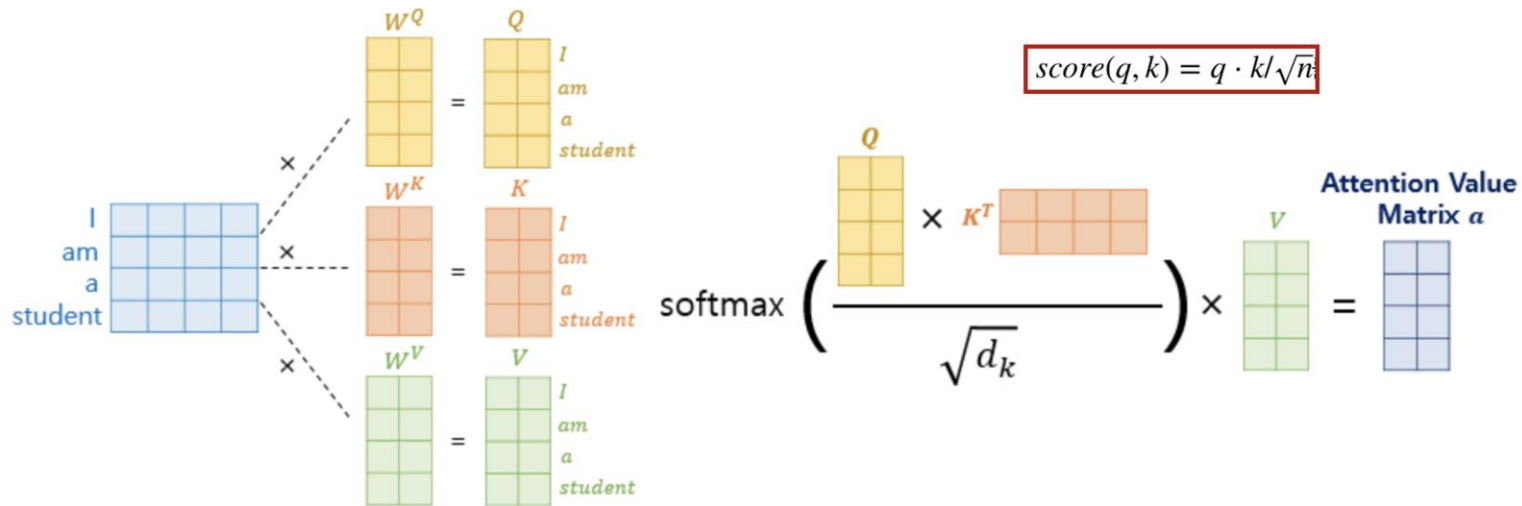
Q @ K^T 결과:

```
tensor([[1.0, 1.0, 0.0],
        [0.0, 0.5, 1.0],
        [1.0, 1.5, 1.0]])
```

I. Deep learning(선수지식)

2. 행렬(Matrix): 전치행렬(행과 열을 서로 맞바꾼 행렬)

* Scaled_dot_product_attention(행렬 연산)



I. Deep learning(선수지식)

2. 행렬(Matrix): 선형 방정식과 연립 방정식을 표현 가능

선형 방정식은 변수들 간의 일차식으로 이루어진 방정식.

선형 방정식을 행렬과 벡터로 나타낼 수 있다.

연립 방정식은 여러 개의 선형 방정식으로 이루어진 방정식의 집합.

$$\begin{aligned} \begin{bmatrix} 2 & 3 & 4 \\ -1 & 5 & -3 \\ 6 & -2 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} &= x_1 \begin{bmatrix} 2 \\ -1 \\ 6 \end{bmatrix} + x_2 \begin{bmatrix} 3 \\ 5 \\ -2 \end{bmatrix} + x_3 \begin{bmatrix} 4 \\ -3 \\ 8 \end{bmatrix} \\ &= \begin{bmatrix} 2x_1 \\ -x_1 \\ 6x_1 \end{bmatrix} + \begin{bmatrix} 3x_2 \\ 5x_2 \\ -2x_2 \end{bmatrix} + \begin{bmatrix} 4x_3 \\ -3x_3 \\ 8x_3 \end{bmatrix} \\ &= \begin{bmatrix} 2x_1 + 3x_2 + 4x_3 \\ -x_1 + 5x_2 - 3x_3 \\ 6x_1 - 2x_2 + 8x_3 \end{bmatrix} \end{aligned}$$

I. Deep learning(선수지식)

2. 행렬의 연산(덧셈/뺄셈):

$$\begin{bmatrix} 3 & 8 \\ 4 & 6 \end{bmatrix} + \begin{bmatrix} 4 & 0 \\ 1 & -9 \end{bmatrix} = \begin{bmatrix} 7 & 8 \\ 5 & -3 \end{bmatrix}$$

Diagram illustrating matrix addition. A yellow arrow connects the top-left elements (3 and 4) with the calculation $3+4=7$ above it. Another yellow arrow connects the bottom-left elements (4 and 1) with the calculation $4+1=5$ above it.

$$\begin{bmatrix} 3 & 8 \\ 4 & 6 \end{bmatrix} - \begin{bmatrix} 4 & 0 \\ 1 & -9 \end{bmatrix} = \begin{bmatrix} -1 & 8 \\ 3 & 15 \end{bmatrix}$$

Diagram illustrating matrix subtraction. A yellow arrow connects the top-left elements (3 and 4) with the calculation $3-4=-1$ above it. Another yellow arrow connects the bottom-left elements (4 and 1) with the calculation $4-1=3$ above it.

$$-\begin{bmatrix} 2 & -4 \\ 7 & 10 \end{bmatrix} = \begin{bmatrix} -2 & 4 \\ -7 & -10 \end{bmatrix}$$

Diagram illustrating scalar multiplication of a matrix by -1. A yellow arrow connects the top-left element (2) with the calculation $-(2)=-2$ above it. Another yellow arrow connects the bottom-left element (7) with the calculation $-(7)=-7$ above it.

$$2 \times \begin{bmatrix} 4 & 0 \\ 1 & -9 \end{bmatrix} = \begin{bmatrix} 8 & 0 \\ 2 & -18 \end{bmatrix}$$

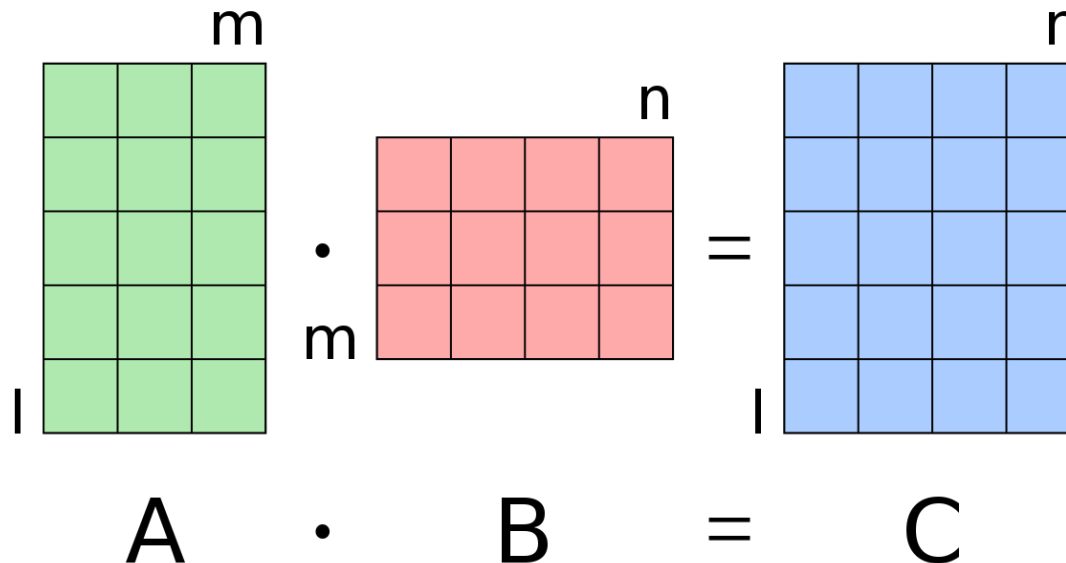
Diagram illustrating scalar multiplication of a matrix by 2. A yellow arrow connects the top-left element (4) with the calculation $2 \times 4 = 8$ above it. Another yellow arrow connects the bottom-left element (1) with the calculation $2 \times 1 = 2$ above it.

I. Deep learning(선수지식)

2. 행렬의 연산(곱):

(행***열**) * (행***열**) : 앞 행렬의 열과 뒤 행렬의 행의 숫자가 같아야 한다.

(행***열**) * (행***열**) : 행렬의 곱은 앞 행렬의 행과 뒤 행렬의 열의 크기를 갖는다
 $\Rightarrow (5 * 3) * (3 * 4) = 5 * 4$



2. 행렬의 연산(곱):

I. Deep learning(선수지식)

1. 행렬과 벡터 간의 곱:

1. A: 3x2 행렬 (특성: 길이, 너비)
2. B: 2x1 벡터 (특성: 가격)
3. $C = A * B$
4. 결과: C는 3x1 벡터 (특성: 길이 *가격*, *너비*가격)

2. 행렬과 행렬 간의 곱:

1. A: 4x3 행렬 (특성: 무게, 길이, 너비)
2. B: 3x2 행렬 (특성: 가격, 수량)
3. $C = A * B$
4. 결과: C는 4x2 행렬 (특성: 무게 *가격*, *길이*가격, *너비*가격, *무게*수량, *길이*수량, *너비*수량)

3. 행렬과 행렬 간의 곱 (브로드캐스팅 사용):

1. A: 2x2 행렬 (특성: 높이, 너비)
2. B: 1x2 행렬 (특성: 비율1, 비율2)
3. $C = A * B$
4. 결과: B가 브로드캐스팅되어 2x2 행렬로 확장되어 계산됨,
C는 2x2 행렬 (특성: 높이 *비율1*, *너비*비율1, 높이 *비율2*, *너비*비율2)

I. Deep learning(선수지식)

- 행렬과 벡터 간의 곱:
 1. A: $[[2, 3], [4, 5], [6, 7]]$ (특성: 길이, 너비)
 2. B: $[[10], [20]]$ (특성: 가격)
 3. $C = A * B$
 4. 결과: $C = [[80], [140], [200]]$ (특성: 길이 *가격*, *너비*가격)
- 행렬과 행렬 간의 곱:
 1. A: $[[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]$ (특성: 무게, 길이, 너비)
 2. B: $[[2, 3], [4, 5], [6, 7]]$ (특성: 가격, 수량)
 3. $C = A * B$
 4. 결과: $C = [[28, 34], [64, 79], [100, 124], [136, 169]]$
(특성: 무게 *가격*, *길이*가격, *너비*가격, *무게*수량, *길이*수량, *너비*수량)
- 행렬과 행렬 간의 곱 (브로드캐스팅 사용):
 1. A: $[[3, 4], [5, 6]]$ (특성: 높이, 너비)
 2. B: $[[0.5, 0.2]]$ (특성: 비율1, 비율2)
 3. $C = A * B$
 4. 결과: B가 브로드캐스팅되어 $[[0.5, 0.2], [0.5, 0.2]]$ 로 확장되어 계산됨,
 $C = [[1.5, 0.8], [2.5, 1.2]]$ (특성: 높이 *비율1*, *너비*비율1, *높이* *비율2*, *너비*비율2)

I. Deep learning(선수지식)

행렬 A (4 x 3)

- 4개의 상품
- 각 상품은 특성 3개: 무게, 길이, 너비

plaintext

복사

편집

```
A =  
[ [1, 2, 3], # 상품 1  
  [4, 5, 6], # 상품 2  
  [7, 8, 9], # 상품 3  
  [10, 11, 12] ] # 상품 4
```

행렬 B (3 x 2)

- 각 특성(무게, 길이, 너비)에 대해 가격, 수량을 제시함

plaintext

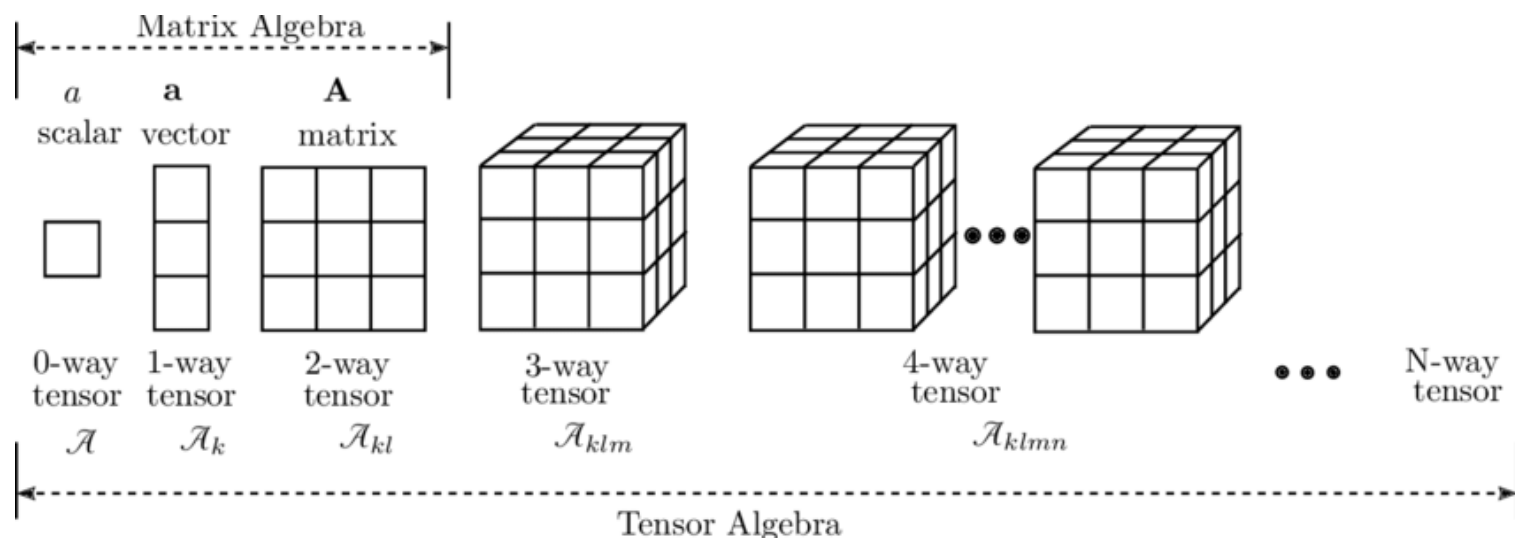
복사

편집

```
B =  
[ [2, 3], # 무게: 가격 2, 수량 3  
  [4, 5], # 길이: 가격 4, 수량 5  
  [6, 7] ] # 너비: 가격 6, 수량 7
```


I. Deep learning(선수지식)

3. 벡터, 행렬, 텐서 :



I. Deep learning(선수지식)

3. 벡터, 행렬, 텐서 :

항목	벡터 (Vector)	행렬 (Matrix)	텐서 (Tensor)
차원 수 (Dim)	1차원 (1D)	2차원 (2D)	3차원 이상 (3D, 4D, ...)
Shape 예시	(3,)	(3, 2)	(2, 3, 4)
구조 형태	[2, 4, 6]	[[1, 2], [3, 4], [5, 6]]	[[[...], [...], [...]], ...]
데이터 의미	1개의 방향성 데이터	행과 열로 구성된 표 형식	여러 행렬 또는 다차원 데이터 묶음
대표적 예시	속도, 가중치 벡터	이미지의 픽셀값, 변환 행렬	배치된 이미지, 문장, 영상 데이터
활용 분야	선형대수, NLP	회귀분석, CNN 필터 등	딥러닝 전반 (CNN, RNN, Transformer)

I. Deep learning(선수지식)

1. 열벡터와 행벡터의 정의

- 열벡터(Column Vector): $v = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ (2×1)
- 행벡터(Row Vector): $w = \begin{bmatrix} 4 & 5 \end{bmatrix}$ (1×2)

```
import torch
```

```
# 열벡터
```

```
v = torch.tensor([[2], [3]]) # shape: (2, 1)  
print("열벡터 v:", v, "shape:", v.shape)
```

```
# 행벡터
```

```
w = torch.tensor([[4, 5]]) # shape: (1, 2)  
print("행벡터 w:", w, "shape:", w.shape)
```

```
열벡터 v: tensor([[2],  
                  [3]]) shape: torch.Size([2, 1])  
행벡터 w: tensor([[4, 5]]) shape: torch.Size([1, 2])
```

I. Deep learning(선수지식)

2. 행렬곱 (Matrix-Vector Multiplication)

행렬 A 와 열벡터 v 의 곱을 계산합니다.

- 행렬: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ (2×2)
- 열벡터: $v = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ (2×1)
- 연산: $y = A \cdot v = \begin{bmatrix} 1 \cdot 2 + 2 \cdot 3 \\ 3 \cdot 2 + 4 \cdot 3 \end{bmatrix} = \begin{bmatrix} 8 \\ 18 \end{bmatrix}$ (2×1)

행렬 A

```
A = torch.tensor([[1, 2], [3, 4]]) # shape: (2, 2)
```

```
print("행렬 A:", A, "shape:", A.shape)
```

행렬곱 A @ v

```
y = A @ v
```

```
print("행렬곱 A @ v:", y, "shape:", y.shape)
```

```
행렬 A: tensor([[1, 2],
                 [3, 4]]) shape: torch.Size([2, 2])
행렬곱 A @ v: tensor([[ 8],
                      [18]]) shape: torch.Size([2, 1])
```

I. Deep learning(선수지식)

3. 행벡터와 열벡터의 내적

행벡터 $w = [4 \ 5]$ 와 열벡터 $v = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$ 의 내적을 계산합니다.

- 연산: $w \cdot v = 4 \cdot 2 + 5 \cdot 3 = 8 + 15 = 23$ (스칼라)

```
# 내적 w @ v
```

```
inner_product = w @ v
```

```
print("내적 w @ v:", inner_product, "shape:", inner_product.shape)
```

```
내적 w @ v: tensor([[23]]) shape: torch.Size([1, 1])
```


I. Deep learning(선수지식)

5. 주의: 행벡터와 행렬곱

행벡터 $w = [4 \ 5]$ 를 A 와 곱하려면, $w \cdot A$ 형태로 계산해야 합니다.

- 연산: $w \cdot A = [4 \ 5] \cdot \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = [4 \cdot 1 + 5 \cdot 3 \quad 4 \cdot 2 + 5 \cdot 4] = [19 \quad 28] \ (1 \times 2)$

```
# w @ A
result_wA = w @ A
print("w @ A:", result_wA, "shape:", result_wA.shape)
```

```
w @ A: tensor([[19, 28]]) shape: torch.Size([1, 2])
```

I. Deep learning(선수지식)

요약

- 열벡터는 $A \cdot v$ 처럼 행렬 오른쪽에 곱할 때 사용됩니다.
- 행벡터는 $w \cdot A$ 처럼 행렬 왼쪽에 곱하거나 내적에 사용됩니다.
- 1차원 배열은 PyTorch/NumPy에서 유연하게 처리되지만, 명확성을 위해 `.unsqueeze(1)` (열벡터) 또는 `.unsqueeze(0)` (행벡터)를 사용하세요.
- 차원 확인: 연산 전 `shape` 를 확인하면 오류를 줄일 수 있습니다.

항목	행렬의 곱 (matrix multiplication)	내적 (dot product)
대상	두 행렬 또는 행렬과 벡터	두 벡터
연산 구조	각 행 $\times$ 각 열	같은 위치끼리 곱해서 다 더함
결과 차원	행렬 (2D 이상 유지)	스칼라(숫자 한 개)
용도	선형 변환, 특징 결합	유사도, 투영 계산 등

I. Deep learning(선수지식)

4. 행렬의 표기 : 튜플/텐서

개념	수학 표기	NumPy / PyTorch
길이 2 벡터	2, 3 또는 $\begin{bmatrix} 2 \\ 3 \end{bmatrix}$	shape = (2,)
2행 1열 열벡터	$\begin{bmatrix} 2 \\ 3 \end{bmatrix}$	shape = (2, 1)
1행 2열 행벡터	$[2, 3]$	shape = (1, 2)

```
a = torch.tensor([2, 3])    # shape: (2,)
b = torch.tensor([[1], [2]]) # shape: (2, 1)
c = torch.matmul(a, b)      # → OK: (2,) · (2, 1) = (1,)
```

I. Deep learning(선수지식)

구분	튜플 (tuple)	텐서의 shape (PyTorch/Numpy)
정의	파이썬의 자료형 (컨테이너 타입)	텐서의 차원 크기 정보 를 담는 튜플형태
예시	(2,): 요소 하나 (값은 2)	(2,): 1차원 텐서, 요소 2개
목적	여러 값을 묶기 위한 자료형	텐서 구조를 정의하는 메타 정보
형태	튜플은 언제나 값 자체를 담음	텐서의 .shape는 튜플을 리턴하지만 값이 아님
요소 개수	<code>len((2,)) = 1</code>	<code>tensor.shape = (2,)</code> → <code>len = 1</code> , but 요소 2개
사용 맥락	일반 파이썬 연산, 리스트처럼 사용	딥러닝 모델 입력 구조나 reshape 등에 사용

I. Deep learning(선수지식)

```
# 튜플
x = (2,)      # 파이썬 튜플, 요소 1개
print(len(x))  # 1
print(x[0])    # 2

# 텐서
import torch
t = torch.tensor([5.1, 3.5])
print(t.shape) # (2,)
print(t)       # 텐서 [5.1, 3.5] → 요소 2개
```

I. Deep learning(선수지식)

✓ 변환 예시 1

python

복사

편집

```
data = [  
    [ 1, 2, 3, 4],  
    [ 5, 6, 7, 8],  
    [ 9,10,11,12]  
],  
[ [13,14,15,16],  
  [17,18,19,20],  
  [21,22,23,24]  
]  
]
```

▶ 구조 분석:

- 바깥 리스트 [...]에는 2개 요소 → 첫 번째 차원: 2
- 각 요소는 길이 3 리스트 → 두 번째 차원: 3
- 그 안의 리스트는 길이 4 → 세 번째 차원: 4

▶ 최종 shape:

scss

복사

편집

```
(2, 3, 4)
```

I. Deep learning(선수지식)



변환 예시 2

python

복사

편집

```
[  
  [1, 2, 3],  
  [4, 5, 6]  
]
```

- 바깥 리스트: 2개 → 2
- 내부 리스트: 3개 → 3

→ shape: (2, 3)

I. Deep learning(선수지식)

✓ 시각적 패턴 예시

리스트 구조

shape

[1, 2, 3]

(3,)

[[1, 2, 3], [4, 5, 6]]

(2, 3)

[[[1,2],[3,4]], [[5,6],[7,8]]]

(2, 2, 2)

[[[[1,2],[3,4]]]]

(1, 1, 2, 2)

I. Deep learning(선수지식)

**-1 : reshape()에서 사용
해당 차원의 크기를 자동으로 계산하도록** PyTorch나 NumPy에 지시하기 위함입니다..

사용 사례	코드 예시	설명
1. 이미지 reshape	<code>x.reshape(-1, 28, 28)</code>	배치 개수를 자동으로 계산 (예: (60000, 28, 28) 등)
2. Flatten (펼치기)	<code>x.view(batch_size, -1)</code>	CNN의 마지막 feature map을 1차원으로 평탄화
3. 임의 차원 제거	<code>x.view(-1)</code>	전체를 1차원 벡터로 만들기
4. 채널 차원 추가 후 reshape	<code>x.reshape(-1, 3, 32, 32)</code>	CIFAR-10: RGB 채널 이미지 재구성
5. 자동 배치 분할	<code>x.reshape(-1, feature_size)</code>	feature의 크기는 고정, 배치 사이즈는 자동
6. RNN input 재구성	<code>x.reshape(seq_len, -1, input_size)</code>	배치 사이즈를 자동으로 맞춤
7. Dynamic batch flattening	<code>x.view(-1, height * width)</code>	예: <code>x = [batch, channel, height, width]</code> 에서 <code>height*width</code> 만 남기기
8. 디코더 출력 평탄화 후 Dense 연결	<code>x = x.view(-1, d_model)</code>	트랜스포머 디코더 출력에서 사용하는 경우
9. Loss 계산용 reshape	<code>x = x.view(-1, num_classes)</code> <code>y = y.view(-1)</code>	CrossEntropyLoss 사용 전 reshape 필요
10. GPU 텐서 일괄 변환	<code>x = x.contiguous().view(-1, d_model)</code>	transpose 후 view를 사용할 경우 contiguous()와 함께 사용

I. Deep learning(선수지식)

PyTorch vs TensorFlow

항목	PyTorch	TensorFlow
기본 차원 순서	channels_first (예: [N, C, H, W])	channels_last (예: [N, H, W, C])
행렬곱 연산자	@, torch.matmul()	tf.matmul()
브로드캐스팅	NumPy 호환, 유연함	NumPy 호환, 약간 보수적
인덱싱 방식	Python/Numpy 스타일 강력 지원	유사하지만 일부 제한 존재
default tensor type	float32 (명시적 변환 자주 필요)	float32 (자동 변환 많음)
GPU 처리	tensor.to('cuda')로 명시	자동 처리 또는 device 지정

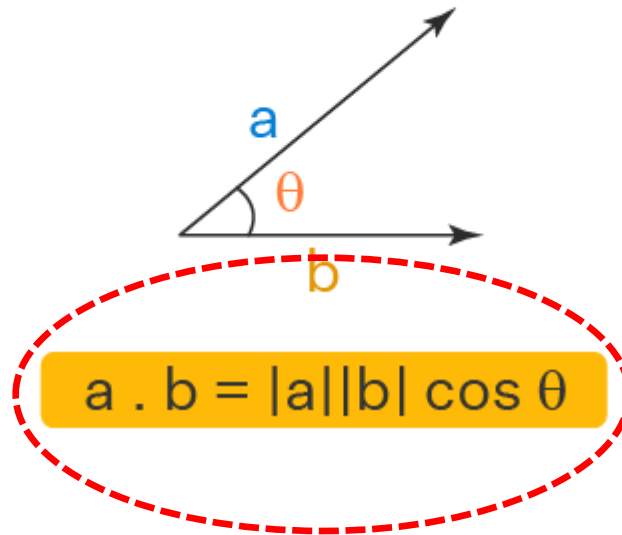
I. Deep learning(선수지식)

실습) 벡터/행렬/텐서에 대해 정리하고 공유해 주세요

I. Deep learning(선수지식)

5. **내적(Dot product):** 두 벡터의 유사성을 측정하는 기준.

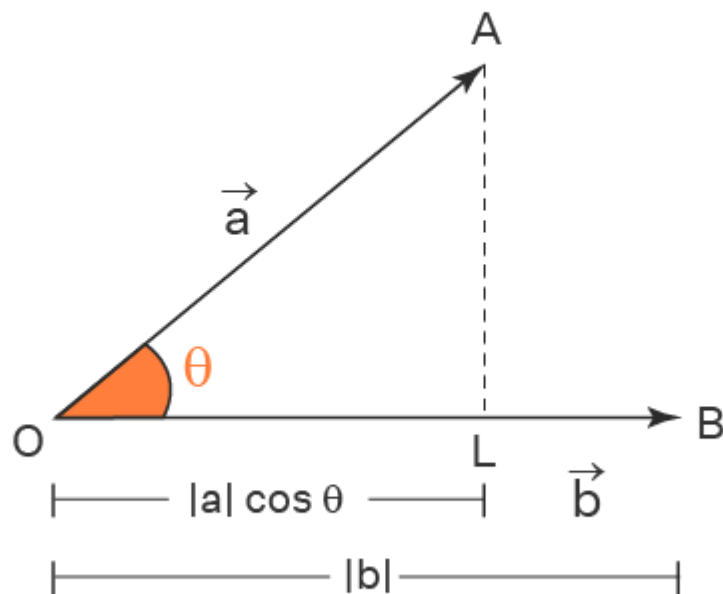
Dot Product



I. Deep learning(선수지식)

5. 내적(Dot product): 두 벡터의 각도값(cosine)을 곱함으로써
방향에 대한 유사성을 포함하게 된다.

Geometrical meaning of Dot Product

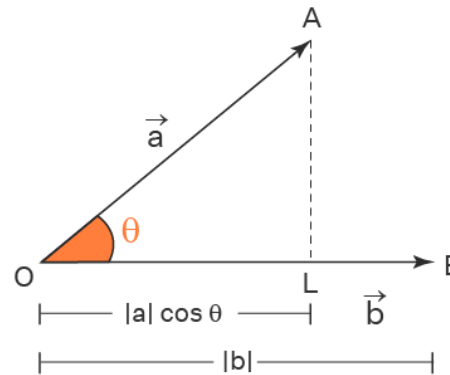


$$\vec{a} \cdot \vec{b} = |\vec{a}| \cdot |\vec{b}| \cos \theta$$

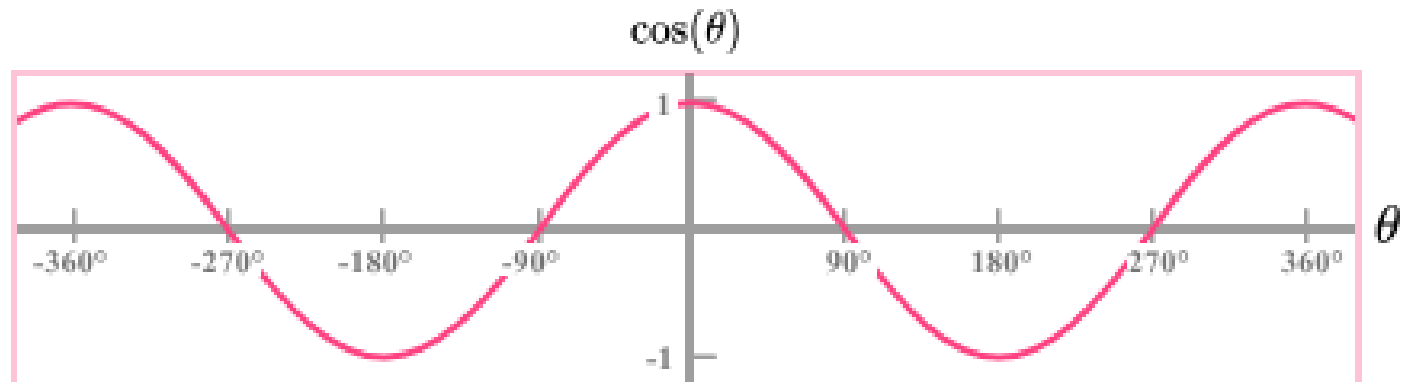
I. Deep learning(선수지식)

5. 내적(Dot product): **방향에 대한 유사성**을 포함하게 된다.

Geometrical meaning of Dot Product



$$a \cdot b = |a| \cdot |b| \cos \theta$$



I. Deep learning(선수지식)

5. 내적(Dot product): 두 벡터의 유사성을 포함하게 된다.

Q, K, V는 단어 벡터를 행으로 하는 문장 행렬

'am' 행과 'student' 열의 값은 Q 행렬에 있던 'am' 벡터와 K 행렬에 있던 'student' 벡터의 내적값을 의미.

각 단어 벡터의 유사도가 모두 기록된 **유사도 행렬**

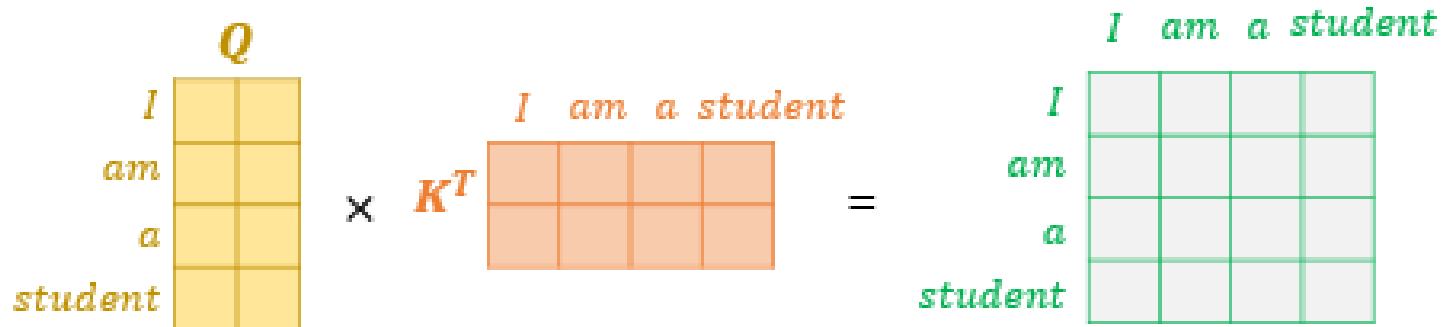
The diagram illustrates the dot product of two matrices, Q and K^T , to produce a similarity matrix. Matrix Q is a 4x2 matrix with rows labeled 'I', 'am', 'a', and 'student'. Matrix K^T is a 2x4 matrix with columns labeled 'I', 'am', 'a', and 'student'. The resulting similarity matrix is a 4x4 matrix with rows and columns labeled 'I', 'am', 'a', and 'student'.

$$\begin{matrix} & Q \\ \begin{matrix} I \\ am \\ a \\ student \end{matrix} & \begin{bmatrix} & \\ & \\ & \\ & \end{bmatrix} \end{matrix} \times \begin{matrix} & K^T \\ \begin{matrix} I & am & a & student \end{matrix} & \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{matrix} = \begin{matrix} & I & am & a & student \\ \begin{matrix} I \\ am \\ a \\ student \end{matrix} & \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{matrix}$$

I. Deep learning(선수지식)

5. 내적(Dot product): 두 벡터의 유사성을 포함하게 된다.

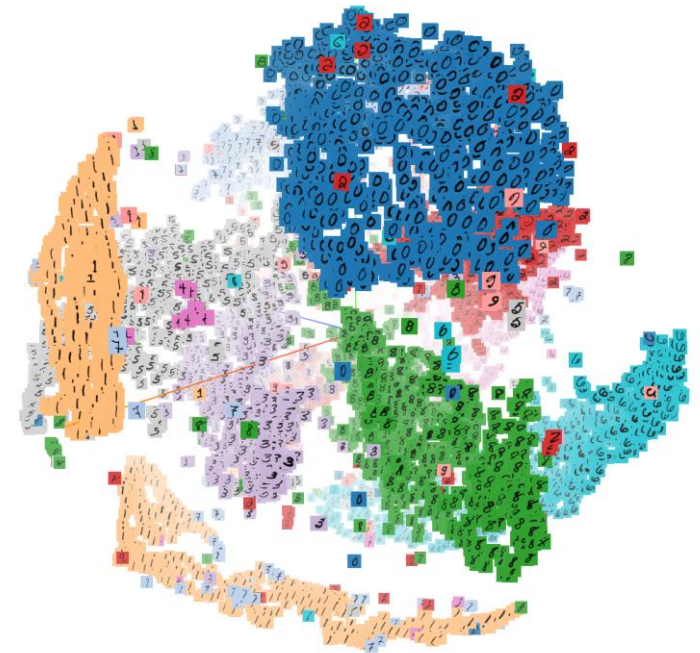
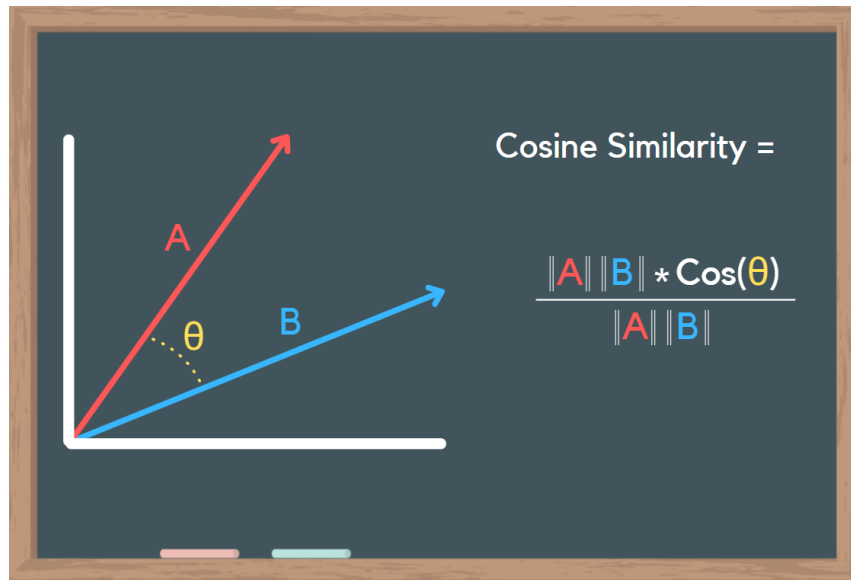
어텐션에서는 단일 벡터 간 유사도만 필요한 게 아니라,
모든 쿼리 단어가 모든 키 단어와 얼마나 연관되는지를 알아야 합니다.
따라서 단일 내적 대신 행렬곱(QK^T)으로 **전체 유사도 행렬**을 구합니다.
이 후 소프트맥스(Softmax)를 적용해서
각 쿼리 단어가 어떤 키 단어에 집중해야 하는지를 확률 분포로 바꿉니다.



내적(dot product)의 집합을 효율적으로 구하기 위해 행렬곱(matrix multiplication)

I. Deep learning(선수지식)

6. Cosine similarity(코사인 유사도)



I. Deep learning(선수지식)

7. Cosine 유사도 사용 사례

- 1) 문서 유사도 분석 (NLP)
벡터 또는 문장 임베딩 벡터 간 유사도 계산
- 2) 문장 임베딩 기반 검색
sentence-transformers, BERT로 임베딩한 후 cosine 유사도 계산
사용 예: 유사 질문 찾기, 뉴스 본문 유사도
- 3) 이미지 검색 / 시각 특징 비교
이미지의 CNN 특징 벡터 사이의 cosine 유사도 계산
- 4) 추천 시스템 : 사용자와 아이템을 임베딩한 후 유사도 기반으로 추천
- 5) 클러스터링, 분류, 군집 해석
군집 내 평균 벡터와 개별 벡터 간 cosine 유사도 계산
- 6) 의미 기반 검색 시스템 (Semantic Search)
문서, 질문, 상품명 등을 벡터로 임베딩한 후 cosine 유사도 순으로 정렬
Pinecone, FAISS 등에서도 cosine distance로 최근접 이웃 검색 수행

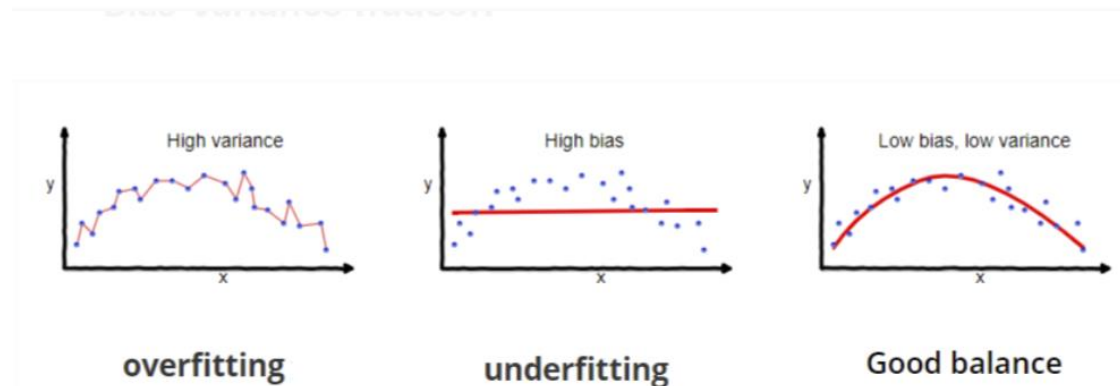
I. Deep learning(선수지식)

8. Regularization : 정규화 ??, 규제화...

Regularization의 주목적 : 모델의 **오버피팅(Overfitting)**을 줄이고자 하는 것.
모델의 일반화 성능을 높이는 방법 중의 하나 :

Data augmentation(데이터 증강), Ensemble model(앙상블),
Dropout(노드 누락), Batch Normalization(배치 정규화),
Early stopping(조기 종료), Multi-Task learning 등

L1, L2 Regularization : Weight 중 일부가 특정 변수에 극단적으로 의존하는 상황
특정 Weight가 과도하게 커지지 않게끔 **제한(regularization)**을 걸어주는 역할



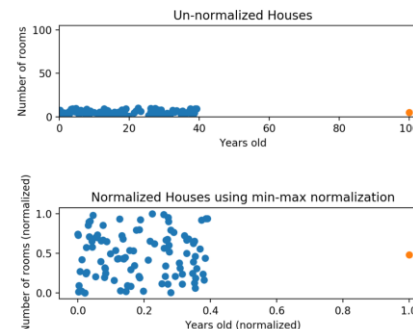
I. Deep learning(선수지식)

8. Normalization : 정규화 ??...

입력 데이터의 스케일을 조정하거나,
데이터의 분포를 조정(상대적인 관계로 분포를 조정)하는데 사용.

- 예를 들어, 모든 입력 특성을 평균이 0이고 표준편차가 1이 되도록 스케일을 조정
StandardScaler, MinMaxScaler.

- 배치 정규화(Batch Normalization)는 **각 레이어의 입력 분포를 정규화**하여, 학습 과정을 안정화하고 학습 속도를 개선하는 기법.
- BN은 데이터의 미세한 통계 차이를 직접적으로 보존하지는 않지만, 이를 표준화된 분포로 변환해 안정적인 학습을 가능하게 합니다.



I. Deep learning(선수지식)

8. Normalization :

입력 데이터의 스케일을 조정하거나, 데이터의 분포를 조정하는데 사용.

```
import numpy as np

# 임의의 데이터 생성
data = np.array([10, 20, 30, 40, 50])

# Z-Score Normalization
normalized_data = (data - np.mean(data)) / np.std(data)

print(normalized_data)
```

```
import numpy as np

# 임의의 데이터 생성
data = np.array([10, 20, 30, 40, 50])

# Min-Max Normalization
normalized_data = (data - np.min(data)) / (np.max(data) - np.min(data))

print(normalized_data)
```

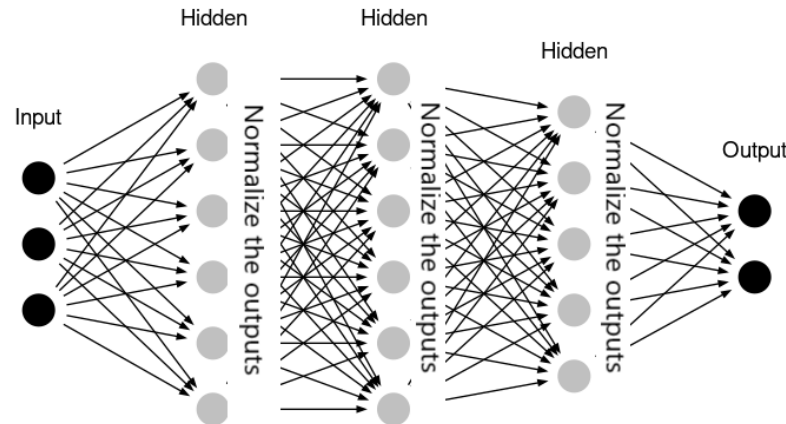
I. Deep learning(선수지식)

8. Normalization : 데이터의 분포를 조정하는데 사용(layer normalization).

배치 정규화(Batch Normalization, BN)

신경망 학습 과정에서 각 배치(batch) 단위로 평균과 분산을 계산해, 출력을 정규화
(입력 데이터는 학습 전에 보통 표준화(Z-score), Min-Max scaling 등으로 정규화)

BN은 학습 과정 내내 모든 레이어 입력을 배치 단위로 정규화
동적인 방식으로, 학습 중 매번 변하는 레이어 출력 분포를 제어
적용 위치는 보통 Dense Layer나 Convolution Layer의 출력 → 활성화 함수(ReLU 등) 앞
CNN에서는 Conv → BN → ReLU 패턴



I. Deep learning(선수지식)

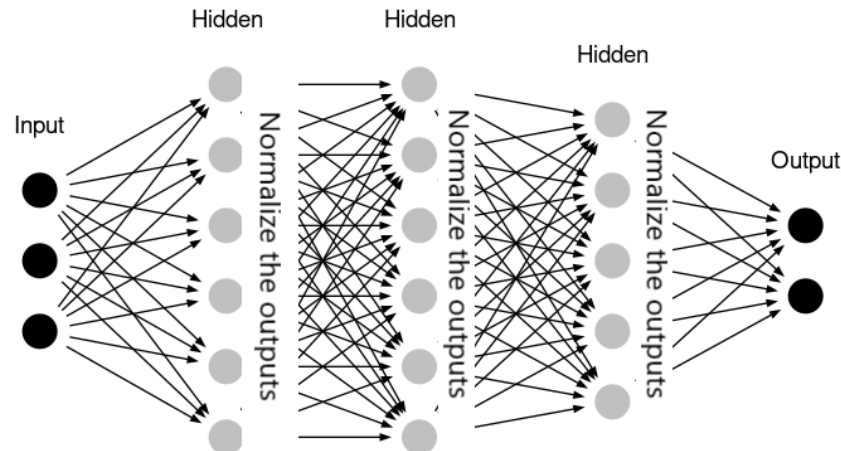
8. Normalization : 데이터의 분포를 조정하는데 사용(layer normalization).

Layer Normalization :

어떤 샘플이 하나의 레이어를 통과했을 때 생긴 출력(feature vector)을 평균과 분산으로 정규화하는 것이 LayerNorm

특정 레이어의 모든 뉴런 값(feature)을 평균 0, 분산 1로 맞추는 정규화 기법

특정 레이어의 출력을 단위로 평균·분산을 구하기 때문에, 레이어 단위 정규화



I. Deep learning(선수지식)

8. Normalization : 데이터의 분포를 조정하는데 사용(layer normalization).

Layer Normalization :

목적 (Purpose)

- **학습 안정화**: 뉴런 값이 지나치게 커지거나 작아지는 것을 막아줍니다.
- **배치 크기 의존성 제거**:

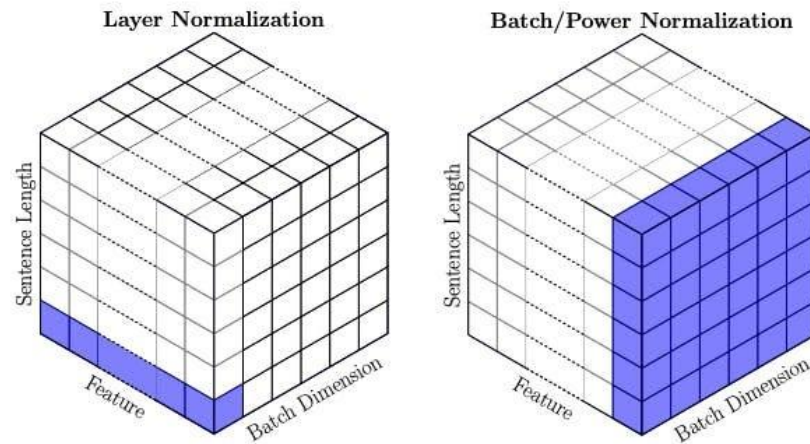
Batch Normalization은 배치(batch) 전체의 통계량을 이용하기 때문에, 배치 크기가 작거나(1) 변하면 잘 동작하지 않습니다. 반면, Layer Normalization은 샘플 내부에서만 통계량을 계산하므로 배치 크기와 무관하게 안정적으로 작동합니다.

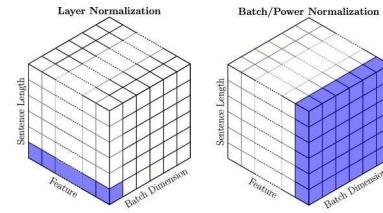
- **순환 신경망(RNN), Transformer에서 유리**: 시퀀스 길이가 다르거나 배치 크기가 유동적인 환경에서 안정적으로 학습이 가능합니다.

I. Deep learning(선수지식)

8. Normalization : 데이터의 분포를 조정하는데 사용(layer normalization).

Layer Normalization vs Batch Normalization





I. Deep learning(선수지식)

8. Normalization : 데이터의 분포를 조정하는데 사용(layer normalization).

Layer Normalization

1. 히든 레이어와 값의 변화

- 히든 레이어는 입력에 가중치(weight)와 편향(bias)을 적용하고, 활성화 함수를 거쳐 출력(feature vector)을 만듭니다.
- Layer Normalization은 바로 이 히든 레이어 출력(feature vector)에 적용됩니다.
(=> **모든 노드의 출력값(=은닉 표현 벡터)**)
- 정규화 기준은 **한 샘플 내부의 feature 축 전체**입니다.
- 즉, 해당 시점의 출력값들을 평균 0, 분산 1로 맞추습니다.

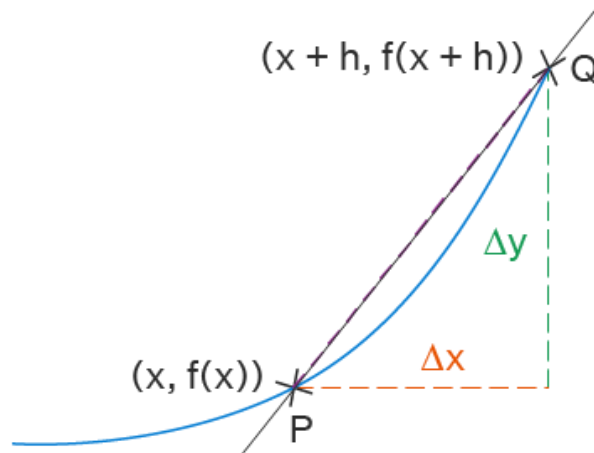
2. 목적

- 은닉 표현의 분포가 계속 변하면(Internal Covariate Shift) 학습이 불안정해집니다.
- Layer Normalization은 이를 안정적인 분포로 조정하여
 - 학습 안정화
 - 기울기 흐름 개선
 - 배치 크기와 무관한 학습 가능성을 보장합니다.

I. Deep learning(선수지식) 미분과 적분

- 9.미분(Differentiation): 미분은 함수의 변화율을 나타내는 개념.
함수의 도함수를 계산하여 **어떤 지점에서의 순간 변화율**을 구할 수 있다.
머신러닝에서는 손실 함수의 기울기(그래디언트)를 계산하고,
이를 최적화 알고리즘에 활용하여 모델을 학습.

Differentiation



$$\begin{aligned}\text{Slope} &= \frac{\Delta y}{\Delta x} \\ &= \frac{f(x + h) - f(x)}{(x + h) - x} \\ &= \frac{f(x + h) - f(x)}{h}\end{aligned}$$

I. Deep learning(선수지식)

9.미분(Differentiation): 어떤 지점에서의 순간 변화율.

Differentiation Rules



상수
미분

Constant Rule

$$\frac{d}{dx} [C] = 0$$

지수
미분

Power Rule

$$\frac{d}{dx} x^n = nx^{n-1}$$

합성함수
미분

Product Rule

$$\frac{d}{dx} [f(x)g(x)] = f'(x)g(x) + f(x)g'(x)$$

분수함수
미분

Quotient Rule

$$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{g(x)f'(x) - f(x)g'(x)}{[g(x)]^2}$$

합성함수
(연쇄법칙)

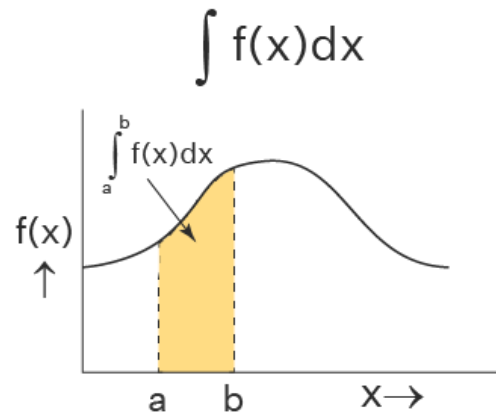
Chain Rule

$$\frac{d}{dx} [f(g(x))] = f'(g(x)) g'(x)$$

I. Deep learning(선수지식)

10. 적분(Integration): 적분은 함수의 **면적**을 구하는 개념.
머신러닝에서는 적분을 사용하여 확률 분포 함수의 면적 계산이나
변수 간의 상관 관계 계산 등에 활용될 수 있다.

Definite Integral



$$\int_a^b f(x)dx = \text{The Area under the curve between the points a and b.}$$
$$= f(b) - f(a)$$

I. Deep learning(선수지식)

11. 연쇄 법칙(Chain Rule): **합성함수(composite function)를 미분**하는 규칙.
합성함수란, 함수 f 와 g 가 있을 때, $f(g(x))$ 와 같이 하나의 함수 안에 또 다른 함수가 있는 경우.

 The Chain Rule

If $y = f(u)$, where $u = g(x)$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

© Maths at Home www.mathsathome.com

 The Chain Rule

For $F(x) = f(g(x))$

$$F'(x) = f'(g(x)) \cdot g'(x)$$

Derivative of outer function Derivative of inner function

© Maths at Home www.mathsathome.com

I. Deep learning(선수지식)

11. 연쇄 법칙(Chain Rule): **합성함수(composite function)를 미분**하는 규칙.

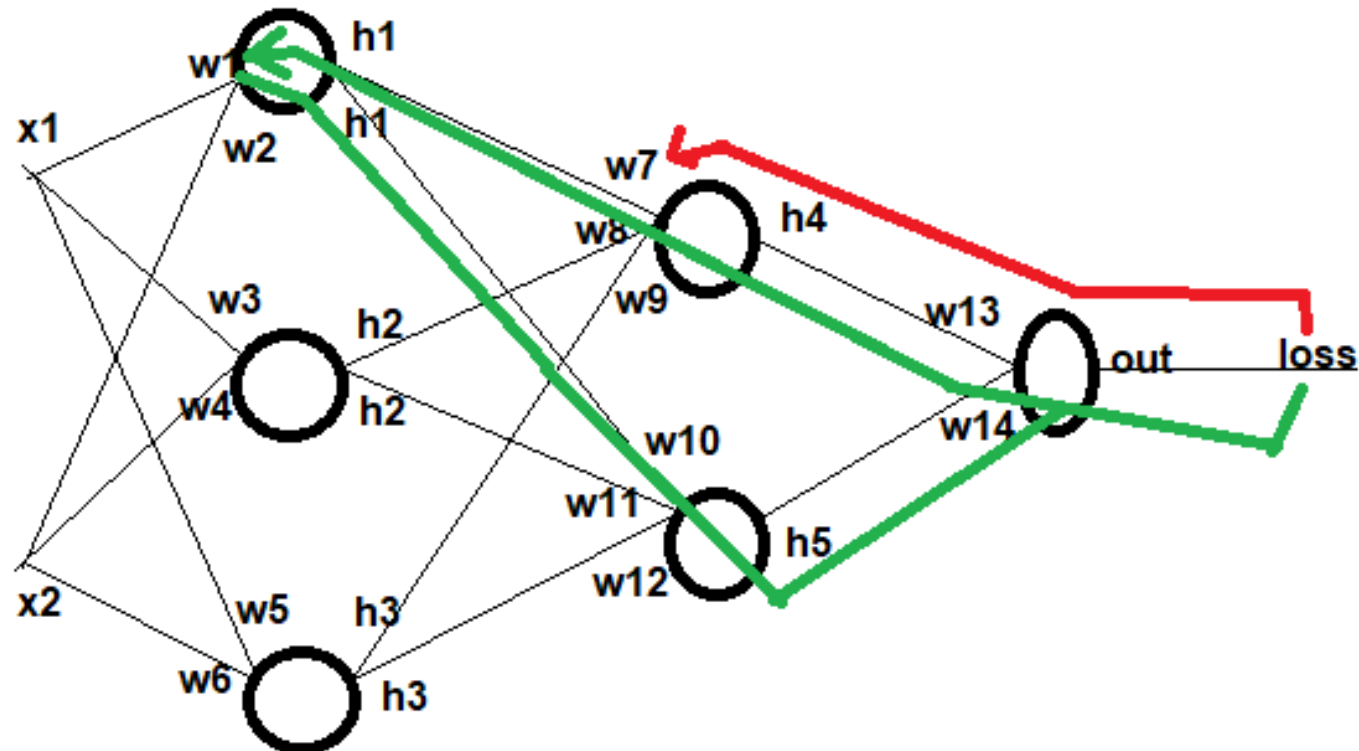
 Differentiate $f(x) = (4x - 3)^5$

$$\frac{dy}{dx} = \left(\begin{array}{c} \text{Differentiate} \\ \text{outer function} \\ \text{Keep the inside} \\ \text{the same} \end{array} \right) \left(\begin{array}{c} \text{Differentiate} \\ \text{inner function} \end{array} \right)$$
$$f(x) = (4x - 3)^5$$

© Maths at Home www.mathsathome.com

I. Deep learning(선수지식)

11. 연쇄 법칙(Chain Rule): 딥러닝에서 오차함수의 최적화를 위해 역전파를 시행할 때 연쇄법칙에 의한 미분값(가중치)를 계산하게 된다



I. Deep learning(선수지식)

11. 연쇄 법칙(Chain Rule): 딥러닝에서 오차함수의 최적화를 위해 역전파를 시행할 때 연쇄법칙에 의한 미분값(가중치)를 계산하게 된다

각 층 = 양파의 껍질: **딥러닝의 각 층은** 양파의 껍질과 같이 순차적으로 연결

함수 합성 = 양파 성장: 각 층은 이전 층의 출력을 입력으로 받아 새로운 출력을 만들어내는 함수와 같다.

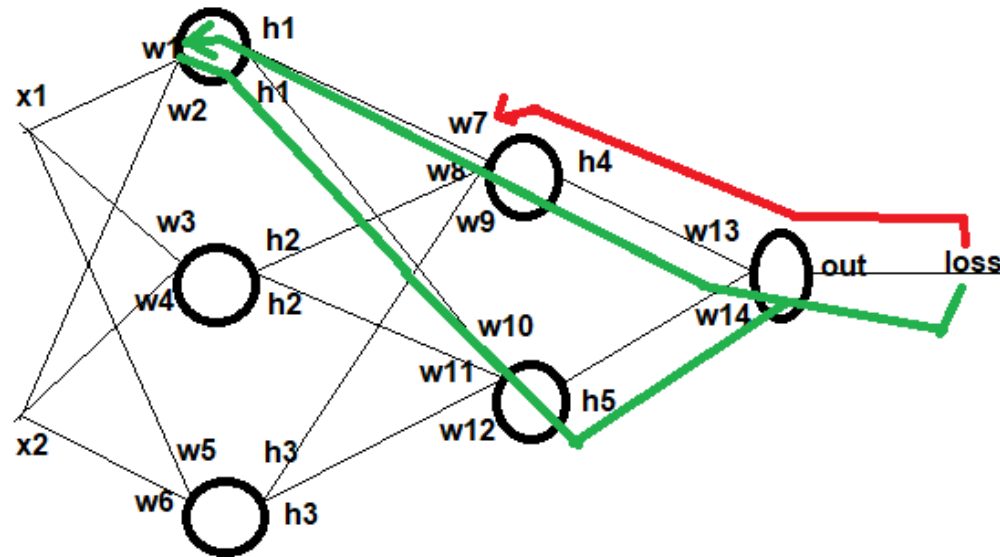
미분 = 영향력 분석: 최종 결과에 대한 **각 층의 영향력을 분석**하는 것이 미분의 목표

체인 룰 = 껍질 벗기며 영향력 전파: 가장 바깥쪽 껍질부터 시작하여 안쪽 껍질로 순차적으로 이동하며 각 껍질의 영향력을 계산하고, 이를 곱하여 최종 결과에 대한 각 층의 전체적인 영향력을 파악하는 방식

이는 양파 껍질을 하나씩 벗겨나가며 각 껍질이 전체 크기에 얼마나 기여했는지 분석하는 것과 논리적으로 같습니다.

I. Deep learning(선수지식)

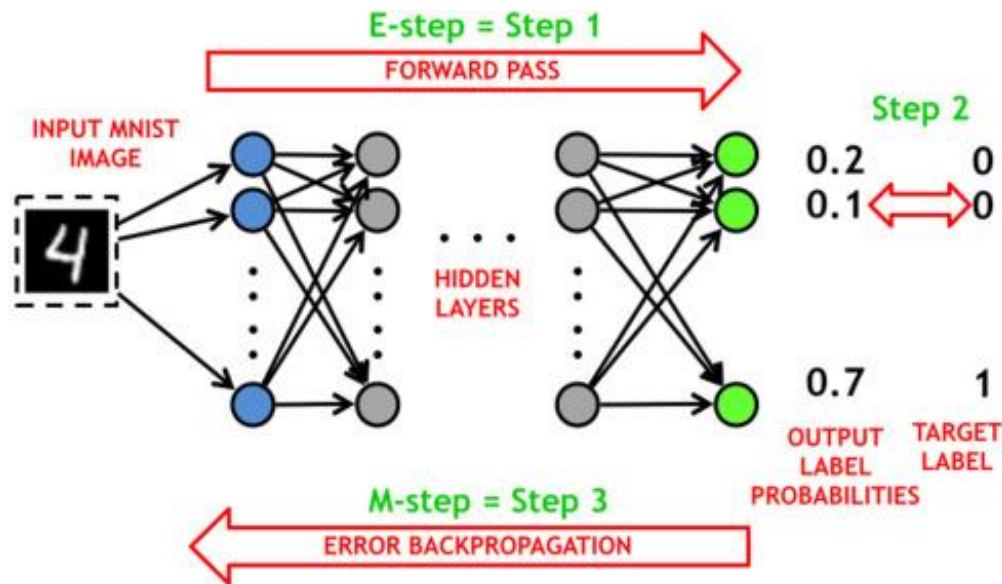
11. 연쇄 법칙(Chain Rule): 딥러닝에서 오차함수의 최적화를 위해 역전파를 시행할 때 연쇄법칙에 의한 미분값(가중치)를 계산하게 된다



I. Deep learning(선수지식)

12. 순전파와 역전파

모델의 예상결과와 실제 결과의 차이(Loss)를 옵티마이저를 통해 최적화하는 과정
(Feed forward, back propagation)



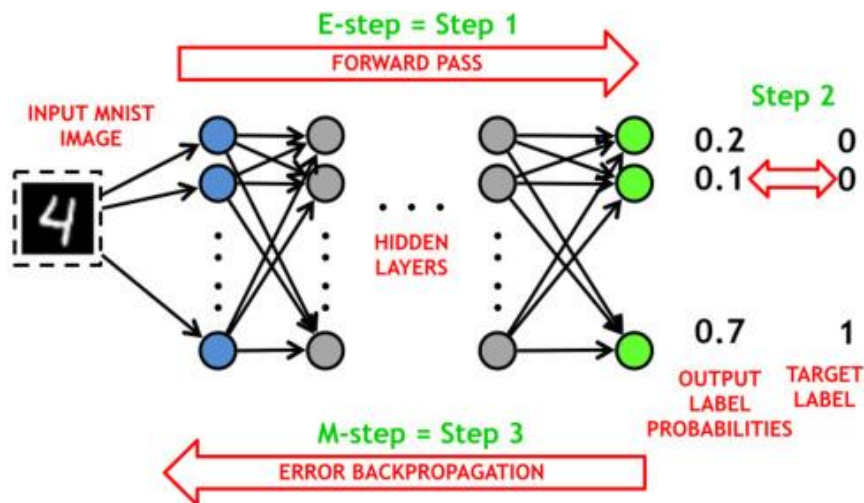
I. Deep learning(선수지식)

13. 에포크(epochs) :

준비된 데이터가 한 번씩 **순전파(forward pass)**와 **역전파(backward pass)**를 통해 학습되는 과정을 포함.

이 과정을 통해 모델의 가중치가 업데이트되고, 에포크를 반복하여 모델의 성능을 향상.

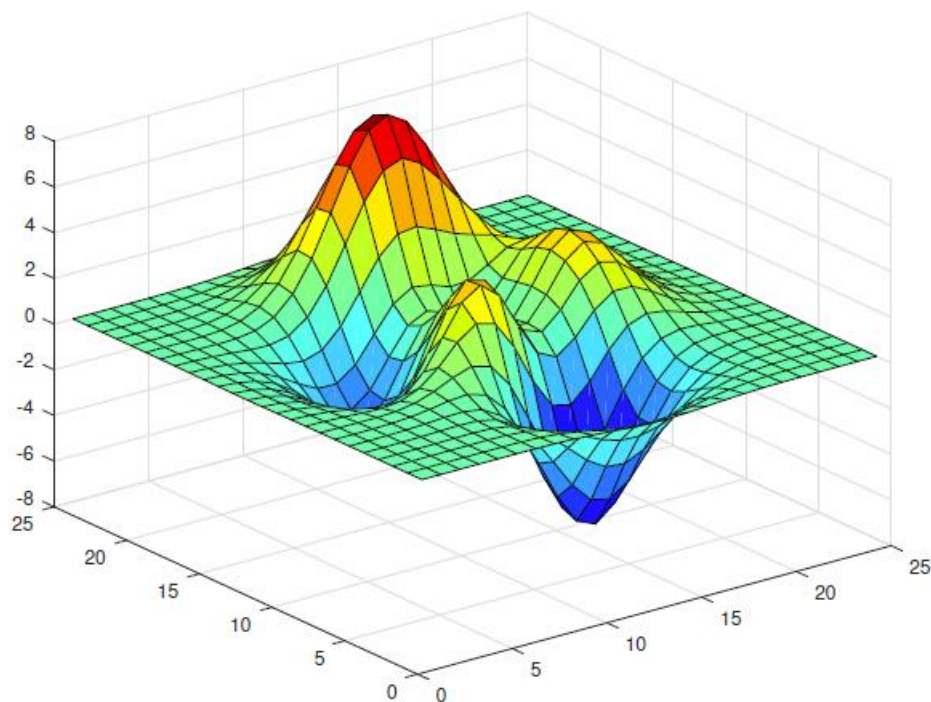
에포크의 수는 학습의 반복 횟수이며, 학습 시간과 모델의 성능에 큰 영향을 준다



I. Deep learning(선수지식)

14. 최적화(Optimization)

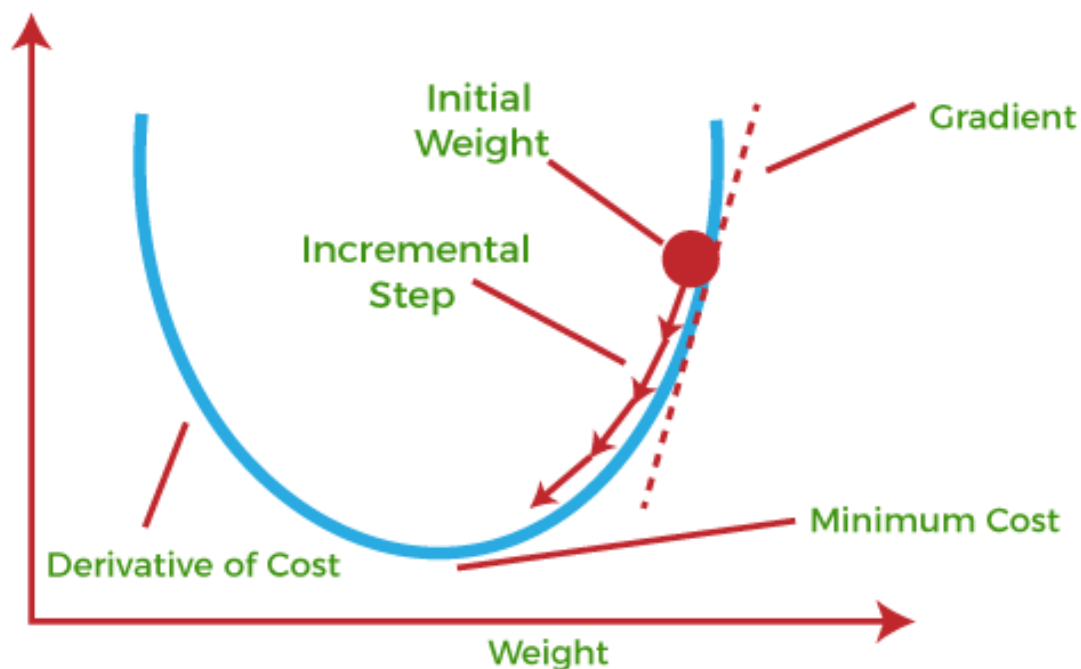
목적함수(손실함수)의 **최솟값이나 최댓값을 찾는 과정**. 머신러닝에서는 모델의 손실 함수를 최소화하는 매개변수 값을 찾기 위해 최적화 알고리즘을 사용.
최적화 알고리즘은 **경사하강법(Gradient Descent)**과 그 변형들이 주로 사용.



I. Deep learning(선수지식)

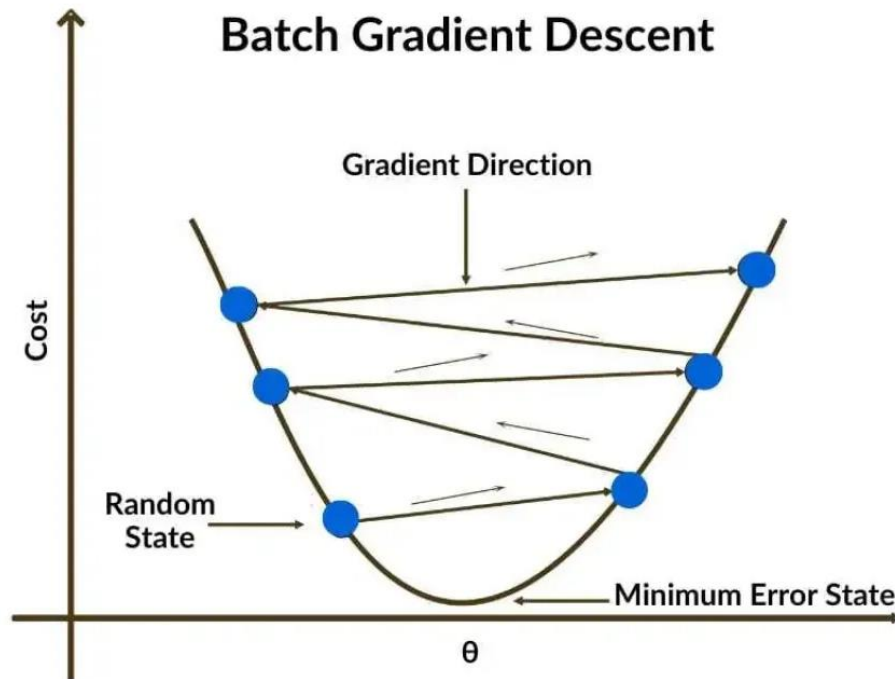
15.경사하강법(Gradient Descent):

최적화 알고리즘 중 가장 기본적이고 널리 사용되는 방법.
목적 함수의 기울기(그래디언트)를 사용하여 매개변수를 반복적으로 조정하면서
목적 함수의 결과값을 최소화하는 방향으로 진행.

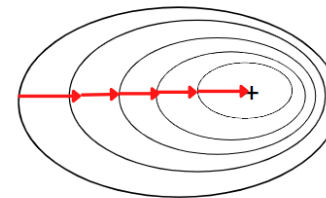


I. Deep learning(선수지식)

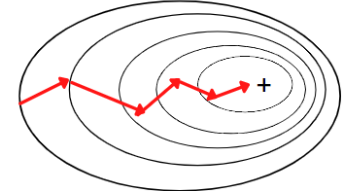
15.경사하강법(Gradient Descent):



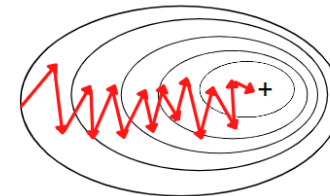
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



I. Deep learning(선수지식)

실습) 경사하강법을 최대한 자세히 정리하고 공유하세요

I. Deep learning(선수지식)

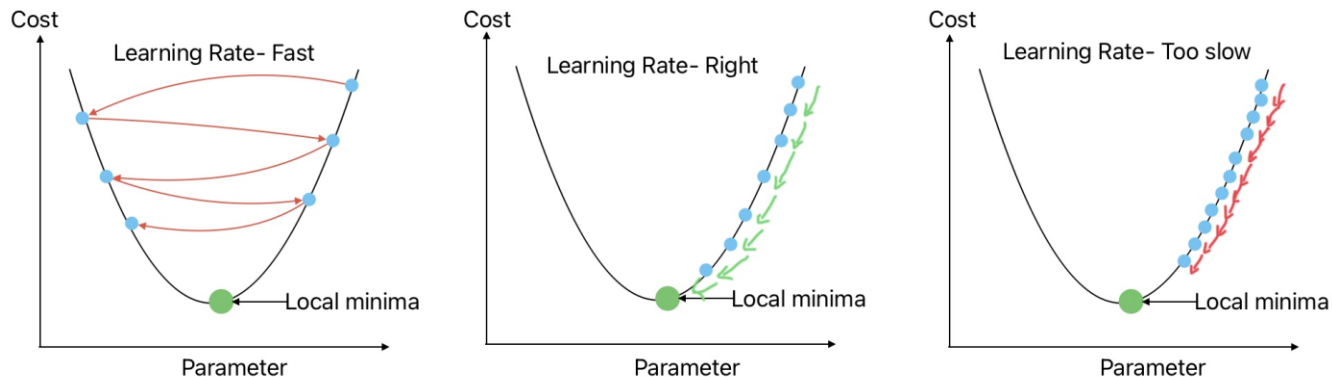
16. 학습률(learning rate): 보통 0.001 또는 $1e-3$

학습률은 모델이 학습할 때마다 얼마나 크게 또는 작게 자신의 지식(모델의 파라미터, 즉 가중치와 편향)을 업데이트할지를 조절하는 변수.

산을 내려갈 때 '보폭'과 같다.

너무 큰 보폭은 최저점을 지나칠 수 있고 너무 작으면 시간이 많이 걸린다.

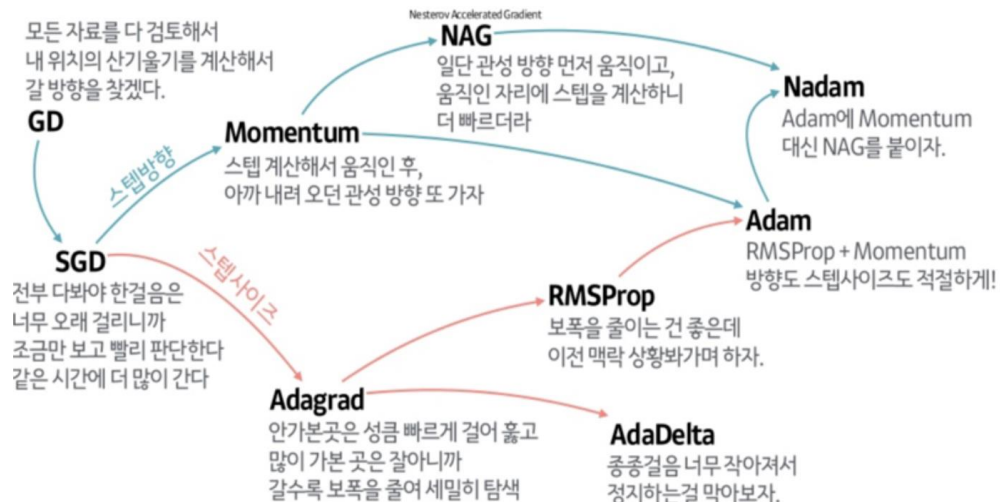
Impact of Step Size or learning rate on Gradient Decent Performance



I. Deep learning(선수지식) optimizer

17. 옵티마이저(optimizer)

모델의 가중치와 편향을 최적화하는 역할. 옵티마이저는 **손실 함수를 최소화**하기 위해 **가중치와 편향을 업데이트**하는 방식을 결정하며, **학습 속도와 수렴 품질에 영향**.



I. Deep learning(선수지식) **optimizer vs 등산**

1) 확률적 경사 하강법(SGD : Stochastic Gradient Descent):

- 등산자가 각 단계마다 **가장 가파른 경사를 따라 내려오는 것**과 비슷합니다.
- 경사면이 가파르기 때문에 빠르게 하산할 수 있지만, 직진하지 않고 좌우로 흔들리거나 노이즈에 민감할 수 있습니다.

2) 모멘텀 최적화(Momentum Optimization):

- 등산자가 **이전에 움직인 방향과 속도를 기억하면서 내려오는 것**과 비슷합니다.
- 이전에 이동한 방향과 크기를 고려하기 때문에 경사면이 완만한 지역에서도 좀 더 일정한 속도로 내려오며, 지역 최소점에서 빠져나올 수 있습니다.

3) 아담(Adam: Adaptive Moment estimation):

- 등산자가 학습 경로에서 학습률을 조정하면서 내려오는 것과 비슷합니다.
- 등산자는 학습 경로에서 **현재 위치에서의 기울기를 계산하고, 이동 평균을 사용하여 학습률을 동적으로 조절**합니다.
- 학습 경로의 특성에 따라 학습률을 적절하게 조절하면서 목표 지점으로 향하며, 안정적으로 수렴할 수 있습니다.

4) 애다그라드(Adagrad: Adaptive Gradient Algorithm):

- 등산자가 경사면의 특성에 따라 **학습률을 조정하면서** 내려오는 것과 비슷합니다.
- 경사면이 높은 지점에서는 학습률을 감소시켜 **더 조심스럽게** 이동하고, 경사면이 낮은 지점에서는 학습률을 증가시켜 **더 빠르게** 이동합니다

I. Deep learning(선수지식) **optimizer vs 등산**

```
import tensorflow as tf

# 학습률 설정 (예시: 0.001)
learning_rate = 0.001

# Adam 옵티마이저를 사용하여 학습률 적용
optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)

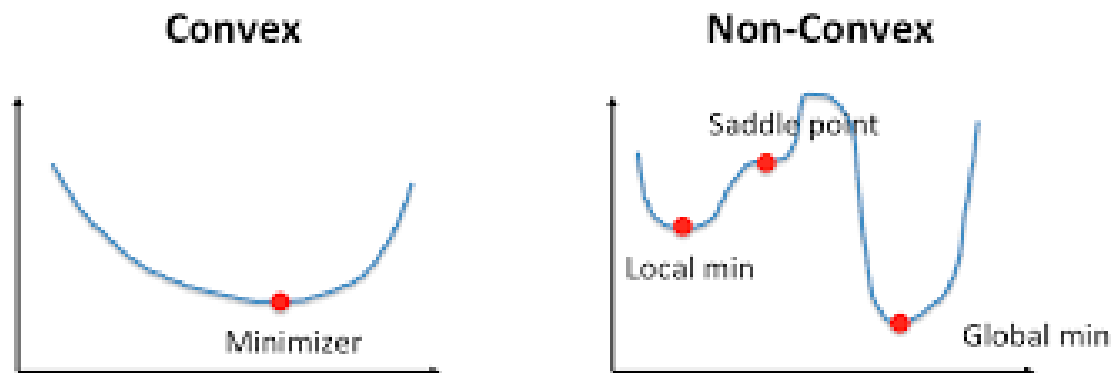
# 모델 컴파일 시 옵티마이저 지정
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])
```

I. Deep learning(선수지식)

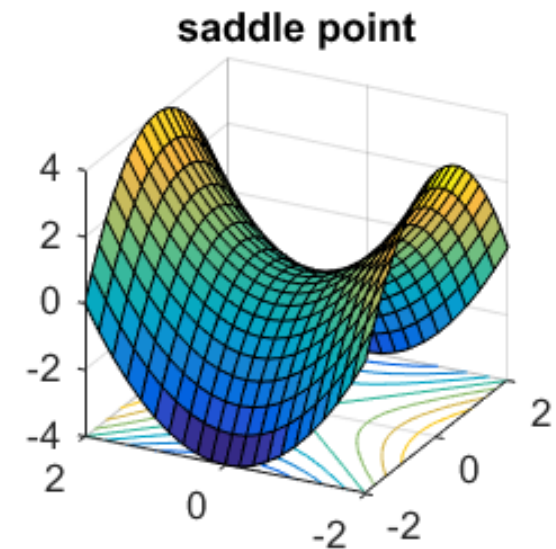
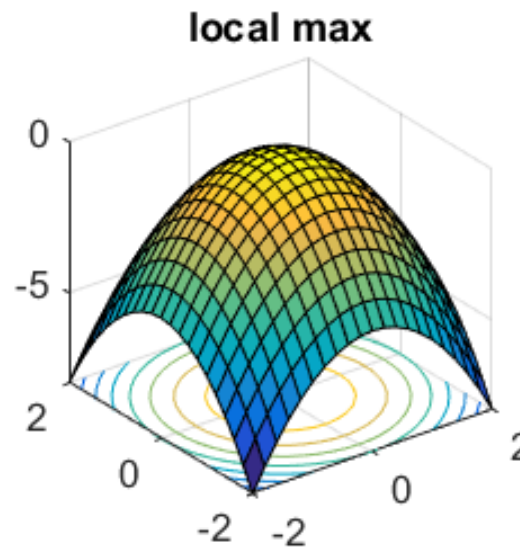
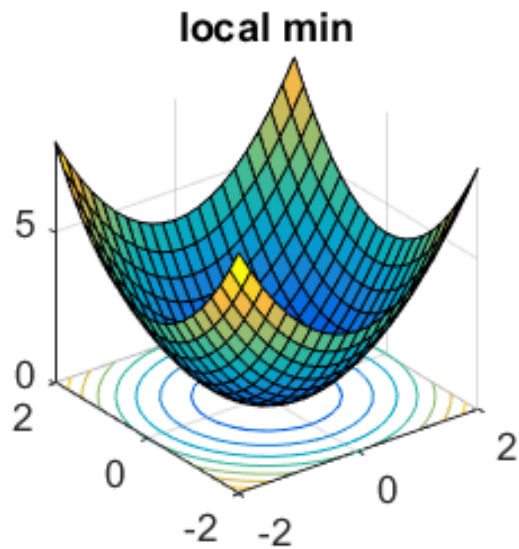
18.볼록 최적화(Convex Optimization):

목적 함수가 **볼록 함수**인 경우에 적용되는 최적화 방법.

볼록 함수는 특정 조건을 만족하며, 이러한 함수에서의 최소값은 전역 최소값.



I. Deep learning(선수지식)



I. Deep learning(선수지식)

19. 활성화 함수(activation function)

딥러닝에서 활성화 함수(Activation Function)는 뉴런의 출력 값을 결정하는 함수
뉴런의 입력에 대해 **어떤 값이 출력될지 결정('활성화')**

- 1) 딥러닝 모델이 **비선형 문제를 해결**하는 데 중요한 역할
비선형 활성화 함수를 사용함으로써 모델은 **더 복잡한 패턴을 학습**할 수 있다.(Relu)
- 2) **출력 범위 제한**: 뉴런의 출력값을 특정 범위 (예: 0과 1 사이, -1과 1 사이)로 제한하여
모델의 안정적인 학습을 돕습니다. 예를 들어, **이미지가 특정 클래스에 속할 확률**을 나타낼
때 0과 1 사이의 값으로 출력하는 데 유용합니다.(sigmoid, softmax)
- 3) **의사 결정**: 특정 임계값을 기준으로 출력을 조절하여 **뉴런이 활성화될지 여부**를
결정합니다. 이는 모델이 중요한 특징을 선택하고 무시할 특징을 걸러내는 데 도움을
줍니다.

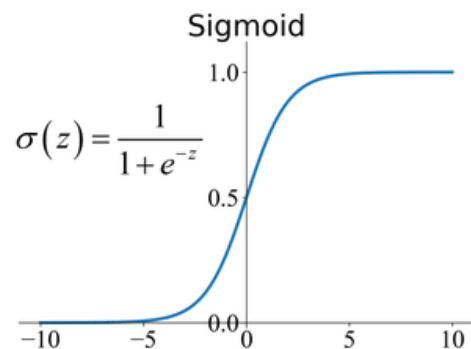
I. Deep learning(선수지식)

19. 활성화 함수

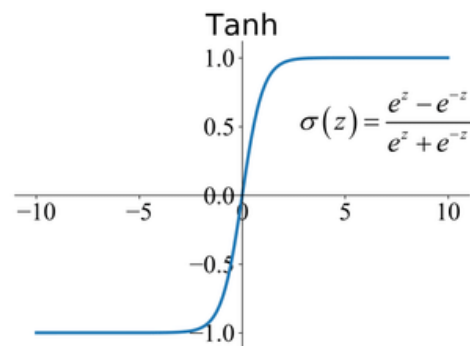
- 1) 시그모이드(Sigmoid) 함수:** 함수의 출력 값은 0과 1 사이.
이진 분류 문제의 출력층에서 사용. 그래디언트 소실 문제 발생.
- 2) 하이퍼볼릭 탄젠트(tanh) 함수:** 출력 범위가 -1에서 1사이
시그모이드 함수보다 더 넓은 출력 범위.
- 3) 렐루(ReLU) 함수:** 입력이 양수일 때는 입력 값을 그대로, 음수일 때는 0을 반환.
간단하면서 효과적인 함수로, 현재 가장 널리 사용
- 4) Leaky ReLU:** ReLU의 변형, 음수 입력에 대해 아주 작은 기울기를 가진다
ReLU의 "죽은 뉴런" 문제를 해결할 수 있다.
- 5) 소프트맥스(Softmax) 함수:** 다중 클래스 분류 문제의 출력층에서 사용
각 클래스에 대한 확률을 출력
각 클래스에 대한 출력값들의 합은 1

I. Deep learning(선수지식)

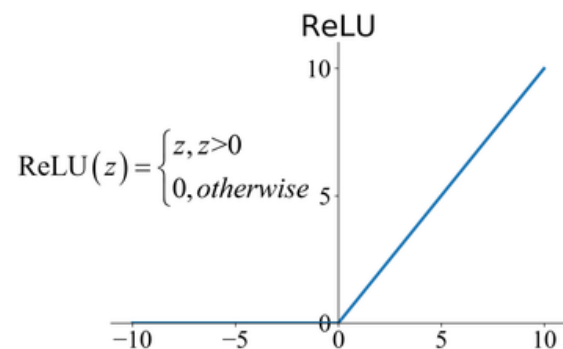
19. 활성화 함수



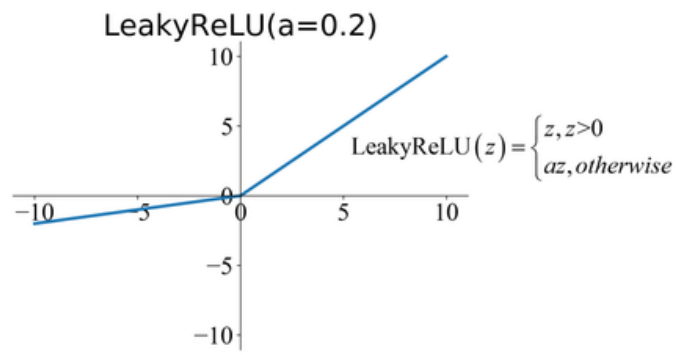
(a)



(b)



(c)

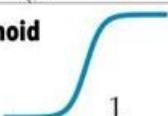
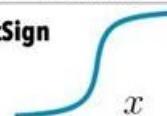
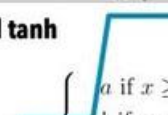
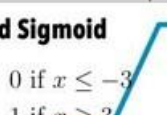
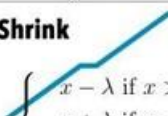
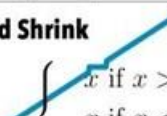


(d)

I. Deep learning(선수지식)

19. 활성화 함수

Neural Network Activation Functions: a small subset!

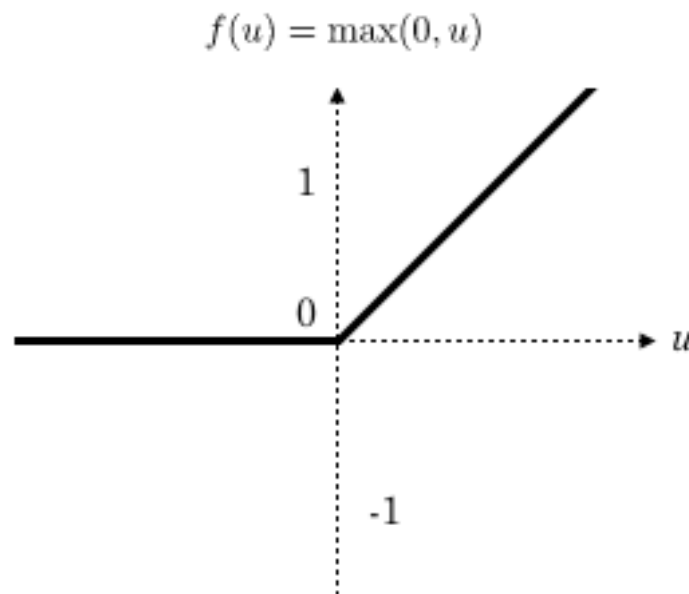
ReLU  $\max(0, x)$	GELU  $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

I. Deep learning(선수지식) Relu

19. Relu 함수 : 활성화 함수

가장 많이 사용되는 활성화 함수, 입력값이 0보다 작으면 0으로 출력하고, 0보다 크면 그 값을 그대로 출력. **음수 값을 제거하고 양수 값을 유지**하는 역할.

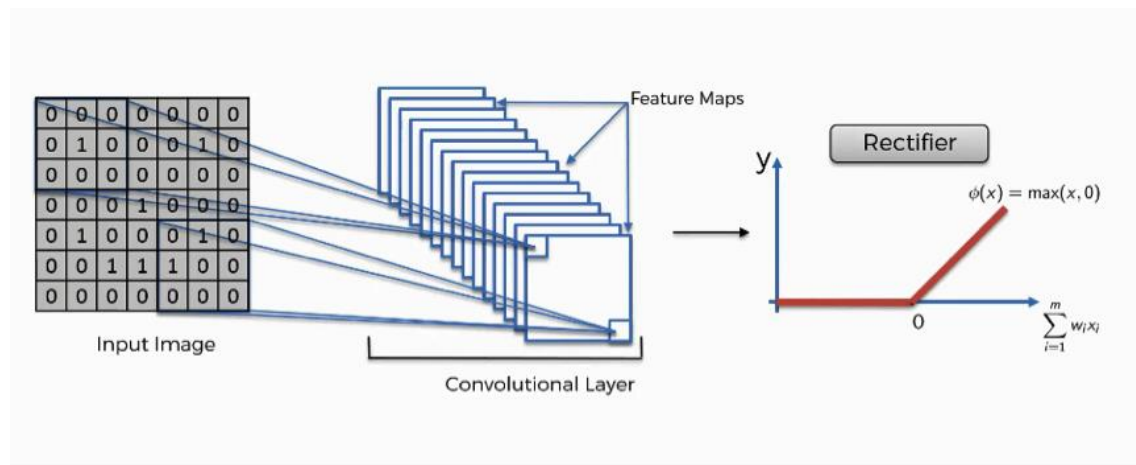
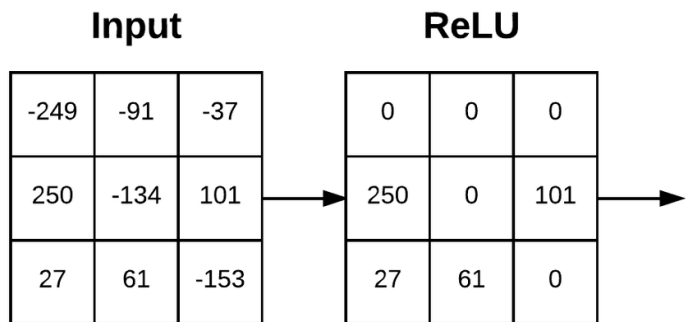
$$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$



I. Deep learning(선수지식) **Rulu**

19. Relu 함수

함수는 **비선형성을 추가하여** CNN이 복잡한 패턴과 특징을 학습할 수 있도록 도와준다.



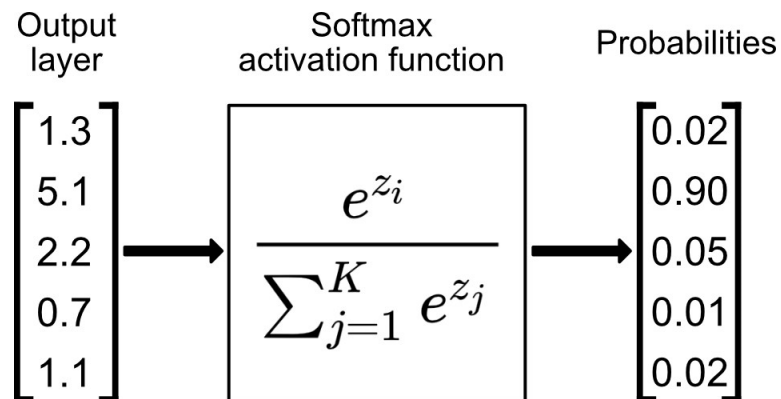
I. Deep learning(선수지식)

19. SoftMax

신경망의 출력층에서 사용되며, 그 결과로 확률 분포를 생성
분류 문제에서 **클래스의 확률을 예측**하는 데 사용

소프트맥스 함수의 핵심은 입력받은 값(logit)을 확률로 변환
모든 입력 값을 지수 함수를 사용하여 '확률'로 변환, **확률들의 합이 1**
각 클래스에 대한 확률 분포를 얻을 수 있다.

- 1.범위: 소프트맥스 함수의 출력은 항상 0과 1 사이, 모든 출력 값의 합은 항상 1
- 2.비선형성: 소프트맥스 함수는 비선형 함수.



I. Deep learning(선수지식)

19. SoftMax

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
# 간단한 순차 모델 정의
```

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)), # 입력층: 784개의 입력, 64개의 뉴런, ReLU 활성화 함수 사용
    Dense(10, activation='softmax') # 출력층: 10개의 뉴런 (클래스 개수), Softmax 활성화 함수 사용 (다중 분류)
])
```

I. Deep learning(선수지식) (**ANN vs DNN**)

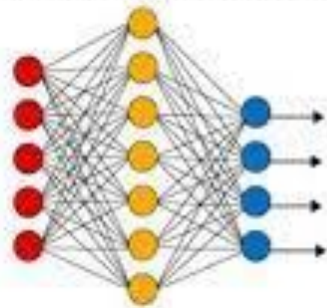
실습) optimizer, 활성화함수를 정리하고 공유하세요

I. Deep learning(선수지식) (ANN vs DNN)

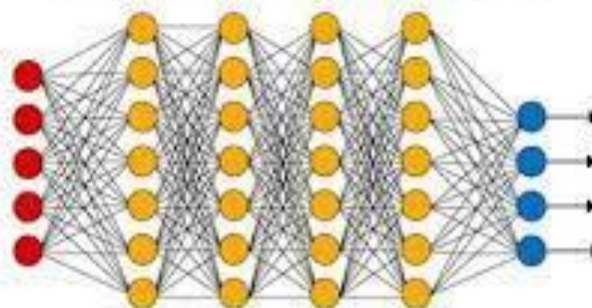
20.완전 연결층(Fully Connected Layer) : FC

인공 신경망에서 가장 기본적인 층, **모든** 입력 뉴런과 출력 뉴런이 **서로 연결되어** 있는 층. 각 입력 뉴런은 출력 뉴런과 **가중치로 연결되어** 있으며, 이 가중치는 학습 과정에서 조정.

Simple Neural Network



Deep Learning Neural Network



● Input Layer

● Hidden Layer

● Output Layer

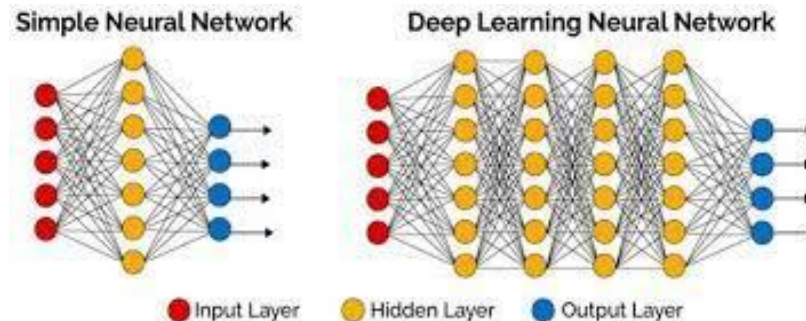
I. Deep learning(선수지식) (ANN vs DNN)

완전 연결층(Fully Connected Layer)

각 뉴런 사이의 연결은 **선형 연산**과 **활성화 함수**를 거쳐 진행된다.

선형 연산은 입력 뉴런의 값과 해당 연결의 **가중치**를 곱한 뒤, 모든 연결의 결과를 합산. 그리고 이 합산 결과는 활성화 함수로 전달되어 **출력값**을 생성. 모든 입력 뉴런과 출력 뉴런 사이의 연결에 대해 반복되며, 출력 뉴런의 최종값이 예측 결과로 사용.

**완전 연결층은 입력과 출력 사이의 가중치를 학습하여
입력 특성과 클래스 사이의 복잡한 관계를 모델링.
입력 데이터를 분류하거나 예측하는 작업을 수행**



I. Deep learning(선수지식)(ANN vs CNN)

21. 배치 (Batch), 드롭아웃 (Dropout), 조기종료(Early Stopping)

모델의 효과적 학습을 돕는 여러 가지 기술들

배치(Batch): 데이터를 한 번에 조금씩 묶어서 학습시키는 방법.
마치 학생이 문제집을 한 페이지씩 풀어나가는 것.

드롭아웃(Dropout): 학습 과정에서 일부 뉴런을 무작위로 '잠시 꺼서' 모델이 특정 부분에만 너무 의존하는 것을 방지.

조기 종료(Early Stopping): 학습이 더 이상 개선되지 않으면 중간에 멈추는 방법

```
class BinaryClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(20, 64),
            nn.ReLU(),
            nn.Dropout(0.3), # Dropout 30%
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Dropout(0.3), # Dropout 30%
            nn.Linear(32, 2) # 이진 분류 (2 클래스)
        )

    def forward(self, x):
        return self.net(x)
```


I. Deep learning(선수지식)(ANN vs CNN)

21. 배치 (Batch), 드롭아웃 (Dropout), 조기종료(Early Stopping)

3. 모델 컴파일

```
model.compile(optimizer='adam',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

4. 조기 종료 콜백 정의

```
early_stopping = EarlyStopping(monitor='val_loss', # 모니터링할 지표 (검증 손실)  
                               patience=5, # 개선이 없으면 5 에포크 동안 기다림  
                               restore_best_weights=True) # 가장 좋은 가중치로 복원
```

5. 배치 크기 설정

```
batch_size = 128
```

6. 모델 학습 (배치 학습, 드롭아웃, 조기 종료 적용)

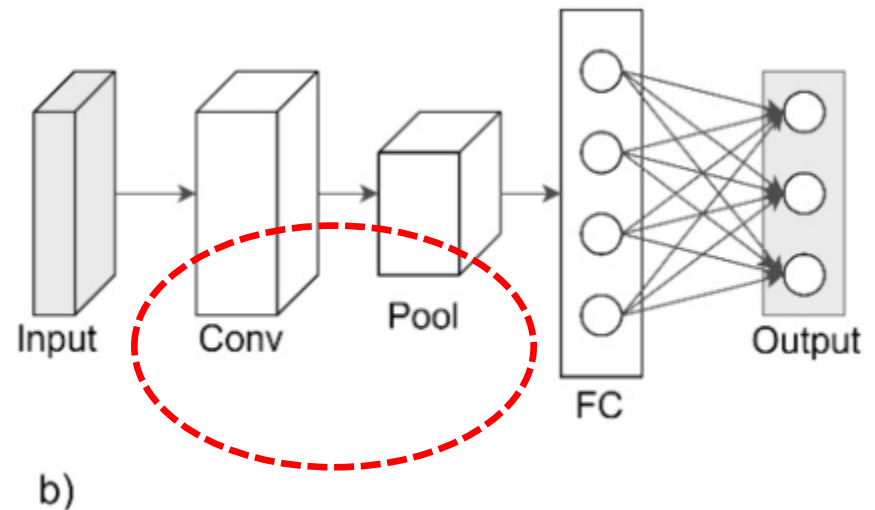
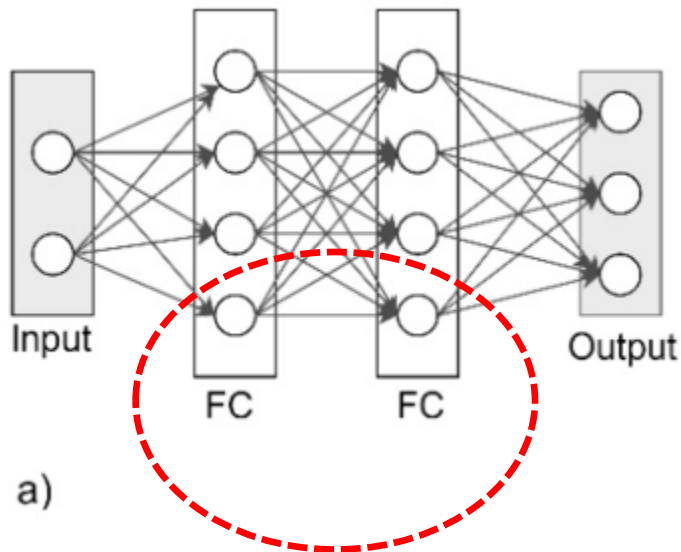
```
history = model.fit(x_train, y_train,  
                   batch_size=batch_size,  
                   epochs=100, # 충분히 큰 에포크 수 설정  
                   validation_split=0.2, # 검증 데이터 비율 (20%)  
                   callbacks=[early_stopping]) # 콜백 리스트에 조기 종료 추가
```

7. 모델 평가

```
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)  
print(f'테스트 손실: {loss:.4f}')print(f'테스트 정확도: {accuracy:.4f}')
```

I. Deep learning(선수지식)(ANN vs CNN)

22. ANN(FC)vs.CNN(Conv+Pool)



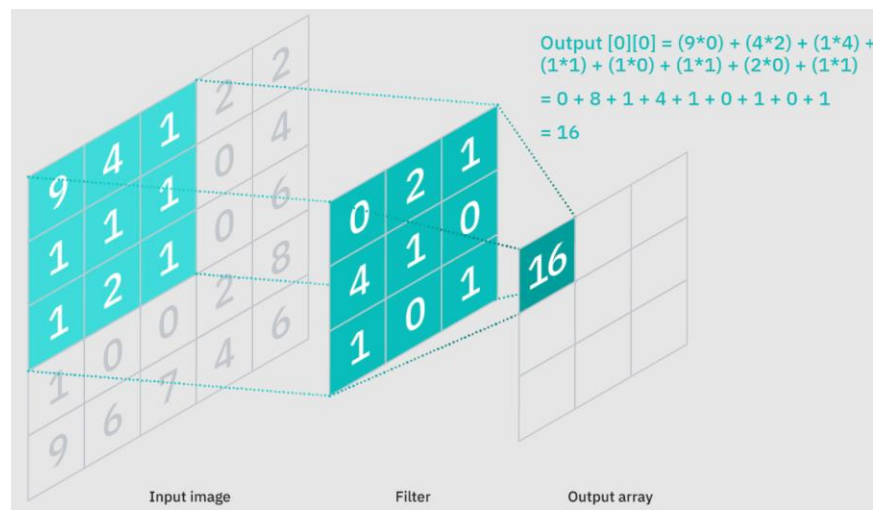
I. Deep learning(선수지식)

Convolution

이미지나 다차원 데이터에 대해 **작은 필터를 이용하여 합성곱 연산을 수행**하는 것.

필터는 입력 데이터의 일부 영역과 가중치를 곱한 값을 모두 더하여 출력을 생성. 필터를 이동하면서 전체 입력 데이터에 대해 **합성곱 연산을 수행하면 특징 맵(Feature Map)이 생성**.

Convolution은 입력 데이터의 **패턴을 감지하고, 이미지에서 특징을 추출**하는 역할을 수행. 또한, 합성곱 연산을 통해 필터의 가중치를 학습하여 이미지의 특징을 감지하는 더 나은 필터를 생성할 수도 있다.

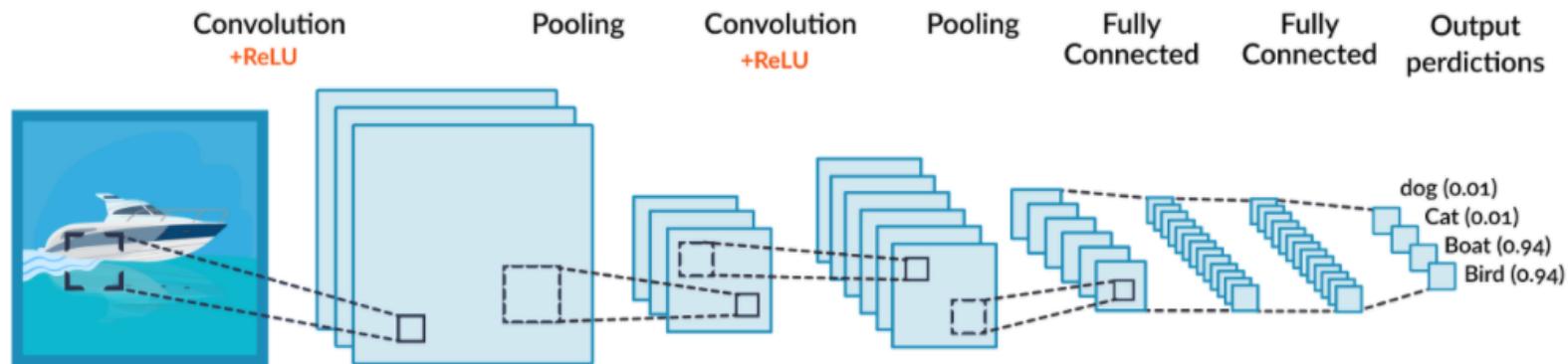


I. Deep learning(선수지식)(convolution의 이해)

22.CNN(Convolution Neural Network)

- convolution NN, 합성곱신경망

- 특히 이미지 및 비디오 처리에 활용
- 핵심 요소는 커널(**convolution** 연산시 입력 관련 기능 추출)이라는 필터
- 입력 데이터의 특징을 추출하여 특징들의 패턴 파악하는 구조
- Convolution과정과 Pooling 과정으로 진행



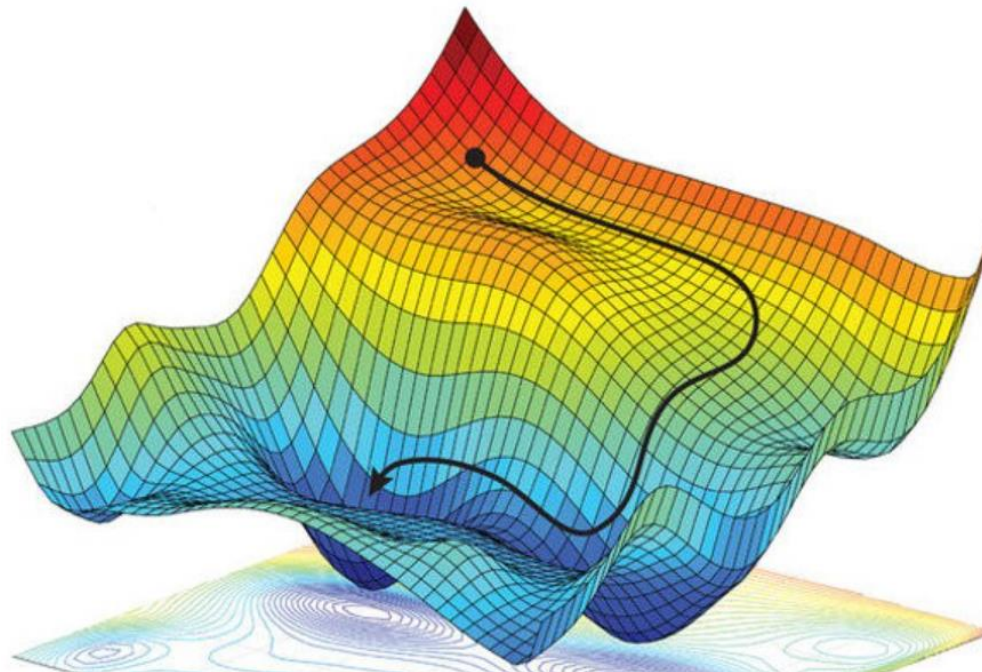
I. Deep learning(선수지식)

23.목적 함수(Objective Function):

Cost function 비용함수(sum of loss function),

loss function 손실함수(model complexity penalty for regularization),

error function 오차함수(for backpropagation)



I. Deep learning(선수지식)

23.목적 함수(Objective Function):

평균 제곱 오차 (Mean Squared Error, MSE):

실제 값과 예측 값 사이의 제곱 오차를 평균한 값. 회귀 문제에서 주로 사용.
큰 오차에 대해 패널티를 부여하고, 작은 오차에 대해 민감하게 반응.

교차 엔트로피 오차 (Cross Entropy Error, CEE):

분류 문제에서 예측 값과 실제 값의 차이를 측정.

이진 분류 : 이진 교차 엔트로피 오차(**Binary Cross Entropy**)가 사용되고,
다중 분류 : 범주형 교차 엔트로피 오차(**Categorical Cross Entropy**)가 사용.

I. Deep learning(선수지식)

23.목적 함수(Objective Function):

딥러닝에서 사용되는 주요 오차함수

sparse categorical crossentropy 희소 범주형 크로스엔트로피

다중 클래스 분류 문제에서 주로 사용되며,
모델이 각 클래스에 대한 확률 분포를 예측하도록 학습할 때 사용
정수 형태의 클래스 레이블을 입력으로 사용

softmax Cross Entropy Loss 소프트맥스 크로스 엔트로피

다중 클래스 분류 문제에서 사용
softmax 활성화 함수와 함께 사용. 각 클래스에 대한 확률 분포
one-hot 인코딩된 클래스 레이블을 입력으로 사용

I. Deep learning(선수지식) **loss function**

23. 딥러닝의 손실 함수(loss function)

모델의 예측값과 실제값 사이의 차이를 측정하는 역할.

이를 통해 모델은 실제값과 최대한 일치하는 예측값을 만들어낼 수 있다.

1) 평균 제곱 오차(Mean Squared Error, MSE):

- 예측값과 실제값 사이의 거리를 제곱하여 계산하기 때문에, 오차가 커질수록 패널티가 크게 부여됩니다.(회귀 문제)

2) 교차 엔트로피 손실(Cross-Entropy Loss):

- 실제값의 확률 분포와 예측값의 확률 분포 사이의 교차 엔트로피를 계산.
- 분류 문제 (MNIST)

3) 이진 교차 엔트로피 손실(Binary Cross-Entropy Loss):

- 이진 분류 문제에서 사용되는 손실 함수로, 두 확률 분포 간의 차이를 측정.
- 이메일 스팸 분류

4) 희소 교차 엔트로피 손실(Sparse Cross-Entropy Loss):

- 다중 클래스 분류 문제에서 사용되는 손실 함수.
- **정수형태로 인코딩된** 실제 레이블과 모델의 예측 확률 분포의 교차 엔트로피를 계산

I. Deep learning(선수지식) **loss function**

손실 함수(loss function)

1.평균 제곱 오차(Mean Squared Error, MSE): vs MAE/RMSE

1. 회귀 문제에서 자주 사용,
2. 예측값과 실제값 사이의 거리를 제곱하여 계산하기 때문에, 오차가 커질수록 패널티가 크게 부여됩니다.(회귀 문제)
3. 미분 계산의 편의성: 머신 러닝에서 모델의 학습은 주로 미분을 사용하여 손실 함수에 대한 모델의 파라미터의 그래디언트를 계산하는 과정으로 이루어 진다. MSE는 제곱 오차의 평균이므로 미분 계산 시 제곱 함수의 미분과 평균 함수의 미분이 서로 상쇄되는 특성을 가진다
4. 이상치의 영향 유무를 분석하고 오차 측정에서 이상치의 영향이 큰 경우 다른 오차 평가 모델을 선택. 이를 통해 모델의 신뢰성을 높이고 실제 데이터에 대한 적합성을 개선할 수 있다.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

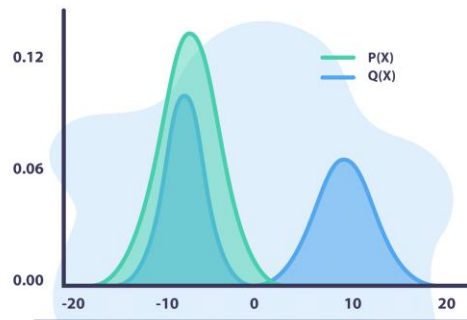
I. Deep learning(선수지식) **loss function**

손실 함수(loss function)

1. 평균 제곱 오차(Mean Squared Error, MSE):

2. 교차 엔트로피 손실(Cross-Entropy Loss): 이미지 분류, 자연어 처리, 생성 모델 등 다항분류에 사용

1. 엔트로피는 무질서를 나타내는 개념으로, 확률 분포에서의 불확실성이 얼마나 큰지를 측정
2. 교차 엔트로피 손실은 **두 확률 분포 간의 유사도를 측정**하는 방법 중 하나. **모델의 예측값과 실제값의 확률 분포를 비교하고 두 분포 사이의 차이를 계산**
3. 만약 모델의 예측값이 실제값과 일치한다면, 두 분포 사이의 차이가 작아지고 엔트로피가 낮아진다. 이는 모델이 더욱 확실한 예측을 수행하고 더욱 정확한 분류를 이룰 수 있게 된다.
4. 만약 모델의 예측값과 실제값 사이에 큰 차이가 있다면, 두 분포 사이의 차이가 커지고 엔트로피가 높아진다. 모델은 더욱 불확실한 예측을 수행하고 분류 결과가 더욱 오류가 발생할 가능성이 크다
5. 교차 엔트로피 손실을 사용하여 모델을 학습시키는 경우, 모델은 실제값과 예측값의 확률 분포를 최대한 비슷하게 만들도록 학습하여 모델이 더욱 정확하고 확실한 예측을 수행하도록 유도



Kullback-Leibler Divergence

I. Deep learning(선수지식) **loss function**

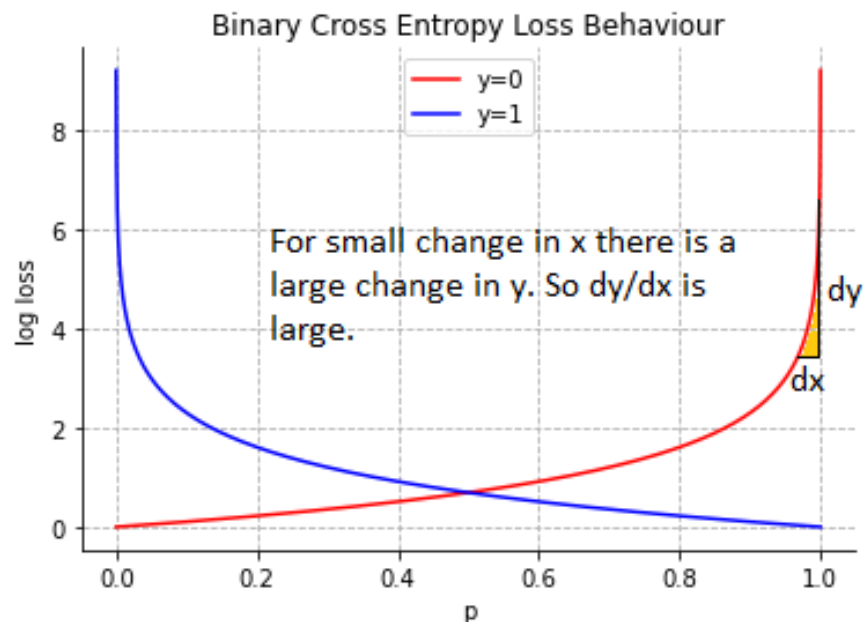
손실 함수(loss function)

1. 평균 제곱 오차(Mean Squared Error, MSE):

2. 교차 엔트로피 손실(Cross-Entropy Loss):

3. 이진 교차 엔트로피 손실(Binary Cross-Entropy Loss):

1. 두 확률 분포 간의 차이를 측정. 이메일 스팸 분류
2. "이진"은 두 가지 클래스(예를 들면, 스팸/일반 메일, 양성/음성 진단) 중 하나를 예측하는 문제.
"교차 엔트로피"는 예측값과 실제 레이블 사이의 엔트로피 차이를 계산하는 방법



I. Deep learning(선수지식) **loss function**

손실 함수(loss function)

1. 평균 제곱 오차(Mean Squared Error, MSE):
2. 교차 엔트로피 손실(Cross-Entropy Loss):
3. 이진 교차 엔트로피 손실(Binary Cross-Entropy Loss):

4. 희소 교차 엔트로피 손실(Sparse Cross-Entropy Loss):

1. 다중 클래스 분류 문제에서 사용되는 손실 함수.
2. **정수형태로 인코딩된 실제 레이블과 모델의 예측 확률 분포의 교차 엔트로피를 계산**
3. "희소"라는 단어는 실제 레이블의 확률 분포가 주로 하나의 클래스에 집중되어 있을 때를 의미

희소(sparse)란?

정답 라벨을 원-핫 벡터(one-hot vector)로 변환하지 않고,

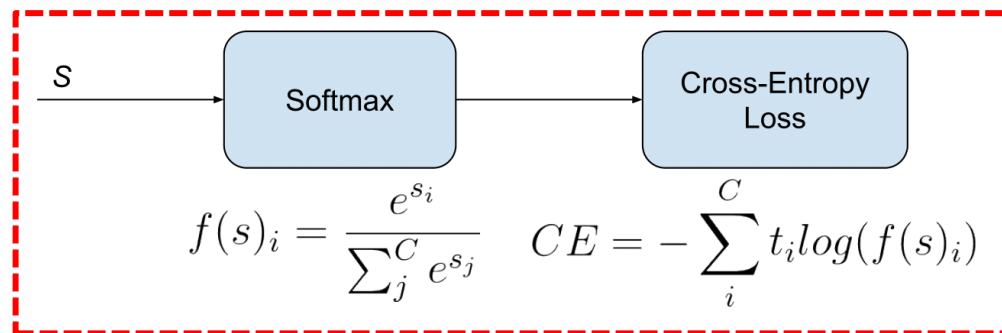
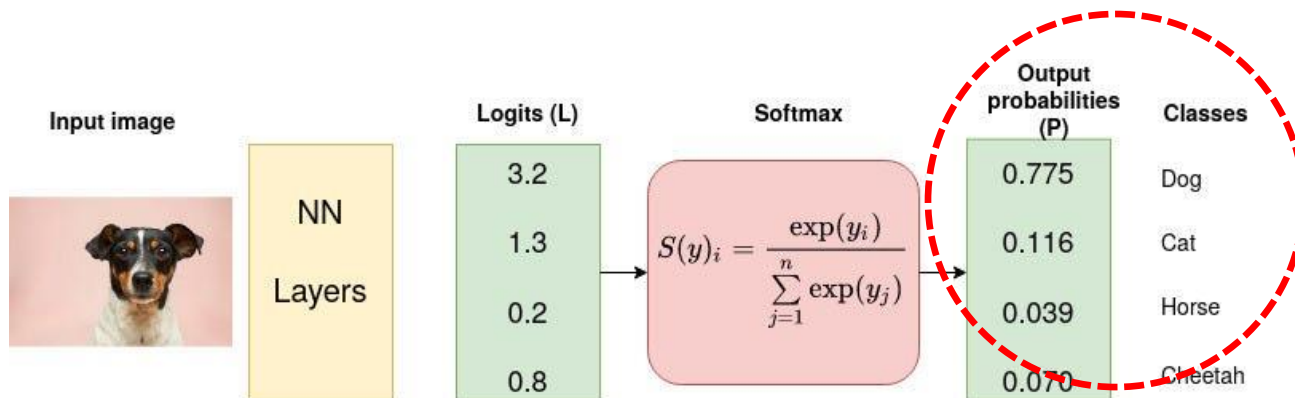
예: [0], [3]처럼 정수 인덱스로 직접 처리합니다.

→ 계산 속도와 메모리 사용을 줄여줍니다.

I. Deep learning(선수지식)

손실 함수(loss function)의 적용 프로세스

최종 활성화 함수의 예상결과와 실제 결과의 차이(Loss)를 옵티마이저를 통해 최적화



I. Deep learning(선수지식)

24.손실 함수(loss function) vs 최종 활성화 함수(final activation function)

Problem Type	Output Type	Final Activation Function	Loss Function
Regression	Numerical value	Linear	Mean Squared Error (MSE)
Classification	Binary outcome	Sigmoid	Binary Cross Entropy
Classification	Single label, multiple classes	Softmax	Cross Entropy

I. Deep learning(선수지식)

범주형 교차 엔트로피(Categorical Cross Entropy)와 선택 기준

손실 함수	설명	사용 사례	선택 기준
교차 엔트로피 (Cross-Entropy)	다중 클래스 분류 문제에서 사용되는 손실 함수.	이미지 분류, 자연어 처리 등 다중 클래스 분류 문제	다중 클래스 분류 문제인지 확인,
이진 교차 엔트로피 (Binary Cross-Entropy)	이진 분류 문제에서 사용되는 손실 함수. 두 개의 클래스 간의 확률 분포 차이를 측정.	이메일 스팸 분류, 감정 분석 등 이진 분류 문제	이진 분류 문제인지 확인, 각 클래스에 대한 확률을 독립적으로 계산하며, 클래스가 상호 배타적이지 않은 경우에 적합
희소 교차 엔트로피 (Sparse Cross-Entropy)	다중 클래스 분류 문제에서 원핫인코딩이 아닌 정수 형태로 레이블이 주어진 경우에 사용되는 손실 함수	단어 분류, 감정 분석 등 자연어의 다중 클래스 분류 문제	다중 클래스 분류 문제인지 확인, 클래스 레이블이 정수 형태로 주어진 경우
범주형 교차 엔트로피 (Categorical Cross-Entropy)	다중 클래스 분류 문제에서 원핫인코딩된 클래스 레이블을 사용하여 확률 분포 차이를 측정하는 손실 함수	이미지 분류, 자연어 처리 등 다중 클래스 분류 문제	다중 클래스 분류 문제인지 확인, 원핫인코딩 사용 여부 확인 , 클래스가 서로 독립적이고 상호 배타적인 경우에 적합

I. Deep learning(선수지식)

실습) 딥러닝의 손실함수를 정리하고 공유하세요

I. Deep learning(선수지식)

25.자연어처리 역사(RNN>Transformer)

1. 기본 RNN (Recurrent Neural Network)

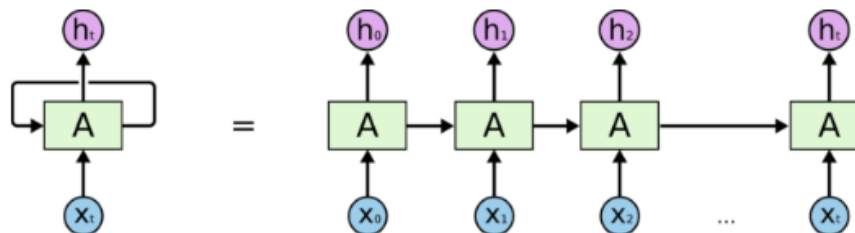
시퀀스 데이터를 처리하는 첫 딥러닝 구조
순차적으로 단어를 입력받고, 은닉 상태로 이전 정보를 기억

문제점

장기 의존성 문제: 앞쪽 정보가 뒤로 갈수록 사라짐 (Gradient Vanishing)

병렬화 불가능: 단어를 한 글자씩 순차적으로 처리해야 함

개선 방향 → LSTM / GRU 개발



An unrolled recurrent neural network.

I. Deep learning(선수지식)

25.자연어처리 역사(RNN>Transformer)

2. LSTM / GRU

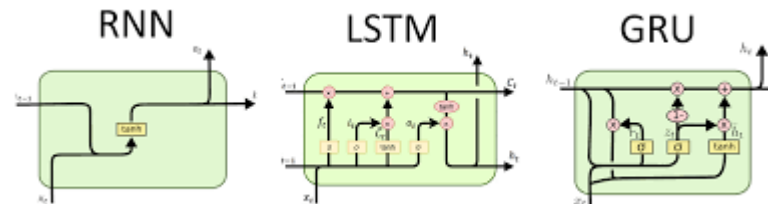
기억 게이트를 도입해 **중요 정보는 오래 기억**, 덜 중요한 건 잊음
RNN의 장기 의존성 문제 완화

문제점

문장 전체를 하나의 벡터에 담는 구조는 여전히 유지

→ 정보 압축 손실, 여전히 순차 처리만 가능

개선 방향 → Seq2Seq 모델로 확장



I. Deep learning(선수지식)

25.자연어처리 역사(RNN>Transformer)

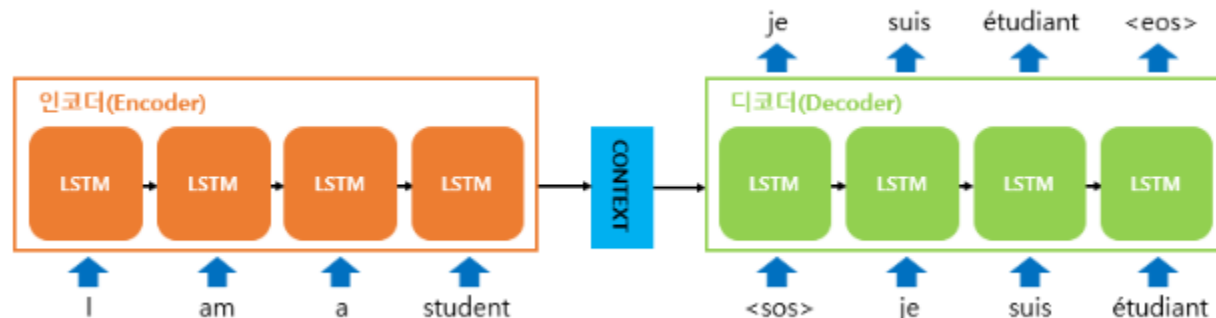
3. Seq2Seq (Encoder-Decoder)

Encoder가 전체 입력 시퀀스를 벡터(context)로 압축
Decoder는 그 벡터를 보고 출력 시퀀스를 생성 (예: 번역)

문제점

고정된 context 벡터에 전체 정보를 압축하려다 보니, 긴 문장에서는 성능 급감
출력이 모든 단어에 대해 동일한 context만 참고

개선 방향 → Attention 도입



I. Deep learning(선수지식)

25.자연어처리 역사(RNN>Transformer)

4. Seq2Seq + Attention

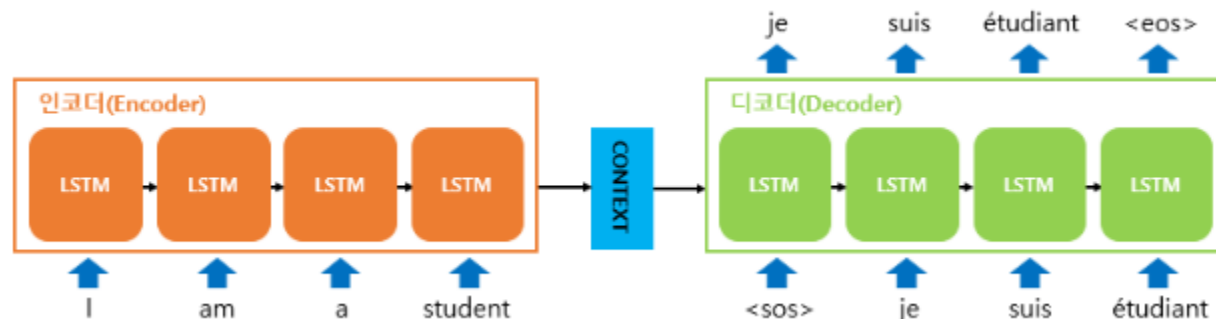
Decoder가 출력 단어를 예측할 때 입력 단어 전체에 다른 가중치를 부여
Q = Decoder 상태, K/V = Encoder 상태 → 중요 단어에 집중 가능

문제점

여전히 순차 처리 기반 (RNN 구조) → 느림

학습 및 병렬화가 제한적

개선 방향 → Self-Attention만 사용하는 구조 개발 → Transformer



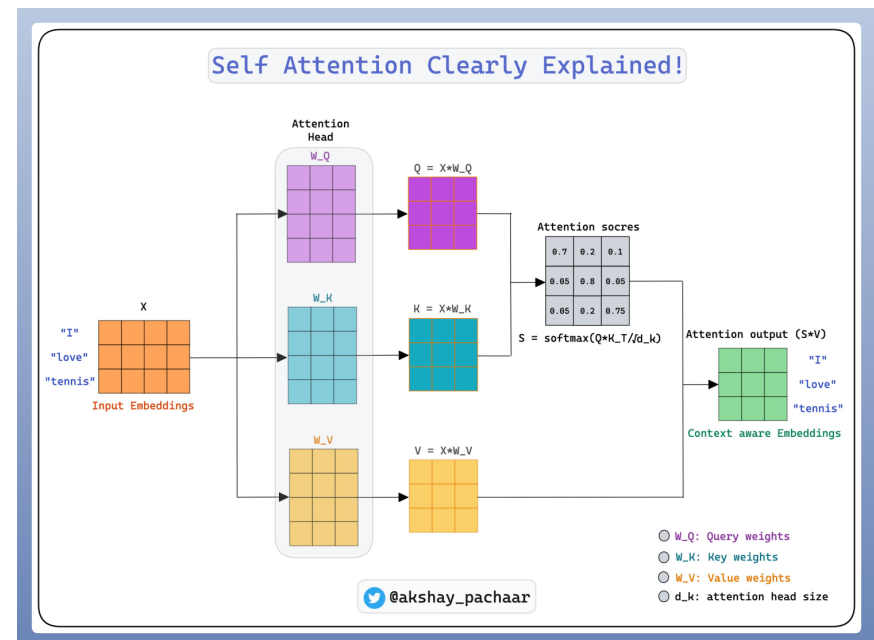
I. Deep learning(선수지식)

25.자연어처리 역사(RNN>Transformer)

5. Transformer (self attention)

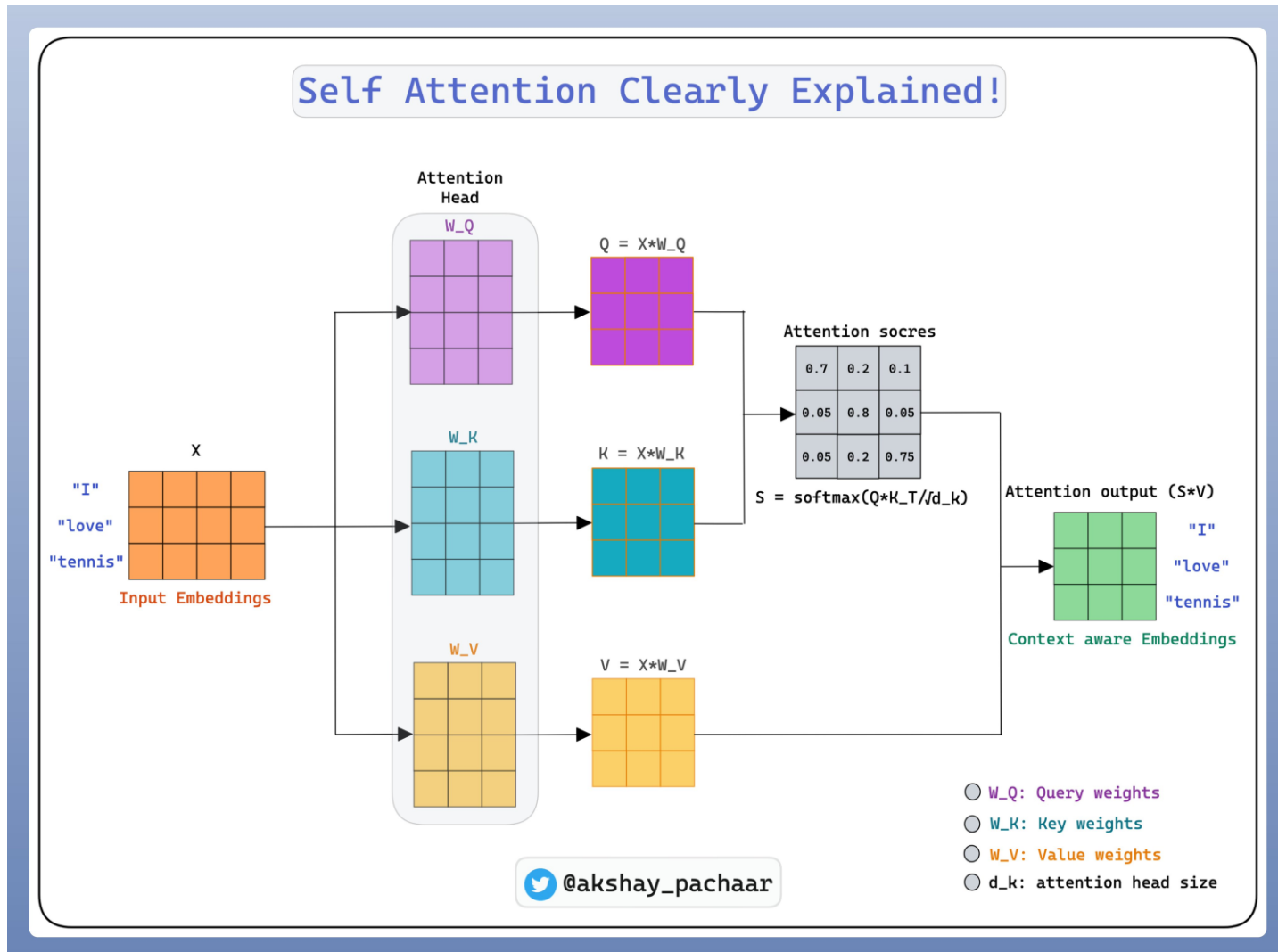
Encoder/Decoder 모두 Self-Attention + Feedforward 구성
Positional Encoding으로 순서 정보 보완

장점
병렬 처리 가능 → 학습 속도 비약적 증가
문장 내 모든 단어 간 관계를 한 번에 계산
긴 문장에서도 성능 우수



I. Deep learning(선수지식)

25.자연어처리 역사(RNN>Transformer)



I. Deep learning(선수지식)

26. Seq2Seq(Sequence-to-Sequence) vs Attention

Attention은 Seq2Seq 모델의 성능을 보완하기 위해 추가된 메커니즘

Seq2Seq 구조

입력 문장 → Encoder → 고정 벡터 context → Decoder → 출력 문장

인코더는 전체 입력을 하나의 벡터(context)로 압축

디코더는 이 context만을 보고 모든 단어를 생성함

예: "나는 학교에 간다" → "I go to school"

문제점: 긴 문장일수록 context 벡터 하나에 모든 정보를 담기 어려움

구분	Seq2Seq	Seq2Seq + Attention
입력 정보 처리	고정된 벡터로 압축	매 시점마다 입력 전체를 재조합
문장 길이 처리	길어질수록 성능 저하	긴 문장에서도 중요한 단어에 집중 가능
병렬화	느림 (RNN 기반)	Attention은 빠르게 병렬화 가능 (특히 Transformer)

I. Deep learning(선수지식)

26. Seq2Seq(Sequence-to-Sequence) vs Attention

항목	Seq2Seq	Attention
목적	입력 시퀀스를 → 출력 시퀀스로 변환	출력의 각 단어가 입력의 어떤 부분을 얼마나 중요하게 여길지 계산
구조	Encoder + Decoder	Decoder에 추가되는 연산 모듈
핵심 구성	RNN / LSTM 기반 인코더-디코더	Query, Key, Value → 가중합
한계	입력을 고정된 벡터로만 요약 → 정보 손실	입력 전체에 가변적 주의 집중 가능

Attention 추가된 Seq2Seq 구조

입력 문장 → Encoder → 모든 hidden state 유지

→ Decoder가 매 스텝마다 입력 단어들 전체를 다르게 가중합해서 사용

디코더는 매 시점마다 입력 전체를 다르게 바라보고

Q (디코더 state), K/V (인코더 hidden states)로 attention 계산
즉, “출력의 각 단어가 입력의 어디에 집중해야 하는지” 판단

I. Deep learning(선수지식)

26. Seq2Seq(Sequence-to-Sequence) vs Attention

입력: “나는 학교에 간다”

출력: “I go to school”

school을 번역할 때는 “학교”에 집중해야 함 → 이걸 계산해주는 게 Attention

기존 Seq2Seq은 “나는 학교에 간다” 전체를 벡터 하나로 보고 “I go to school”을 생성하지만, Attention이 추가되면 “school”을 생성할 때 “학교” 쪽에 더 집중하게 됩니다.

I. Deep learning(선수지식)

27. Attention vs Self-Attention

Attention:

무엇?: 두 개의 서로 다른 시퀀스(예: 영어 문장 → 프랑스어 문장) 간 관계를 학습.

사용 예: 기계 번역에서 소스 문장(입력)과 타겟 문장(출력)의 단어 관계를 파악.

특징: Query는 타겟에서, Key/Value는 소스에서 가져옴. 주로 인코더-디코더 구조에서 사용.

비유: "영어 문장에서 어떤 단어가 프랑스어 문장에 영향을 미칠까?"를 찾는 과정.

Self-Attention:

무엇?: 같은 시퀀스 내 단어들 간의 관계를 학습.

사용 예: "고양이가 나무 위에 있다"에서 "고양이"와 "나무"의 관계를 이해.

특징: Query, Key, Value 모두 같은 시퀀스에서 생성. 문맥 이해에 강력.

비유: "이 문장에서 단어들이 서로 어떻게 연결될까?"를 알아내는 과정.

I. Deep learning(선수지식)

27. Attention vs Self-Attention

Self-Attention의 등장 배경

기존의 문제점: RNN 기반 모델의 한계

- 순차 처리만 가능: 병렬화 불가능 → 학습 속도 느림
- 장기 의존성 문제: 앞뒤 단어가 멀리 떨어지면 관계를 반영하기 어려움
- 정보 손실: 고정된 길이의 벡터로 문장을 요약할 때 정보가 잘림

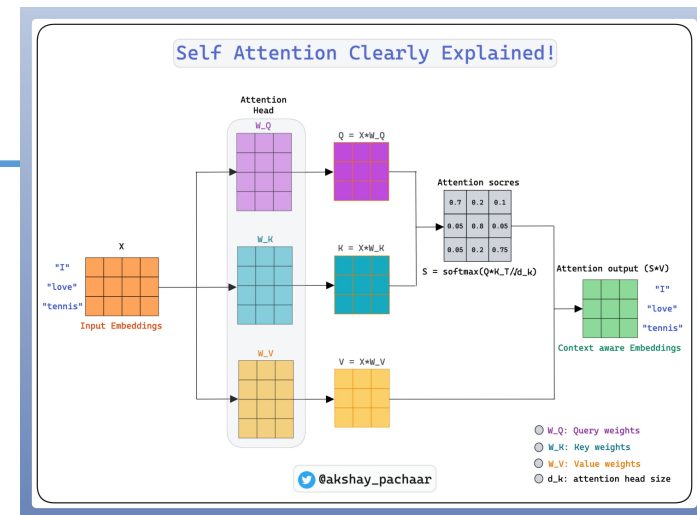
해결책:

Self-Attention의 등장

모든 단어가 서로를 바라볼 수 있게 함

즉, 문장 안의 모든 단어가 Query, Key, Value가 되어 모든 단어쌍의 관계를 계산

병렬 계산 가능 + 긴 거리의 의존 관계도 반영 가능



I. Deep learning(선수지식)

28. Q와 K의 내적이 필요한 이유

1. 내적(dot product)은 유사도(similarity)를 측정하는 간단한 방법

벡터 간 내적은 방향이 얼마나 비슷한가를 나타내는 값

두 벡터가 비슷한 방향을 가리키면 내적은 크고 양수

두 벡터가 서로 수직이면 내적은 0

두 벡터가 반대 방향이면 내적은 음수

즉, $Q \cdot K^T$ 가 클수록 → Query가 Key와 더 연관되어 있다는 뜻!

$$Q \cdot K = \|Q\| \cdot \|K\| \cdot \cos(\theta)$$

"The cat sat on the mat."

$Q = \text{"cat"}$

$K = [\text{"the"}, \text{"cat"}, \text{"sat"}, \text{"on"}, \text{"the"}, \text{"mat"}]$

$Q \cdot K^T$ 는 "cat이 문장 내 어느 단어에 집중할까?"를 계산

만약 Q 와 K ("mat")의 내적이 크다면 → "cat과 mat은 연관 있다"고 판단하여 attention weight가 높아지고, 그 단어에 더 집중합니다.

I. Deep learning(선수지식)

29. 자연어 처리 딥러닝을 위한 필수 개념

토큰화 (Tokenization)

문장을 단어, 문자, subword 등의 작은 단위로 나누는 과정예: "I love NLP" → ["I", "love", "NLP"]

정수 인코딩 (Integer Encoding)

텍스트 토큰을 숫자로 매핑하는 과정 (ex: 단어 → 고유 ID) 어휘 사전 (Vocabulary) 말뭉치에서 사용되는 모든 토큰의 집합 (고유 단어 목록)

패딩 (Padding)

서로 다른 길이의 문장을 동일한 길이로 맞추기 위한 0 등의 추가 작업예: [5, 2] → [5, 2, 0]

임베딩 (Embedding)

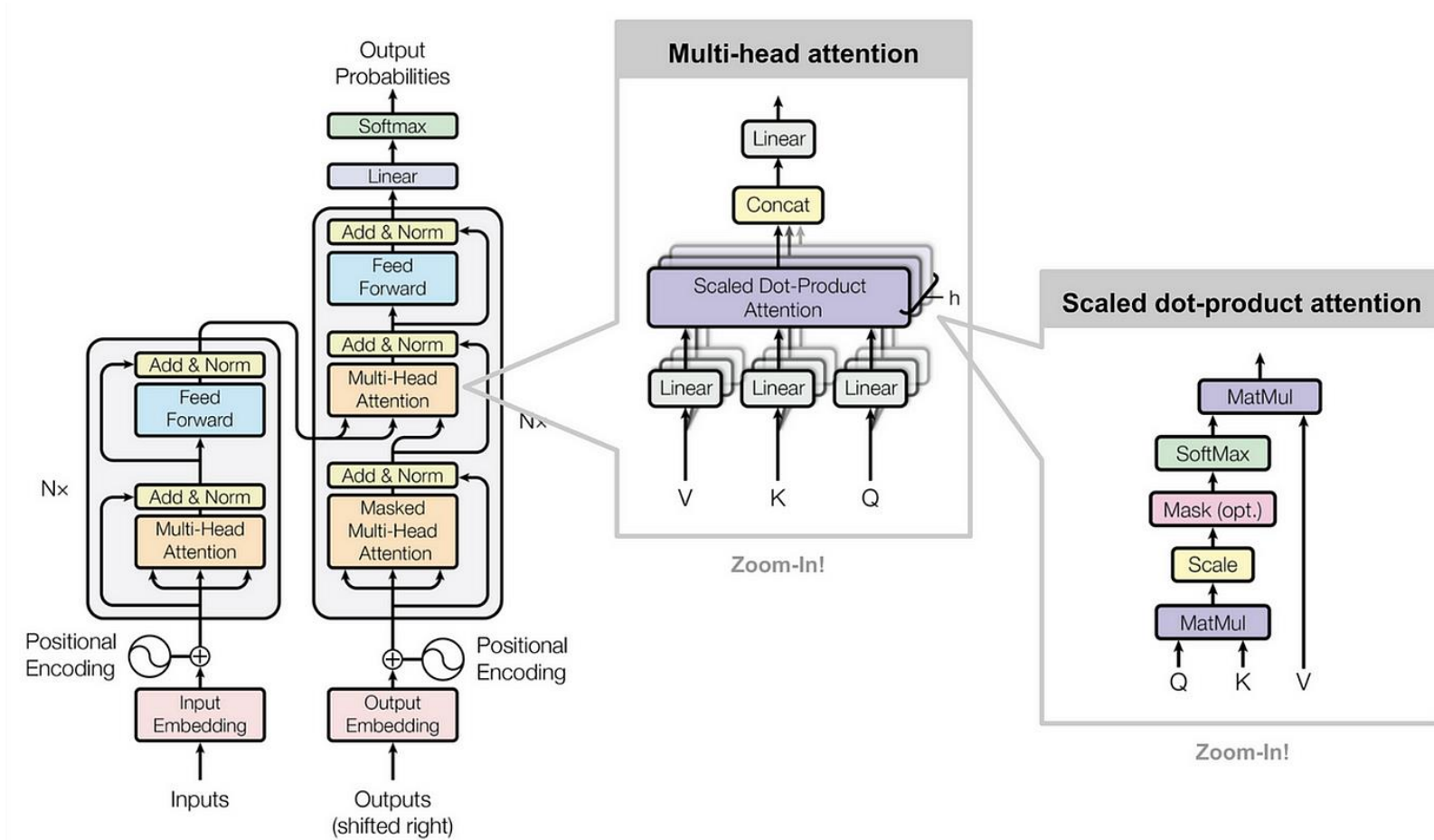
어를 고차원에서 저차원 연속 벡터로 표현하는 방식(ex: Word2Vec, GloVe, FastText, Embedding Layer)

임베딩 벡터 (Embedding Vector)

단어를 의미 공간에서 나타낸 벡터, 유사어 간 거리가 가까움

I. Deep learning(선수지식)

30. transformer를 이해하기 위한 용어



I. Deep learning(선수지식)

30. transformer를 이해하기 위한 용어

용어	설명
Token	문장을 잘게 쪼갠 단위 (단어, subword 등)
Embedding	Token을 벡터로 변환하는 층
Positional Encoding	위치 정보를 벡터에 추가하는 방식 (Transformer는 순서를 직접 알 수 없기 때문)
Self-Attention	입력 시퀀스 내 단어들이 서로에게 얼마만큼 집중할지 계산
Multi-Head Attention	여러 개의 attention head를 통해 다양한 관계를 동시에 학습
Query (Q)	지금 단어가 어떤 정보를 찾고 싶은지
Key (K)	각 단어가 가진 특징 또는 정체성
Value (V)	각 단어가 가진 실제 정보
Attention Score	$Q \cdot K^T$ 로 계산한 유사도 (얼마나 집중할지 결정)
Softmax	Attention Score를 확률처럼 정규화
Scaled Dot-Product Attention	$Q \cdot K^T$ 결과를 $\sqrt{d_k}$ 로 나누어 안정성 향상
Add & Norm	Residual connection 후 Layer Normalization
Feedforward Layer	Attention 뒤에 붙는 완전 연결층 (비선형 함수 포함)
Layer Normalization	입력 벡터를 정규화하여 학습 안정화
Residual Connection	이전 값을 더해 정보 손실 방지
Encoder Block	Self-Attention + Feedforward Layer로 구성된 블록
Decoder Block	Masked Self-Attention + Encoder-Decoder Attention + Feedforward
Masked Attention	미래 단어를 가리지 않도록 마스킹 (훈련 시 필수)
Transformer Encoder	전체 입력을 인코딩하는 부분 (BERT는 이 구조만 씀)
Transformer Decoder	인코딩된 정보를 바탕으로 출력 생성 (GPT는 이 구조 기반)